PawMatch: Animal Adoption and Maintenance Tool

OCR A Level Computer Science (H446) Programming Project

Sienna Sisodia

Contents

1 Analysis

2 Design

3 Development (iterative development and testing to inform development)

4 Testing to inform evaluation

5 Evaluation

Appendix A AI use log

Appendix B References

Model version. This copy keeps Sienna's analysis and design text as written and fills every gap with work produced by an AI (Claude Opus 5.5, Anthropic) on 23 October 2026 from her analysis and design, as a comparison. It is not her work and must not be handed in as hers (JCQ AI rules). Each addition is marked 'Model version addition'. The code it describes is in the rebuild/ folder.

Paw Match

By

Sienna Sisodia

Problem definition ✦ Stakeholders

According to BBC On average, the RSPCA received 374 reports of animal cruelty every day in June to August 2024. The Royal Society for the Prevention of Cruelty to Animals (RSPCA) received 34,401 cruelty reports between June and August 2024, up from 25,887 in the same period the year before (BBC, 2024). These statistics show us how the number of mistreated animals is increasing each year. I want to help reduce the likelihood of an animal receiving mistreatment from its owner.

My client is a friend at St Dominics called Matilde, who has a part time job looking after animals when their owners are away. She looks after a range of different animals from dogs and cats to chickens. We have various conversations about her work where she has made me aware how common it is for animals to be in unsafe environments, where signs of neglect are seen from the owners.

I aim to make an adoption management system which is for one adoption agency, which will allow the staff to manage the animals up for adoption and their personal details and will allow any potential rescuers to see if they are suitable to adopt an animal.

Adoption agency staff are a key stakeholder for this project. This application will have a private part using an admin login for staff and requires an interface to allow them to interact with this software. This will contain details of all the animals, and

it will allow the staff to delete, modify, edit and add different animals. This system should reduce the amount of manual administration required and allows for a consistent form of assessing potential applicants.

Another key stakeholder is potential adopters who are the primary users of this system. Their experience with animals may vary significantly, ranging from first time pet owners to experienced animal carers. They will be using the public side accessed by a user's own login details stored within a large database, with hashing in order to keep the passwords safe if the database is breached. My system will also require a simple and easy to use interface allowing them to browse available animals, view information about each animal and complete an adoption questionnaire. This system will gather user information which will be processed by a compatibility scoring algorithm which will rank the right animals according to their suitability.

Another possible stakeholder is the animals. Although animals do not directly interact with the system, they are an important stakeholder as they are affected by the outcome of adoption decisions. This system's key aim is to improve animal welfare by encouraging more informed adoption choices and reducing likelihood of animals being placed in unsuitable homes.

Due to my client's experience caring for a variety of animals, she is skilled and knowledgeable about the environments required for an animal to live happily and safely and has asked me to incorporate the compatibility matcher with questions and a set criteria to examine whether they are suitable. Examples of the set criteria may include: the amount of available living space, access to a garden, experience with animals, income and more. Her feedback will be used throughout development to ensure the system meets the needs of animal welfare professionals.

Although many adoption websites allow users to browse animals, they often rely on their own judgement whether or not they are fit to adopt. This may lead to adoptions where the owner does not fully understand the commitment required which may lead to neglect. In some cases, unsuitable adoptions may result in animals being returned to shelters, creating additional pressure on adoption agencies and causing unnecessary stress for animals involved. This application should provide guidance whether a particular animal is compatible with their lifestyle, living arrangements, financial situation or previous experience. Additionally, current adoption agencies may use manual assessments and application forms by staff to which this may be time consuming and influenced by subjective judgement. My system will differ as it provides a consistent method of evaluating applicants through a scoring algorithm.

Use of Computational Methods

The most suitable solution for a problem like this is to create a computer based adoption matching system. This is because the problem includes storing animal data, adopter information, comparing different factors and producing recommendations. A manual system may require staff to compare each adopter with each animal individually, which could take significantly

longer and will require more effort. Using a manual way of storing data may also lead to inconsistencies, missing information and errors, whereas using a database allows data to be searched efficiently and stored consistently. A computational solution allows this problem to be solved more efficiently by using algorithms, data validation, automated scoring and ensuring consistent data handling

Thinking Abstractly

The process of abstraction is removing unnecessary details in order to simplify the approach of a problem, making it easier to understand and solve. Ignoring insignificant details will reduce the amount of unnecessary coding and make the system easier to maintain. Within my system, it will be used in order to focus specifically on matching an adopter with an animal.

For example, the system does not need to store every detail about an adopter's personal life. It will only need to collect information that affects animal suitability such as the living space, garden access, experience with animals. Similarly, with the animal profiles it will only store useful information. It won't include its nicknames, or minor habits or favourite toy as this is not as important for the matching system.

This will make the system easier to design and implement as irrelevant details are ignored. This will also make the system more efficient by focusing on the most important factors that affect suitability.

Thinking Ahead

During the development of this system, I will need to think ahead in order to identify the requirements of my user and relate to the issues they might face or be facing.

I have to anticipate how users would like their information to be presented in order to attract them to adopting an animal, it must also be fully accessible and easy to use.

I will also need to plan how the data will be stored before implementation. For example, animal records and adopter records must be stored persistently so information is not lost when the program closes. I will also need to consider validation as users should not be able to leave empty fields or enter invalid data. This will help ensure that the matching algorithm is using accurate and correct information.

I will also need to think about the structures and designs that can be reused on different pages before I start developing my project. I also have to consider the technologies I will use to develop my system, as there are many different technologies I can use with each their own pros and cons.

Another key part of my system to which I will need to think ahead is the compatibility scoring system. This system should not simply recommend animals based on experience or breed. Instead, it should compare multiple factors such as living space, experience, time available, children and animal requirements.

Thinking Concurrently

Processing multiple tasks at once will reduce the processing time of my app, reduce waiting time for my user and increase the overall efficiency of my system. When the user completes the adoption questionnaire, the system may need to retrieve animal records, compare the adopter answers with requirements, calculate compatibility scores and display results.

It is also important within the login page, where the credentials that are entered need to be validated and sanitised to prevent any injection attacks, whilst finding user profile. After this stage is cleared, multiple elements will need to be loaded at the same time.

The system also needs to be able to handle different users carrying out tasks at the same time. For example, an adopter could be viewing suitable animal matches while an admin user updates or adds an animal profile. I will need to design the system, so these processes are independent, and the data is able to remain consistent.

Concurrency may not be a main focus of my project, but it will allow me to design a system which is responsive and efficient as more animal records and adopter applications are added.

Thinking Procedurally with Decomposition

Decomposition is the process of splitting a large process into small more manageable sections. This will make the system easier to design, implement, test and improve. Instead of trying to build the whole system at once, I can separate it into different components.

The main components which build up my system will include animal records with their different cards, the adopter questionnaire with the matching algorithm, compatibility score calculation, results page, validation and the admin control page. The animal records section will store details about each animal and the adopter's questionnaire collects information about users home and lifestyle. The matching algorithm will compare adopter answers with each animal's requirements and calculate compatibility score.

I will also split the application into the front end and back end sections. The front end is what the user or the admin user will experience, which will have the login forms and present the different features of the application. The backend is where all the logic will be processed and will include a relational database for secure storage of user data.

By using decomposition on my system, it will allow each section to be developed and tested separately. For example, I can first test whether the animal records are being stored and added correctly, and if all the information is shown in the correct format. I can then test whether the adopter questionnaire collects all the information required and validates the inputs. Then I can test whether the matching algorithm calculates scores correctly. This way of testing will make errors easier to identify and fix.

Procedural thinking will allow me to further separate each function of front and backend and allow me to design my application using modules of reusable and readable code. The different subprograms will be kept in their own areas so I can work on them without affecting other areas. I will use this method in both the front and back ends of this project.

Thinking Logically

My program needs to be designed to make it simple to use but also logical to follow. This is important as users are guided through the adoption process clearly, from logging in to receiving suitable animal recommendations.

My system should hopefully follow a sequential process, for the user to view:

1. User creates an account or logs in using username and password
2. Once logins, users directed to main page, with the animals
3. The adopter will then complete a questionnaire
4. A compatibility score is shown for animals and ranked from suitable to least suitable and the user can browse.

For the admin to view:

1. Admin user will login in using their username and passwords
2. The user will be directed to the admin control page
3. The animal is able to add, edit or delete records similar to a CRUD interface.
4. The system will validate the animal record to make sure required fields are completed and data in correct format and if it is valid will be saved to the database.
5. The updated animal information is then available for adopters to view and matching algorithms to use.

The system will also use logical decision making. For example, if the username or password is incorrect, the user cannot access their account. If required questionnaires fields are left blank, the system should display an error message. In the compatibility scoring algorithm, if an animal cannot live with children and the adopter has children then the compatibility score should decrease. If an animal requires a garden and the adopter has one, then the score should increase.

This also shows that my system will use sequencing, selection, validation, iteration and sorting. It is not simply just displaying the animal profiles but processing data logically to produce suitable recommendations.

Research - Approach and Objectives

I will take a structured approach to my research by using both primary and secondary research in order to understand the problem from different perspectives and ensure my final system meets user needs and expectations:

1. I will inspect existing adoption websites, analyse their design, structure, features and adoption process, while looking at features I may want to include or avoid.
2. I will conduct an interview with my client (Matilde) to get a better understanding of what she wants within this project and reflect this in the design of my system.
3. I will carry out a stakeholder survey to find out more about their preferences. I will use google forms and this will help me see if my problem is important and worth solving and will help me in developing parts of the system such as the questionnaire.
4. I will assess the different technologies that can be used to produce my application.
5. I will identify software and hardware requirements for my project to run effectively.

My research objectives are:

1. Understand current problems with adoption systems.
2. Identify which information needs to be stored about each animal
3. Identify which information needs to be collected from adopters
4. Find out what factors are most important when matching an adopter to an animal
5. Understand and take inspiration from existing adoption websites
6. Understand user expectations for the interface
7. Use the research findings to help decide the final features and requirements of my system

Research of Competitors

Example 1 - Dogs Trust

Dogs trust 'offer practical solutions and support to help owners and dogs stay together wherever possible' (Dogs Trust, 2026). They allow users to complete an application form which describes their home, lifestyle and preferences. Staff at the adoption home then compare applications with suitable dogs.

Add filters Clear filters Sort

0 filters added +

Newest Listed

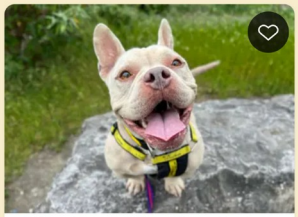


Woody Reserved

Poodle (Standard)

West Calder (Edinburgh)

Male 8 + years Large



Meatball

American Bully Pocket

Cardiff

Male 2 - 5 years Medium

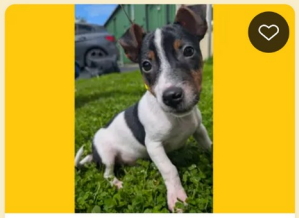


Lenny

Collie (Border)

Ballymena (Northern Ireland)

Male 1 - 2 years Medium



Wunder

Terrier (Jack Russell)

Ballymena (Northern Ireland)

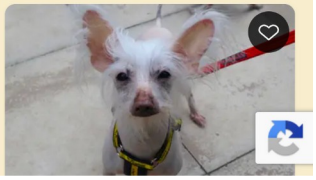
Male Under 6 months Small



How rehoming works

Everything you need to know about adopting a dog from us.





Meatball

I'm looking for my forever home. Could you be my perfect match?

Apply to adopt | Add to favourites

Share my profile

Age	Gender	Location
2 - 5 years	Male	Cardiff (Cardiff)

Ref ID	Size	Breed
3644721	Medium	American Bully Pocket Cross

May live with: Primary school children, Secondary school children

Are you right for Meatball?

Meet Mr Handsome himself, Meatball! He's a 4 year old Pocket bully cross French Bulldog who is looking for his special someone. He will need adopters who are able to build up any time alone really gradually for him as he is used to having company all the time. He can be a little selective with other dogs, so is looking to be the only Prince in his new home, with owners taking his socialisation with other dogs nice and slowly with nice calm walking friends. He's able to share his home with older primary school aged childre, and we cannot wait to see him all settled in his new home.

Is Meatball right for you?

Meatball is still in his prime, at just 4 years old. He is yet to meet anyone he doesn't absolutely love, and has been a delight with all of our staff here. He loves snuggling up in a cosy bed after his walks, and thinks having a fuss is way more exciting than playing with toys at the moment.

How our rehoming process works

1. **Create an account and fill out our application form**

   In our form you can tell us all about your home, your lifestyle and the kind of dogs you're interested in. You won't be applying for a specific dog, but you can add favourites to give us an idea of the dogs you like. We'll use this information to find a great match for you.

   Apply to adopt

2. **Choose a rehoming centre**

   We'll also ask you to select a rehoming centre. The team at this centre will look after your application and assess you against all suitable dogs in their care. This doesn't have to be your nearest centre, but you will need to travel there within a few days once we've found...

   Show details

3. **We'll review your application**

   We'll review your application against the dogs in our care at your chosen centre. If we think we have the right dog for you, we'll be in touch as soon as possible. If we don't hear back from you, we will close your application for now, and you can reapply when you're ready.

4. **We'll keep your application open for three months and keep looking for a match**

   If we don't have a dog for you right now, we'll keep your application open for three months and keep you in mind for any new dogs that arrive. If within this time you see another dog you are interested in, you won't need to apply again but you can amend your application by...

   Show details

5. **When we find a match, we'll invite you to meet them**

   If we've found a dog who seems right for you, we'll invite you to come and meet them at the rehoming centre.

   Some of the dogs in our care will need to meet potential owners several times to get to know one another. This lets us see you're compatible and gives you...

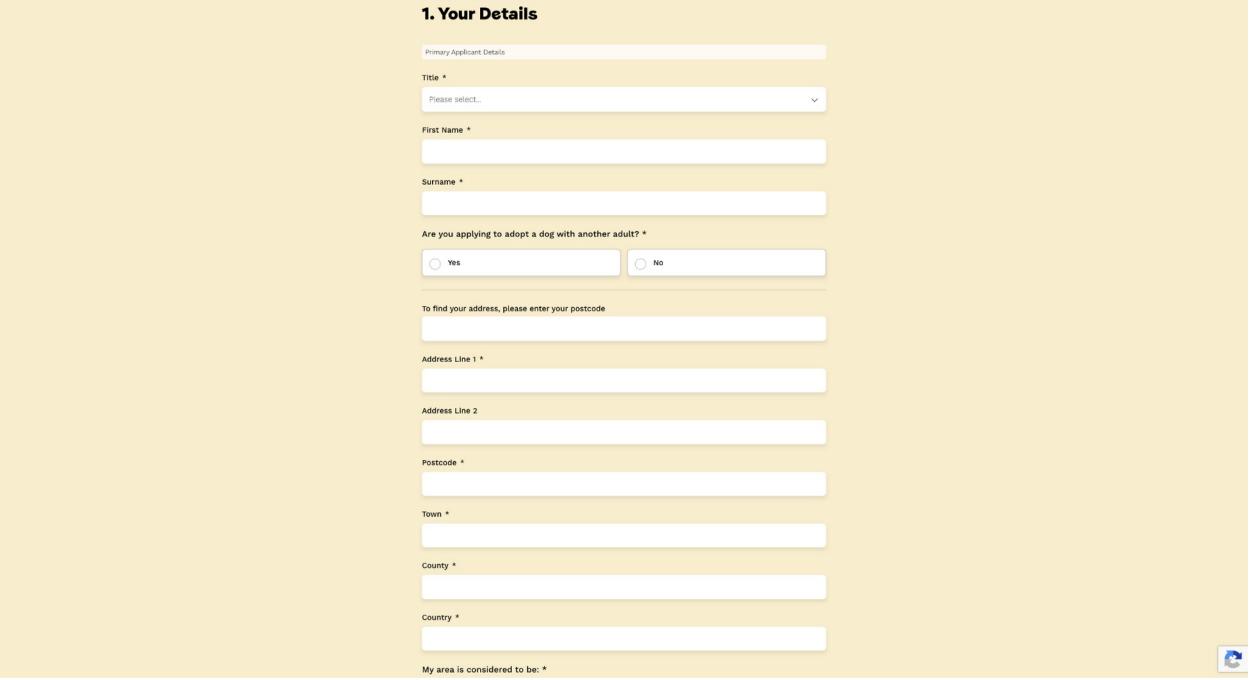
   Show details

6. **If we haven't found the right match, we'll continue the search together**

   If we haven't found the right dog for you within three months, we'll let you know your application is closed. We'll invite you to apply again so we have up-to-date information about you, and we'll keep looking.

7. **We'll support you to embark on a new life with your dog**

   When we've matched you with a dog, we'll help you welcome them to your home. After



Positives	Negatives
Dog profiles display clear information about the dog such as the age, gender, breed, size, location and whether the dog can live with children.	The system only supports dogs so people interested in different animals must use another service.
The profile also includes the dog's personality, behaviour and required home environment. This helps the user with making a decision on how the dog acts rather than from looks.	Users are not immediately told how well they match. Especially as suitability is assessed manually, so users have to wait.
Users only have to complete the application once rather than for each dog	The system does not explain which individual requirements the adopter meets or fails to meet
Users can add up to four favourite dogs.	The user must select a particular rehoming centre and be able to travel there when a match is found
Applications remain active for three months, so the user can be considered for dogs that arrive later.	Not every available dog is shown on the website meaning users cannot browse the complete set of possible matches themselves

Positives	Negatives
Dogs are assessed by behavioural and veterinary staff before being matched, making the final decision more credible.	The bright yellow colour may be distracting and make it harder to read.
There is a strong yellow, black and white colour scheme making their branding recognisable.	The large photographs and sections mean that only a small amount of information is visible at once, so users must scroll frequently.
Large photographs are used for each dog to make it more appealing to adopt	Some pages are very long and contain several sections such as the 'About us' page, which may feel overwhelming.
There is plenty of white space between sections, preventing pages from appearing cluttered	The similar appearance of every dog card may make results look repetitive.
Important buttons such as apply to adopt and add to favourites stand out.	On smaller screens, the large images and vertically arranged content could require significantly more scrolling.

IMPACT ON MY SOLUTION

Dogs trust has influenced me to collect information about the adopter's home, lifestyle and preferences only once and reuse it when comparing with different animals. I will also create similar animal cards which have detailed information about each animal. It will also contain another section with personality, behaviour and home requirements.

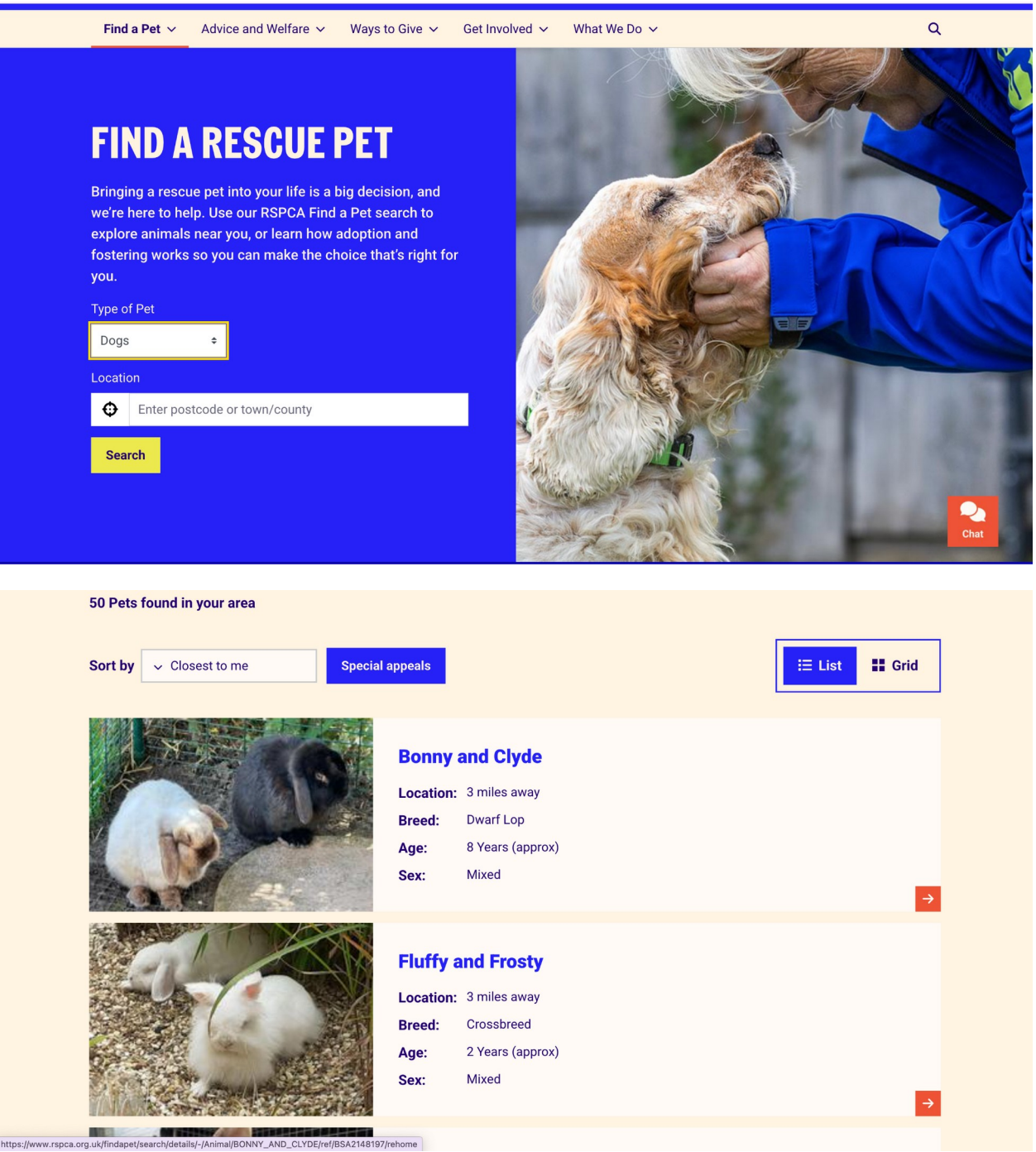
However, my system will differ as it allows for many different species and will automatically give partial feedback whether they are suitable but ultimately leave the final decision to staff.

They have also influenced me to use a consistent colour scheme, clear headings with organised animal cards. I will make buttons easy to identify.

I will avoid bright colours, long pages and make mine less overwhelming and simple to use.

Example 2 - RSPCA

The RPSCA Find a pet website allows a user to search for several different types of animals. Its pages use photographs, clear heading and separate content sections to present animal information.



Back to results





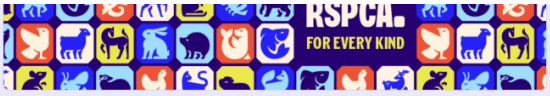




Bonny and Clyde

Location:	3 miles away
Breed:	Dwarf Lop
Age:	8 Years (approx)
Sex:	Mixed
Colour:	Mixed
Ref no:	2380 and 2381
Indoor only:	No

Add to my favourites



Rabbit Adoption Application Form

Application to adopt a Rabbit.

To start your application we need to know what you're looking for in a Rabbit and learn about your home and your lifestyle.

sienna.sisodia1@gmail.com Switch account

Not shared

* Indicates required question

Have you seen a Rabbit or Pair at RSPCA North West London and South Hertfordshire that you're interested in? *

- [ ] Yes
- [ ] No

If yes, please tell us the animals name.

Your answer

Next Clear form

Positives	Negatives
Large animal photographs make the website visually appealing and immediately attract users' attention.	Applications are all reviewed manually by staff. The user does not receive an immediate result and should normally wait up to 48hours for a response. (Although this may vary between branches)
The colour scheme fonts and button styles remain consistent which allows the website to appear professional	The website contains many navigation options as it covers more than adoption, which may overwhelm some users. The number of filters and controls may make the search area appear crowded.
They support a wide range of species, rabbits, hamsters, guinea pigs. This is more similar to what I want within my system as it doesn't just support cats and dogs.	Some sections which are important and not important have a similar appearance which may make it hard to show what is important and allow it to stand out.
Each animal has an individual profile explaining needs, allowing adopters to consider suitability before applying. Applications also are reviewed by trained staff meaning factors for a computer to assess still can be considered.	Some important details are only available after opening the full animal profile.
They also have a feature for support after adoption, including behaviour services and pet care advice. This also could reduce chances of adoption failing after the animal entered a new home.	It may be hard to objectively analyse if they are suitable for the animal due to the lack of numeracy.
RSPCA may suggest alternative animals when the selected animal is unsuitable.	Procedures and costs may differ between RSPCA centres and branches, which does not allow for consistency which may confuse the user.
Search controls and filters are clearly labelled helping users understand what information they need to enter.	The published process requires applications to be started from an animal's profile. This may cause users to repeat parts of the process when interested in another animal.
Information is divided into sections with headings, helping users distinguish between different types of contexts.	Some centres only allow people to view animals by appointment after choosing a pet, this reduces the ability to visit freely and compare animals in person before applying.

IMPACT ON MY SOLUTION

The RSPCA has influenced me to support a wide range of animal species and organise them into clear categories. I may include simple species categories, but the main method of finding suitable animals will be the compatibility questionnaire and ranked recommendations. It has also inspired me with its layout and visual design. I like its consistent colour scheme with clear headings.

However, unlike the RSPCA, my system will calculate a numerical suitability score, explaining which requirements have and have not been met. It will also use a simpler navigation system focused solely on adoption, preventing users from becoming overwhelmed.

Example 3 - Petfinder

Pet finder is my closest competitor to my proposed system as it provides a short adopter questionnaire, and the system displays compatible pets. This website also includes a “See how you match” option. Animal listings contain information of the dogs such as age, housetrained ability and whether it can live with other animals. Users can also favourite animals.

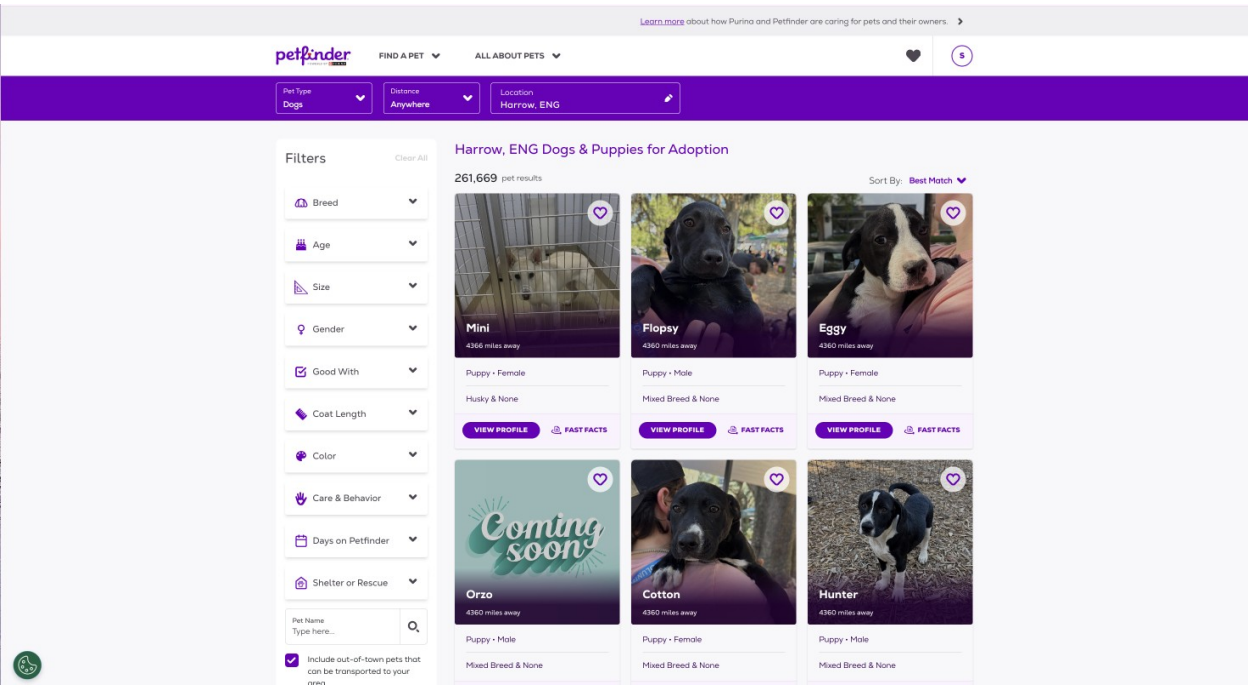
100%

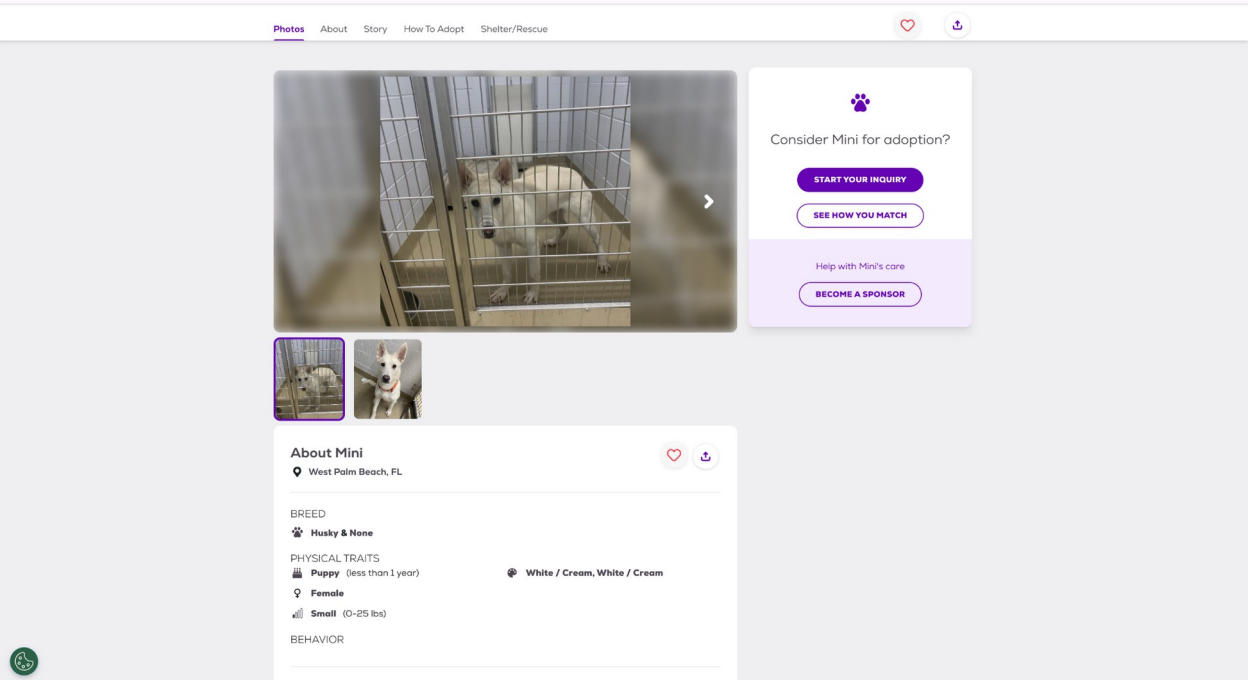
I'd like to adopt a	Dog
I am looking to adopt for	Myself
I am a ___ pet owner	First-time
I currently have	No dog(s) or cat(s)
My home has	A yard
There are ___ as part of my HOA or lease	Dog restrictions
My HOA or lease will not allow a	(No breed restrictions)
My HOA or lease will not allow a dog that is	(No size restriction)
Age preference	A puppy
I would like to adopt a	No preference
Size preference	(No size preference)
My dog's activity level should be	Very active
My dog must be	(No preference)
I am ___ to adopting a dog with special needs	Not open
A breed that I really like is	No preference

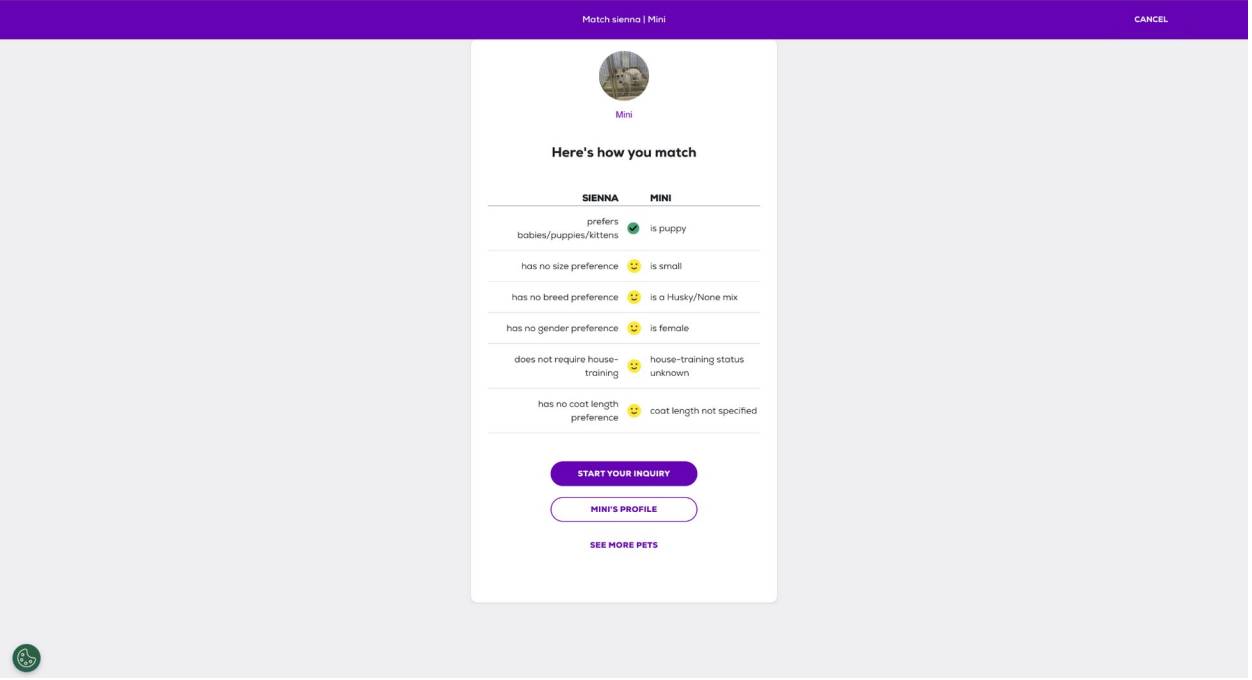
Ready to Meet Your New Best Friend?

Based on your quiz answers, we think one of these pets could be your perfect match!

SEE YOUR MATCHES







Positives	Negatives
The search filters allow users to quickly narrow down the large number of available animals	The large number of filters and listings could overwhelm some users.
Animal cards are very clear and display a photo and important information such as the name, age and breed	It may not be as colourful and bright as the other examples.
The animal profiles are detailed and show information about the personality, medical needs and compatibility with children or other pets.	The automated matching appears to focus mainly on cats and dogs rather than equal support for every species.
Users can save animals as favourites so they can return to them later.	A visible suitability percentage is not provided making it harder for users to compare matches
The adopter questionnaire can provide more personalised recommendations then a basic search.	Information can be incomplete or marked as ‘unknown’ as each shelter is responsible for entering its own animal data
Important actions such as ‘start your enquiry’ and ‘see how you match’ are clearly labelled and easy to identify, allowing ease of use for the user.	The rescue organisation may ask the user to complete another official application, causing information to be entered more than once. This also may mean the information provided may be inconsistent as different rescue organisations manage their own listings.

Positives	Negatives
Pet finder brings listings from thousands of different shelters and rescues into one website.	Several actions appear on the same profile, including enquiring, matching and sponsoring which may compete for users' attention.
Data can be saved by creating a login.	Some less important promotional content may distract users from main adoption features.
Bold headings divide profiles into clear sections such as behaviour, health compatibility and adoption information.	As individual shelters manage their own listings, an animal's availability may not be updated immediately. Users might need to contact the shelter to confirm whether an animal is still available which may be an inconvenience for the user.

IMPACT ON MY SOLUTION

Petfinder's system has strongly influenced the design of my system. I am inspired how the survey is divided into clear manageable questions rather than displaying a long form at once. I will use a similar step by step structure with clearly labelled answer options and progress indicator, so the user knows how much of the questionnaire remains. I also like how it compares the adopters' preferences with animal characteristics in a clear table with visual symbols to show if each factor is compatible.

However, my system will provide more detailed feedback by displaying a numerical compatibility score and explaining which animal requirements have and have not been met. This will allow the several animals to be compared more objectively.

Overall Conclusion

My research shows that each competitor contains useful features, but none fully have all the requirements I want for my own system. Considering the design of dogs' trust, it has a recognisable and consistent interface where the information is presented very clearly. Dogs trust has a thorough application process even though it is only for dogs and does not provide immediate feedback. However, the frequent bright yellow may be visually distracting and long pages which can require excessive scrolling.

The RSPCA supports several species and organises animals into clear categories, but users must browse and select an animal before suitability is assessed. Their professional colour scheme makes it very recognisable and trustworthy, which may make people more intrigued to adopt. However, because the website contains services beyond adoption, its navigation can appear crowded and some important information may not stand out. Users will not be given instant or numerical feedback as they browse for an animal then begin the application. Only after staff are able to manually assess the suitability.

Petfinder is the closest competitor to my proposed system as it uses an adopter questionnaire and compares the user's preferences with individual animals. However, it does not provide a clear numerical score making it harder to compare with many animals. Information may also be inconsistent because individual rescue organisations maintain their own animal records.

Client Interview

1. **What factors do people fail to consider before adopting an animal?**

   “I think a lot of people choose animals based on how cute they are without thinking how much work they can be. They often forget to consider things like how much space they have, how much time they can spend with their animal, how much it costs to look after, and whether they have experience caring for that type of animal. Some people also do not think about whether their home is suitable for the animal’s needs.”

2. **What information should be stored about each animal?**

   "Each animal should have basic information such as its name, age, breed, species, sex and a photo of the animal. It can also contain an extra information section such as personality, medical conditions, exercise needs, whether it gets along with children or other pets and special requirements other adopters should know about."

3. **How should the system respond if the user is not suitable for the particular animal?**

   "I don't think the system should completely stop someone from adopting an animal, but it should warn them they might not be a good match. It should explain which requirements they don't meet and recommend other animals. This helps people make better decisions and reduce chances of animals ending up in the wrong home, but also not diverting people from adopting."

4. **Do you think users should be able to apply for adoption directly through the system?**

   "No, I don't think users should be able to apply for adoption directly through the system. Although the system can help identify suitable animals, I still think adoption agencies should still carry out further checks in person. If staff are able to meet the adopter face-to-face they can assess personality, commitment and understanding of responsibility in owning a pet. It should aid the original process not completely take over that decision"

5. **What key tasks should the staff have on their side of the system?**

   "The key features the staff should have is being able to add, edit and remove animal records, update animal information, upload photos and mark animals as available and unavailable for adoption by deleting. The database should also be kept up to date."

6. **What features would make the system easy to use for both staff and adopters?**

   "I think the main thing is that it should be simple and not confusing. People should be able to find what they are looking for quickly without clicking through loads of pages. It should be easy to browse animals and use the questionnaire, and for staff it should be straightforward to update animal information. Clear buttons, a clean layout and simple instructions."

7. **What features should be displayed on the homepage?**

   "I think the homepage should show information about how the adoption process works and a button to complete the questionnaire. The homepage should be welcoming and make it clear what the website is for."

8. **Should users be able to search for animals, if so, what filters should be available?**

   "Yes, I think users should be able to search for animals as it makes it much easier to find a suitable pet. Filter categories could be the type of animal, breed, age, size and activity level, but mainly type of animal and size."

9. **What information should users provide when creating an account?**

"Basic information such as name, email, address, phone number and a password when creating an account. You could also include the age to ensure they are old enough to adopt the animal."

10. Should some factors have a greater impact on the compatibility algorithm than others and if so which factors?

"Yes, I think some factors should matter more than others. Things like available space, allergies and whether there are young children in the house should have a bigger impact. For example, if someone is allergic to cats, they should not be strongly matched with a cat, even if the rest of their answers are suitable. Less serious factors such as preferred age or breed, should not be considered as important as the animal's safety and wellbeing."

11. Do you want the website to look a specific way? Any colours, fonts etc. It would also help if you gave specific examples of apps, you like

"I would like the website to have a cute and friendly look rather than it being too serious. Soft pink and pastel colours would make it feel more welcoming and enjoyable to use. I think the design should be simple but visually appealing, with lots of photos of the animals and clear buttons. The main thing is that it feels approachable and encourages people to engage with the adoption process."

12. Do you have any security concerns regarding the app?

"Yes, I think security is important because the system will store users' accounts and animal records. Passwords should be protected, and only staff should be able to add, edit or delete animal information. Regular users should only be able to browse animals and complete questionnaires, not change anything in the database."

13. Which devices do you want it to work on?

"I want the system to work on both laptop and desktop computers, as these will be mostly used by adoption agency staff."

14. What should happen after a user receives their compatibility results?

"I think the system should show users the results in a clear way and provide them with more information about any animals that are a good match. What happens after that would depend on how the adoption agency wants to use the system."

15. Are there any features that you believe are essential for the success of this system?

"The compatibility scoring system is probably the most important feature as it helps match animals with suitable owners. Additionally, the animal profiles need to be detailed and up to date so the users will have all the information they need before making a decision"

Overall, the client interview provided valuable insight into the function and design requirements for this system. The client identified that many people fail to consider factors such as available space, time commitment, allergies and previous experience before adopting an animal. Therefore, these factors will be incorporated into the compatibility questionnaire and scoring algorithm. The compatibility algorithm should use weighted factors so that important requirements such as allergies, children, other pets and living space have a greater impact than softer preferences such as preferred breed or age. The client also highlighted the importance of maintaining and up to date detailed animal profiles and providing staff with ability to edit, add and remove animals through an administrative interface. For the design, the client wants a simple, easy to use interface with clear navigation, animal photographs and soft pastel colours to make it a friendly interface. The client also is very focused on security ensuring only staff can change details within the database regarding the animals.

Stakeholder Survey

This stakeholder survey received 27 responses, which gave me feedback from a range of potential users.

QUESTIONS:

1. **Have you ever owned a pet before?**
    A. Yes
    B. No

2. **If you did own a pet, did you adopt it from a centre?**

   A. Yes

   B. No

   C. N/A

3. **How would you normally choose a pet?**

   A. Appearance

   B. Personality

   C. Breed

   D. Lifestyle Compatibility

   E. Advice from experts

4. **How important do you think it is to research an animal before adopting?**

   A. Very Important

   B. Important

   C. Neither

   D. Not Important

   E. Not very important

5. **Which factors do you think should be considered when matching an animal to an adopter? (Select many)**

   A. Living Space

   B. Garden Access

   C. Income

   D. Experience with animals

   E. Time available

   F. Presence of Children

   G. Other pets

6. **How important is it that an adoption website is easy to use?**

A. Very important (1)

B. Slightly important (5)

7. Would a compatibility score influence your decision when adopting an animal?

A. Yes

B. No

8. Do you think adoption agencies should use a compatibility scoring system?

A. Yes

B. No

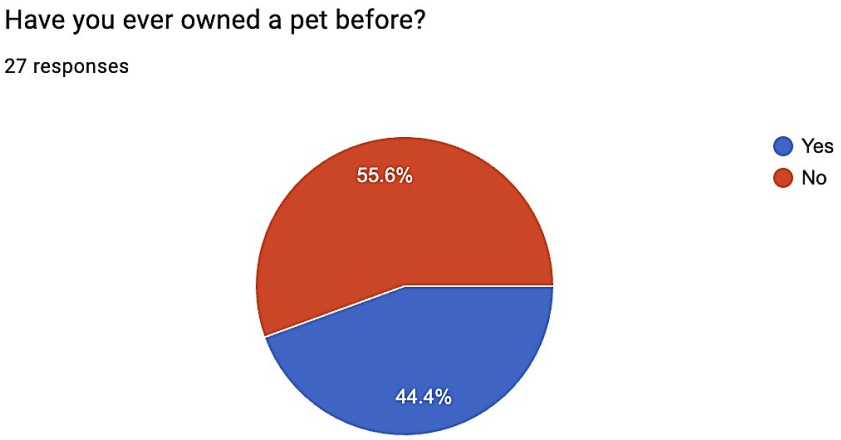
9. How concerned are you about the welfare of adopted animals?

A. Concerned (1)

B. Not Concerned (5)

RESPONSES:

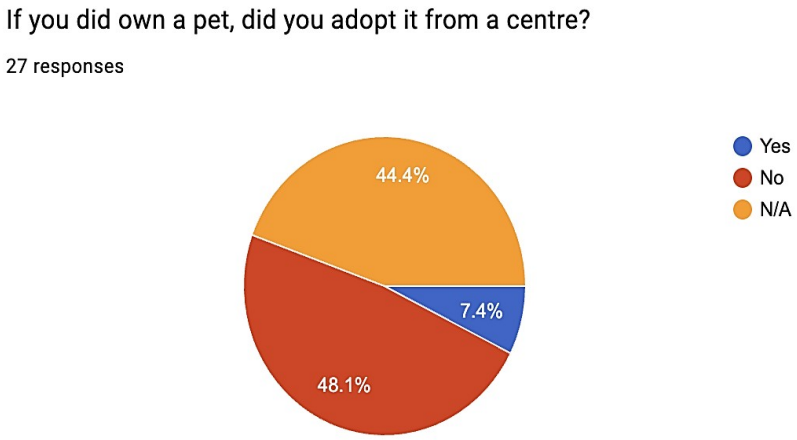
1.



Analysis:

44.4% of respondents said yes, while 55.6% said no. This shows that my survey consists of a mixture of people with and without pet ownership experience. This is useful as my system needs to be able to support experienced and inexperienced adopters. This means the system should not assume that users will already understand animal care or adoption requirements.

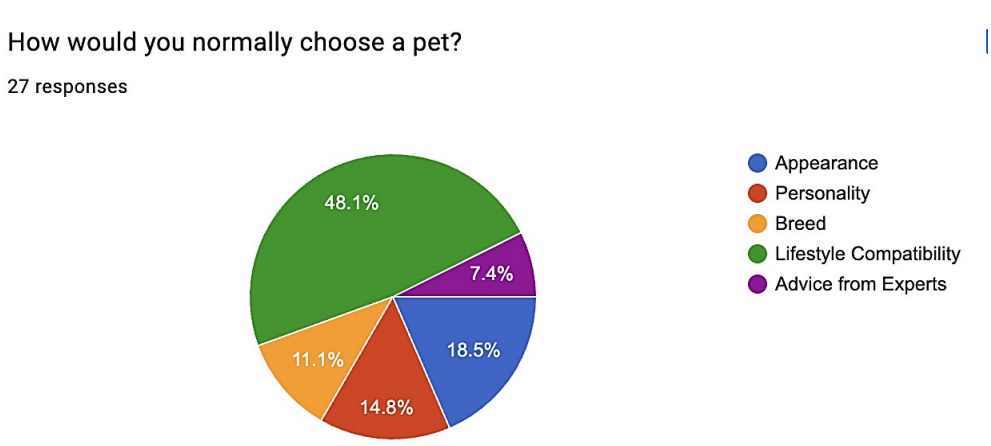
2.



Analysis:

These results show that very few respondents have experience adopting from an adoption centre. This means my solution should clearly explain the adoption process and avoid making it confusing for users who have no experience.

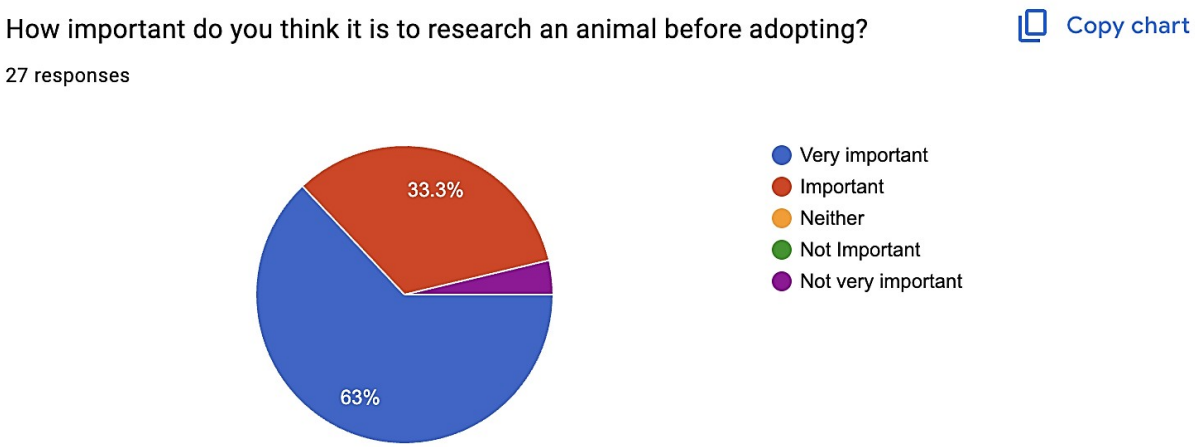
3.



Analysis:

The most common answer was lifestyle compatibility, chosen by 48.1% of respondents. This shows most users already do care about which animal fits their lifestyle rather than just how the animal looks, even though a few care about the aesthetics of the animal. This will allow my system to be suitable for those looking to adopt.

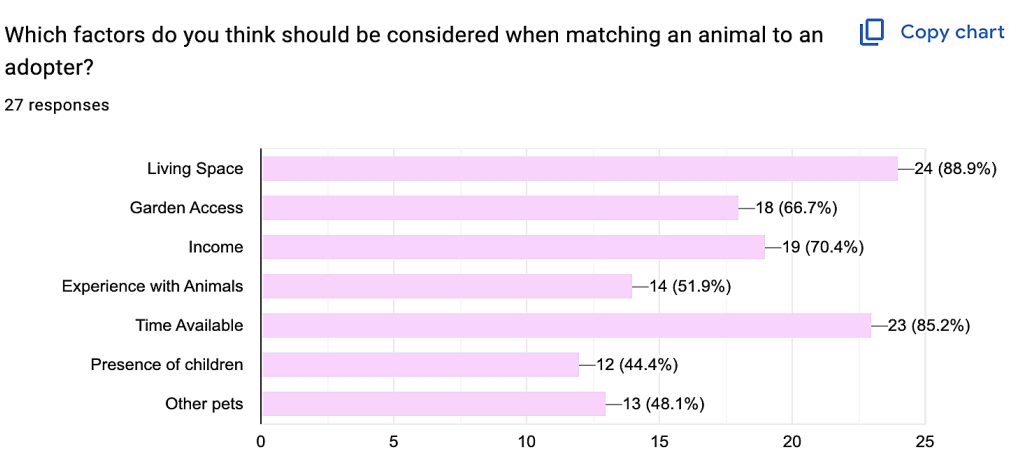
4.



Analysis:

63% of respondents said it was very important, and 33.3% said it was important. This means that 96.3% of respondents agreed that research is important before adopting an animal. This supports my idea that websites should include detailed animal information.

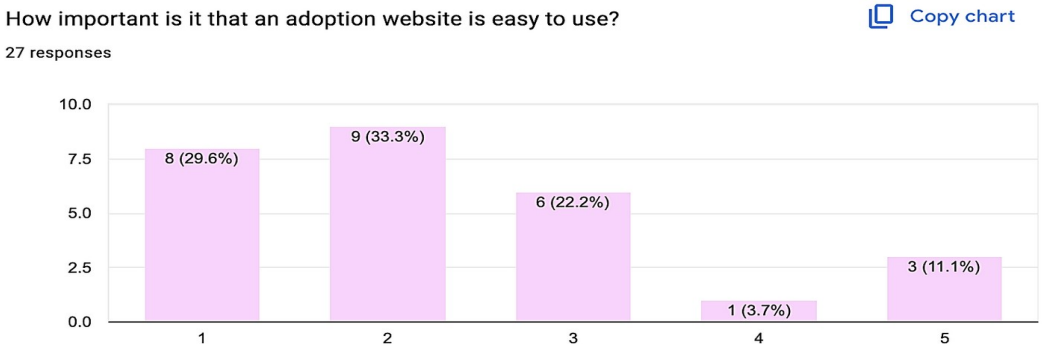
5.



Analysis:

The most selected factors were living space at 88.9% and time available at 85.2% showing that respondents think an adopter's environment and lifestyle are the most important parts of a successful match. Income and garden access were also highly chosen showing that practical care needs should be considered primarily.

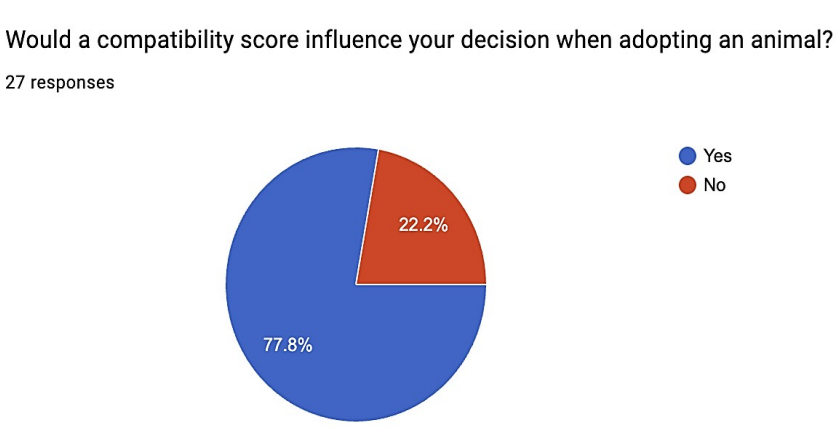
6.



Analysis:

Most responses were grouped around the higher end of the scale, which suggests that ease of use is important to users. As a result, I would like my website to have a simple layout, clear navigation, readable text and easy process. I aim to make it as accessible as possible.

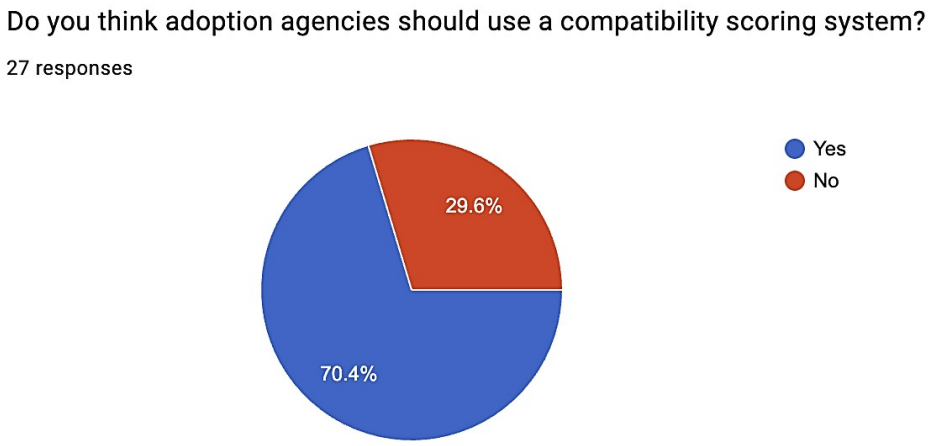
7.



Analysis:

77.8% of respondents said yes and 22.2% said no. This shows that most users may find a compatibility score helpful when deciding whether an animal would be suitable for them.

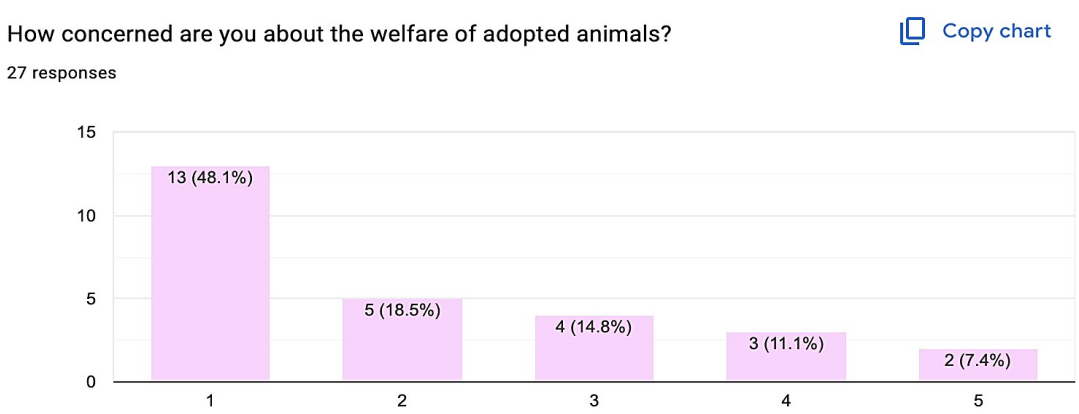
8.



Analysis:

70.4% of respondents said yes, while 29.6% said no. This shows that respondents support the idea of adoption centres using this scoring system, but not everyone agrees. Therefore, the compatibility score should be used as a guidance but not as the deciding factor.

9.



Analysis:

The largest group of respondents selected 1, with 48.1% and 18.5% selected 2. This shows that many respondents are concerned about animal welfare. This supports the purpose of this system as it aims to reduce unsuitable adoptions by helping users find animals and reduce chances of animals being placed in unsuitable homes.

Overall:

Overall, the survey shows that there is a clear need for a pet adoption system that helps users make informed decisions. Most respondents have shown that they agree that research is important before adopting an animal and would be influenced by the compatibility score. This survey also shows that the important factors include living space, time available, income, garden access and experience with animals.

My solution should include detailed animal profiles, a user questionnaire with a compatibility scoring system, where lifestyle compatibility will be the most important and popular factor in choosing an animal. This system should also be easy to use as users will have different levels of experiences with pet ownership and adoption.

However, one limitation of this survey is that it only received 27 responses, so it may not represent all potential adopters, and some respondents may have limited experience with adoption centres. It still provided me with guidance about common user expectations, especially importance of ease of use, animal welfare and compatibility scoring.

This survey was insightful to help me understand what potential users would expect from an animal adoption system, especially the need for clear information, lifestyle based matching and a compatibility score.

Prefixes: E—Examples | Q—Client Interview |S—Stakeholder Survey

Features

#	Feature	Notes	Justification	Source
1	Clean user interface	Calm colour scheme, clear headings, identifiable buttons and a clear layout	Makes system easier to understand and use, especially for those not confident with technology	S6, E3, E1, E2
2	Navigation bar across different pages	Allows users to move between pages easily	Competitor websites used simple navigation, which helps reduce confusion and makes system more user friendly	E1, E2, E3, S6

#	Feature	Notes	Justification	Source
3	User Home page	This system will include a clear home page introducing the purpose of adoption matching system	Simple starting point, and helps user understand system is focused on adoption suitability rather than browsing animals	E1, E2, E3
4	Animal preview Page	Users will be able to view available animals using compact animal cards. Each card will show key information such as name, species, breed, age and image.	Compact cards will make it easier for users to compare animals without needing to open every profile individually	E1, E2, E3
5	Animal Profile Page	Each animal will have a detailed profile page containing information such as name, species, breed, age, sex, personality, medical needs, exercise need and suitability requirements	The client stated that the animal profiles should show key details about the animal, which is also required for the matching algorithm	Q2, E1, E2
6	Multi Species Categories	Animals will be organised by species such as dogs, cats and rabbits	The RSPCA also has different animal categories, whereas dogs trust is just based around dogs.	E1, E2
7	Adopter Questionnaire	Users complete a questionnaire about their lifestyle and animal experience and will include many other answers which will contribute to the compatibility scoring algorithm.	The client stated that adopters often fail to consider important factors before adopting, this will allow them to reflect on their suitability before decisions are made.	Q1, S4, S5
8	Step by step questionnaire layout	This questionnaire will be split question by question instead of being displayed as one long form, with dropdown options.	This makes the process easier to complete and reduces the chance of users feeling overwhelmed.	E3, S6
9	Input Validation	To check if data that is required is not left blank and the data is valid.	Preventing the incomplete or incorrect data being used by the matching algorithm, recommendations more reliable	Q15, S8

#	Feature	Notes	Justification	Source
10	Compatibility scoring algorithm	The system will compare the adopter's answers with each animals requirements and calculate a numerical score.	The main computational feature, allows animals to be recommended on suitability	E3, S7, Q10
11	Ranked animal recommendations	After the questionnaire is submitted, animals will be displayed in order from most suitable to least suitable	Removes need for user to manually search through every animal	Q14
12	Requirements Breakdown	The system will show which requirements the adopters meet and which requirements they do not meet for each animal.	Helps users understand why an animal has received a particular score.	E3, Q3
13	Unsuitable match warning	If the user does not meet the requirements, the system will display a clear warning	The client said the system should not completely stop a user from adoption but warn them if they are unsuitable and why	Q3
14	User registration system	Users can create an account by registering with personal details such as name, email and password	Users can save progress, access their questionnaire and personalise their own experience	Q9
15	User and Admin Login and Logout system	Registered users will be able to securely login and logout their accounts and staff can login securely to access management side of the system	User login ensures data is only accessible to the correct user and admin login prevents normal user from changing animal records and protects accuracy over database	Q12
16	Admin animal management	Admins should be able to add, edit and delete animal records	Animal information should be kept up to date when new animals arrive, details change or animals are adopted	Q5
17	Database Storage	System will need to have a database alongside to store animal records, user accounts, admin login details and adopter questionnaire details	So data can be stored, searched, updated and used by the matching algorithm	Q15

#	Feature	Notes	Justification	Source
18	Responsive streamlit design	I will use Streamlit layout tools so pages remain clear on different screen sizes.	This makes system easier to use on different laptops or tablets without information being cut off or difficult to read	Q6, E1, E2, E3, S6

Hardware and Software Requirements

The technology used to build this system is based on the features that my client requires. The list of requirements is as follows:

- Clean user interface
- User registration and login
- Admin login

- Database storage for animal records, user accounts and questionnaire answers
- CRUD functionality for admins
- Input validation
- Compatibility scoring algorithm
- Ranked animal recommendations

For this project, I will develop my system on my MacBook, which has the required hardware and software capabilities to develop, run and test my system.

Developer

Hardware:

Device: MacBook Air, 13 Inch, M3 2024

Processor: Apple M3 Chip

RAM: 8GB

Display: I will use the 13.6 inch built in retina display alongside a 13 inch monitor

Input: Keyboard and Trackpad

Internet Connection: This is required in order to install packages, access documentation and test deployment.

This hardware is suitable as my system will not require advanced graphics processing or a powerful server. The main processing will be form input, database queries, validation and the algorithm, which is perfectly suited for my laptop.

Software:

Operating system: MacOS Sequoia 15.0

IDE: Visual studio code

Programming Language: Python

User interface framework: Streamlit

Database: SQLite

Database module: sqlite3

Browser: Chrome

I will use some additional libraries such as pandas which may be used to organise and display data in tables.

I have chosen python as my main programming language as it is the one I am most confident in, therefore I will not have to spend as much time trying to learn another language when python is suitable enough. It is suitable to create the matching

algorithm, validating user inputs and all the functionalities my system requires. It allows me to be more efficient and focus on the computational parts of the system such as the comparison between adopter answers and animal requirements producing compatibility scores.

For my user and admin interface I will be using streamlit. It is simpler to use than using tkinter and it provides built in tools such as buttons, forms, dropdowns, columns, containers and page navigation. This allows me to create the pages without using a full HTML, CSS and JS website.

For data storage I will use SQLite as it is what I have the most experience in. It is suitable and easy to use to store data required for my system such as animal records, user accounts, admin login details, questionnaire answers and compatibility results.

User ✦ Admins

My product will be a streamlit application, so users and admins will only require a device that accesses the system through any web browser., If deployed online, users will require an internet connection and they do not need to install any modules. They could access from a laptop, desktop computer or tablet, which may require a mouse, keyboard and a monitor to present the graphical user interface or even touchscreen if using a tablet.

Limitations

The system will be built in order for one specific animal shelter rather than a large organisation such as the RSPCA. This means that the database will only need to store the animals, users and admin accounts linked to that shelter. This is a large limitation as creating a system for multiple shelters would require extra features such as shelter admin accounts and location based searching and overall a larger database structure. This will be extremely difficult with the experience I have and would increase the complexity of my project. Furthermore, this would make development and testing less realistic within the time available.

The system will only be focusing on common adoption animals such as dogs, cats and rabbits rather than every possible species. This is due to different animals having different care requirements, and every possible animal would make the matching algorithm too broad and difficult to test within the time available. By focusing on a smaller number of species it will allow the compatibility algorithm to be more accurate and relevant. More species could be added within the future by expanding the animal requirements stored in the database.

Additionally the compatibility score is only there to provide guidance and will not replace the core decision making of adoption staff within the centre. Although it's able to compare adopters answers it will not be able to take in external factors such as personal circumstance or behavioural detail. It will also not be able to receive evidence to match their answers on

the questionnaire. Therefore, the final adoption decision is made by staff, whilst the questionnaire is a guide in order to help those looking to adopt make a better decision.

The system will also not be able to complete the full adoption process as it cannot handle legal contracts, home checks, payments and final adoption approval. This would require more security, data protection and verification processes which widens the scope increasing development time. My system's main focus is helping users consider suitability before choosing an animal.

I cannot ensure the accuracy of the recommendations will be perfect every time, but rather they depend on the users answering the questionnaire honestly. If a user gives inaccurate answers on the questionnaire it will not reflect their true suitability. I will try to reduce this by using clear wording and validation but it cannot fully prevent users from entering inaccurate information.

The matching algorithm is rule based rather than using AI or ML. As a rule based algorithm is easier to explain, test and justify within the project. It will also be clearer to break down and show a user why an animal has or has not been recommended to the users. Additionally, machine learning would require a large dataset of previous adoptions which is not realistic for this project.

Animal records will need to be updated manually by admin users. This may lead to inconsistencies, or animals showing available when they are not. This is not something I can control but rather within the staff's responsibility to add edit and delete records so the database can be kept up to date.

Some advanced features such as animations, live chats would not be included and some of the formatting and design will be simplified. It would increase development and testing time without improving the main purpose of the system.

Success Criteria

Prefixes: E—Examples | Q—Client Interview|S—Stakeholder Survey

#	Feature	Requirements	Source
1	Navigation and Clean user interface	Pages must use a calm colour scheme, clear headings and identifiable buttons	**S6, E1, E2, E3**
		The interface must remain readable on laptop and tablet sized screens	**Q6, S6**
		The system must include a navigation menu to access home page, animal preview page, profile page, and login page	**E1, E2, E3, S6**
		The menu and interface style must remain consistent across all the pages	**E1, E2, E3, S6**
2	Home Page	Should explain the purpose of the animal adoption matching system, especially including that the system focuses on suitability rather than only appearance	**Q1, Q3, E1, E2, E3**
		Should have a clear button or navigation option that directs users to complete the adoption questionnaire	**Q1, S5, S6**
3	Animal preview page	The page should display every available animal using compact animal cards	**E1, E2, E3**
		Each card should show the species, name, breed, age and image	**Q2, E1, E2**
		The system must support at least 3 animal species including dogs, cat and rabbits	**E2**
		Users must be able to open a full animal profile from the preview page	**E1, E2, E3**
4	Animal Profile Page	Each profile must display the animals name, species, breed, age and sex	**E1, E2, Q2**
		Each profile also must include personality, medical needs, exercise needs and suitability requirements	**Q2**
		Animal profile information must be retrieved from the database	**Q15**
		Users must be able to return to the animal preview page or results page	**S6, E1, E2, E3**
5	User and Admin registration and login page	Users must be able to create an account, using name, email and password and it can include other features such as age to ensure they are at a suitable age	**Q9, Q12**
		The system must reject registration if required fields are empty and the email format is invalid, by validating inputs	**Q12, Q15**
		Registered users must be able to login and logout using their correct account details	**Q12**
		The admin login details should allow admins to access their own section with the CRUD interface	**Q12, Q5**

#	Feature	Requirements	Source
		Incorrect Login details must display an error message and prevent access to the account	**Q12, S6**
6	Adopter Quest'naire Page	The questionnaire must collect information about the user's home, lifestyle, garden access, children, other pets, time available and previous animal experience	**Q1, S5**
		The questionnaire must be split into clear sections or individual questions rather than appearing as one long form	**E3, S6**
		The questionnaire must use suitable input methods such as dropdowns, radio buttons or selection boxes but preferably dropdowns.	**E3, S6**
		The system must prevent the questionnaire from being submitted if important questions are unanswered	**Q15, S8**
7	Results and recomm'ns page	The system must compare adopter's questionnaire with each animal's requirements	**Q10, Q1, S5**
		The system must calculate a numerical compatibility score for each animal	**Q10, S7**
		The top 4 animals must be displayed from highest compatibility score to lowest at a minimum	**Q14, S7**
		The results page must show which key requirements the user meets and does not meet for each animal	**Q3, E3**
		The system must display a warning if the user does not meet important requirements for an animal	**Q3**
		The results page must explain that the compatibility scores are guidance, and the adoption staff have the final say	**Q3**
8	Admin manage't page	This page should be accessed separately with a separate admin login	**Q12**
		Admin users must be able to add a new animal record with all the information required	**Q5, Q2**
		Admin users must be able to edit existing animal records and save the updated information to the database	**Q5, Q15**
		Admin users must be able to delete animal records when animals adopted or no longer available	**Q5**
		Normal users must not be able to access the admin management page	**Q12**
9	Database system	The database must store animal records, user account details, admin login details and questionnaire answers	**Q15**
		The database must allow animal records to be added, viewed, edited and deleted through the admin system	**Q5, Q15**
		Questionnaire answers must be stored or processed so that they can be used by the compatibility scoring algorithm	**Q10, Q15**
		Data must be retrieved correctly so animal profiles and recommendations display accurate information	**Q15, Q2, Q10**

Model version addition

How each success criterion will be measured

Each requirement in the table above is numbered by group and row: 5.2 is the second requirement of group 5. Where possible the measure is a number (a count, a time, a capacity or a rating), so the evaluation can say met, partly met or not met. Tests named T are timing and capacity checks in the post-development plan.

#	Requirement (from the table above)	How it will be measured
1.1	Pages must use a calm colour scheme, clear headings and identifiable buttons	One theme file sets every colour; body text contrast at least 4.5 : 1 (WCAG AA); every action is a labelled button
1.2	The interface must remain readable on laptop and tablet sized screens	At 1280 px and 820 px (tablet) wide: no sideways scrolling and no text cut off
1.3	The system must include a navigation menu to access home page, animal preview page, profile page, and login page	The top menu holds Home, Animals, Questionnaire, My matches and Log in / My account on 7 of 7 pages
1.4	The menu and interface style must remain consistent across all the pages	0 pages with their own colours or CSS
2.1	Should explain the purpose of the animal adoption matching system, especially including that the system focuses on suitability rather than only appearance	The home page states the purpose and the word 'suitability'
2.2	Should have a clear button or navigation option that directs users to complete the adoption questionnaire	1 click from Home to the questionnaire
3.1	The page should display every available animal using compact animal cards	Number of cards = number of animal records (13 of 13; 500 of 500 in the load test)
3.2	Each card should show the species, name, breed, age and image	5 of 5 facts on every card: species, name, breed, age, image
3.3	The system must support at least 3 animal species including dogs, cat and rabbits	At least 3 species stored and shown
3.4	Users must be able to open a full animal profile from the preview page	1 click from a card to its profile
4.1	Each profile must display the animals name, species, breed, age and sex	5 of 5 shown: name, species, breed, age, sex
4.2	Each profile also must include personality, medical needs, exercise needs and suitability requirements	Personality, medical needs, exercise needs and suitability all shown
4.3	Animal profile information must be retrieved from the database	Profile is loaded by animal_id from SQLite; editing the record changes the page
4.4	Users must be able to return to the animal preview page or results page	1 click back to the list or to My matches
5.1	Users must be able to create an account, using name, email and password and it can include other features such as age to ensure they are at a suitable age	A valid sign-up stores 1 user row with a hashed password
5.2	The system must reject registration if required fields are empty and the email format is invalid, by validating inputs	100 % of the blank and invalid cases A2-A6 and A11 are rejected with a message
5.3	Registered users must be able to login and logout using their correct account details	Correct details log in; Log out clears the session
5.4	The admin login details should allow admins to access their own section with the CRUD interface	A staff account sees 'Manage animals'; an adopter sees no staff link
5.5	Incorrect Login details must display an error message and prevent access to the account	Wrong password and unknown email give the same message and no session
6.1	The questionnaire must collect information about the user's home, lifestyle, garden access, children, other pets, time available and previous animal experience	9 questions cover home, garden, children, other pets, activity, hours alone, experience, species, size
6.2	The questionnaire must be split into clear sections or individual questions rather than appearing as one long form	4 steps and a review step; 4 of 5 testers rate the layout 8/10 or more for clarity
6.3	The questionnaire must use suitable input methods such as dropdowns, radio buttons or selection boxes but preferably dropdowns.	0 free-text boxes in the questionnaire
6.4	The system must prevent the questionnaire from being submitted if important questions are unanswered	A blank answer blocks Next; an incomplete set saves 0 rows
7.1	The system must compare adopter's questionnaire with each animal's requirements	Every animal gets a result (N results for N animals)
7.2	The system must calculate a numerical compatibility score for each animal	A percentage from 0 to 100 (1 decimal place); 500 animals scored and stored in under 2 seconds
7.3	The top 4 animals must be displayed from highest compatibility score to lowest at a minimum	At least 4 shown, best first
7.4	The results page must show which key requirements the user meets and does not meet for each animal	Every result lists what is met and what is not
7.5	The system must display a warning if the user does not meet important requirements for an animal	An unmet hours-alone, children or space requirement shows a warning
7.6	The results page must explain that the compatibility scores are guidance, and the adoption staff have the final say	The results page says scores are guidance and staff decide
8.1	This page should be accessed separately with a separate admin	The staff page is reached only through a staff login

	login	
8.2	Admin users must be able to add a new animal record with all the information required	A complete add form stores a record; any invalid field stores nothing
8.3	Admin users must be able to edit existing animal records and save the updated information to the database	An edit is saved and shown on the profile
8.4	Admin users must be able to delete animal records when animals adopted or no longer available	Delete only after the confirmation box is ticked; the record and its results are removed
8.5	Normal users must not be able to access the admin management page	An adopter is refused every staff action, even if the page is bypassed
9.1	The database must store animal records, user account details, admin login details and questionnaire answers	Stores users, animals, responses and results; at least 500 animals and 100 users
9.2	The database must allow animal records to be added, viewed, edited and deleted through the admin system	Animals can be added, viewed, edited and deleted
9.3	Questionnaire answers must be stored or processed so that they can be used by the compatibility scoring algorithm	Answers are saved and used for the score
9.4	Data must be retrieved correctly so animal profiles and recommendations display accurate information	Values read back equal the values stored

References

BBC News (2024) *RSPCA receives record number of animal cruelty reports*. Available at: https://www.bbc.co.uk/news/articles/cwyrnlj5632o (Accessed: 19 June 2026).

Dogs Trust (n.d.) *About us*. Available at: https://www.dogstrust.org.uk/about-us (Accessed: 25 June 2026).

Dogs Trust (n.d.) *How to adopt a dog*. Available at: https://www.dogstrust.org.uk/rehoming/how-to-adopt (Accessed: 25 June 2026).

Petfinder (n.d.) *Adopter profile: Finding your perfect pet*. Available at: https://www.petfinder.com/pet-adopter-profile/ (Accessed: 25 June 2026).

Petfinder (n.d.) *Pet Finder Quiz*. Available at: https://www.petfinder.com/user/profile/create/ (Accessed: 25 June 2026).

RSPCA (n.d.) *Find a pet*. Available at: https://www.rspca.org.uk/findapet (Accessed: 25 June 2026).

RSPCA (n.d.) *Pet adoption process guide*. Available at: https://www.rspca.org.uk/findapet/adopt (Accessed: 25 June 2026).

sienna

Design

✔ Introduction/Overall

Proposed Layout

Model version addition

Hierarchy chart

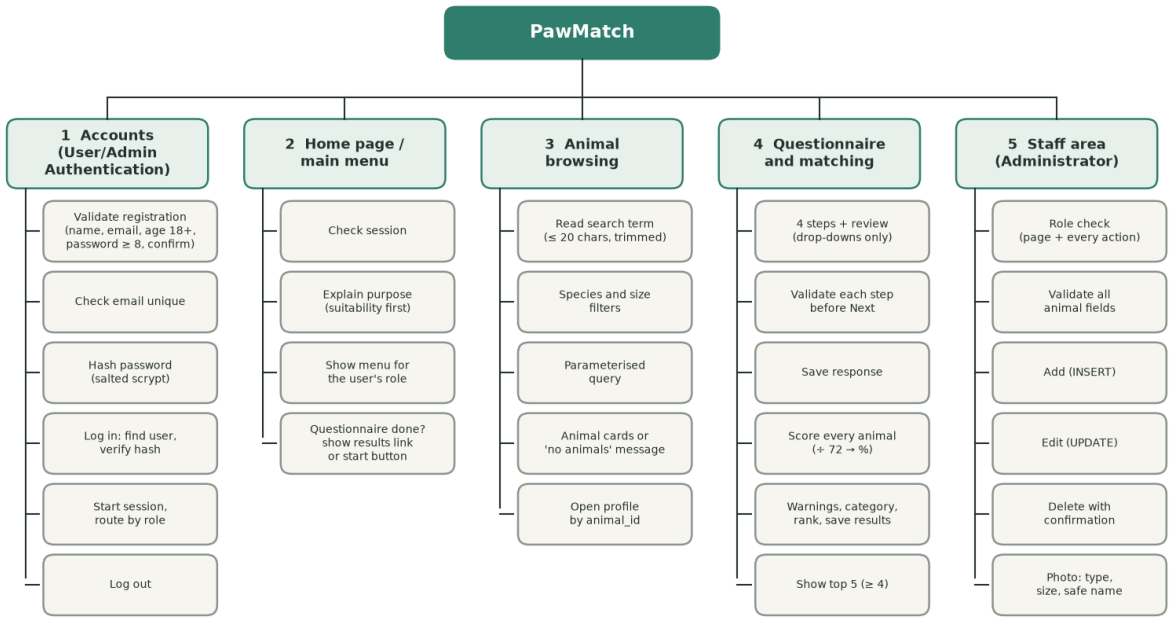


Diagram D1: PawMatch broken down into its five components and their sub-tasks.

Why this breakdown: each box is small enough to build and test on its own, and the order follows what each part needs. Accounts and the database come first because every other part needs a logged-in user and stored animals; browsing needs animals; matching needs both the questionnaire answers and the animals. The scoring and validation boxes need no screen at all, so they can be built and fully tested before any page exists (Development, stage 1).

Structure of the solution

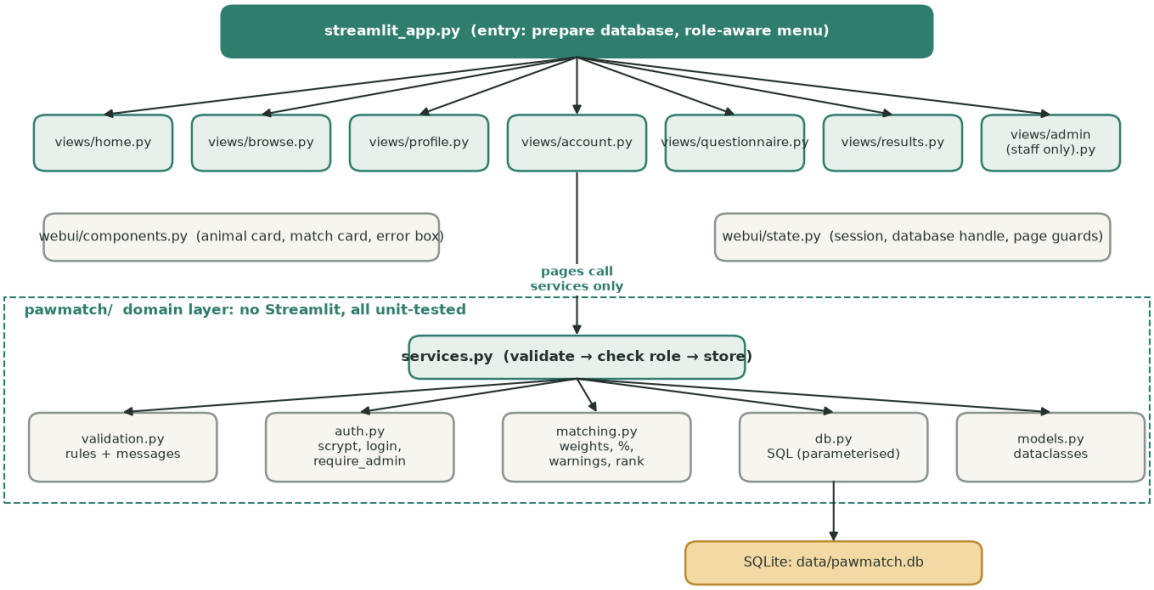


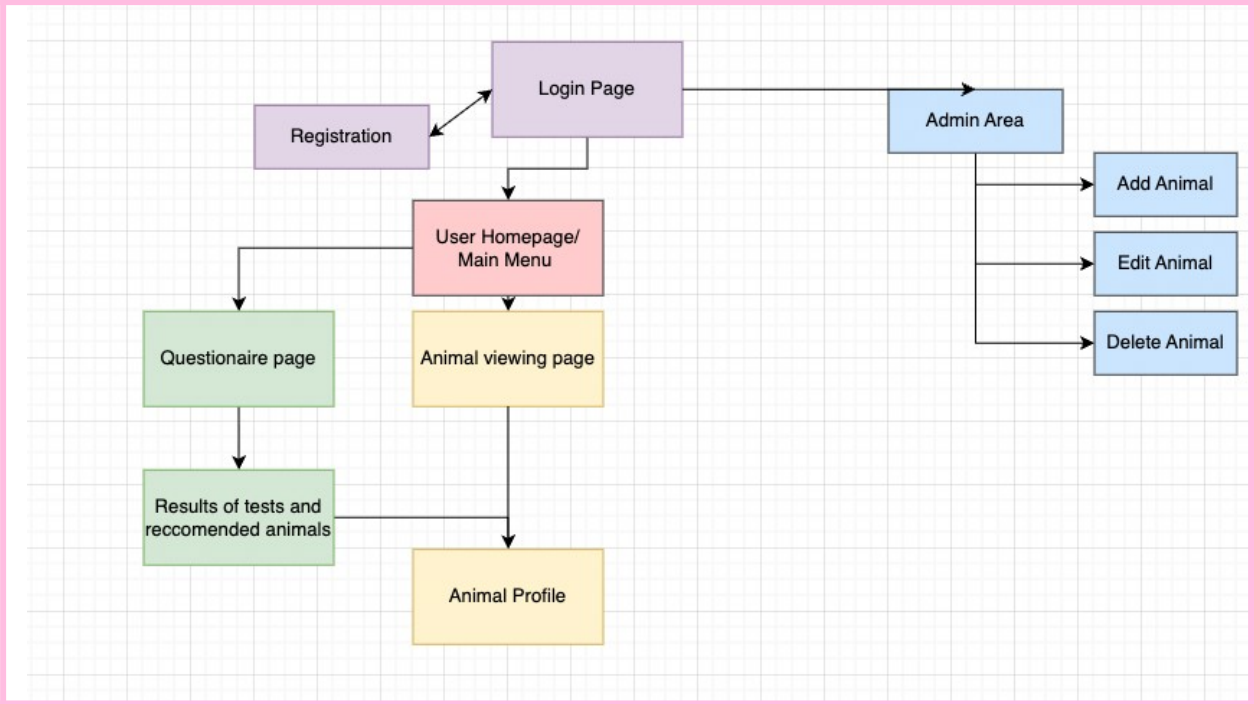
Diagram D2: modules and layers. Pages only call services; only db.py runs SQL.

Two layers. The pawmatch package holds all rules and data access and never imports Streamlit, so every rule can be unit-tested. The webui pages read input, call one service and show the result. This means a change to a rule happens in one place, and a page cannot skip a check because the check is inside the service, not the page.

The system can be decomposed into five main components. Separating the system into smaller components makes it easier to design, develop, test and maintain whilst each section performs a specific function.

Section	Front End Justification	Backend Justification
User/Admin Authentication	Will allow users to register and existing users to login, and allow admins to access their page safely and securely. Registration will collect the users name, email, age and password. The interface should provide clear feedback when details are missing or forgotten from its validation system. It will also show an error message if login details are incorrect	The system will validate entered details, such as: - email cannot be blank and must be valid and unique - Password minimum 8 characters - Age must be a valid number - Login fields cannot be blank The passwords will be hashed in order to be kept secure and safe. During login, enter details will be compared against records in the users table. A session will then be created when authentication is successful. For the admin side, if the login details are incorrect it will not
User Homepage/Main Menu	Will provide the main navigation area after the user logs in. It will allow the user to access the questionnaire, browse available animals and view their account. It will also explain the purpose of the adoption matching system. It will be clear and	The system will check the user's session to make sure they are logged in. It may retrieve the email and recent questionnaire information from the database so that personalised information can be displayed.
Administrator Menu	Provides staff with a dashboard to manage existing animals and their records. Admins will be clearly able to add new animals, edit existing details and remove records. To add animals it will store: Name, species, age, breed,	The system will check that the logged in account has the admin role before access is allowed. Parameterised SQL commands will insert, update and delete animal records. All inputs will be

	children compatible, garden required, medical notes and an image. Forms will be clearly Labelled and display confirmation or error messages.	out function will clear the admins session and return them to the login page, preventing another person using the same device which will prevent unwanted
Animal Viewing	Allows users to browse available animals using clearly presented animal cards. Each card will show information such as the animal name, image, species, breed and age. Users will be able to select a card to view a more	Animal records will be retrieved from the animals table. Search and filter selections will be used to change the database query or filter the retrieved records. Selecting an animal will use its unique animal ID to
Questionnaire and Recommendations	Will use a form in order to allow users to answer questions about their home, lifestyle, experience and preferences. The questionnaire will have questions such as the housing type, access to garden, animal loneliness times, children, pets, experience, activity labels, what species and size preferred. These questions would not appear all at one time to not overwhelm the user and the input type would be drop downs or radio buttons. After submitting the questionnaire, the system will display	Responses will be validated and compared with the requirements of each available animal. A weighted compatibility algorithm will calculate a percentage score for every animal. The animals will then be sorted from highest to lowest score so the top recommendations can be displayed. The questionnaire responses and match results may also be stored within the database with the users ID (email)



Database structure

Tables	Purpose
Users	Stores registered user and administrator account details
Animals	Stores user profiles, care needs and matching requirements
Questionnaire responses	Stores answers submitted by users
Matchresults	Stores calculated compatibility scores between responses and animals

One user can complete multiple questionnaire responses, each questionnaire can produce multiple compatibility results because it is compared to every available animal. Each animal can therefore appear in multiple match results belonging to different users or questionnaire attempts.

-Users **1** → **Many** Questionnaire responses

-Questionnaire responses **1** → **Many** matchresults -

animals **1** → **Many** matchresults

Model version addition

Entity relationship diagram



Diagram D3: the four tables, their keys and the one-to-many links.

Two fields are added to the questionnaire table (other_pets, experience) so the answers to those questions are stored even though the Design does not score them, and match_results also stores the met / unmet lists and the bonus, so the results page can be shown again without re-scoring. Foreign keys use ON DELETE CASCADE so no result can point to an animal or response that no longer exists.

Data structures and classes

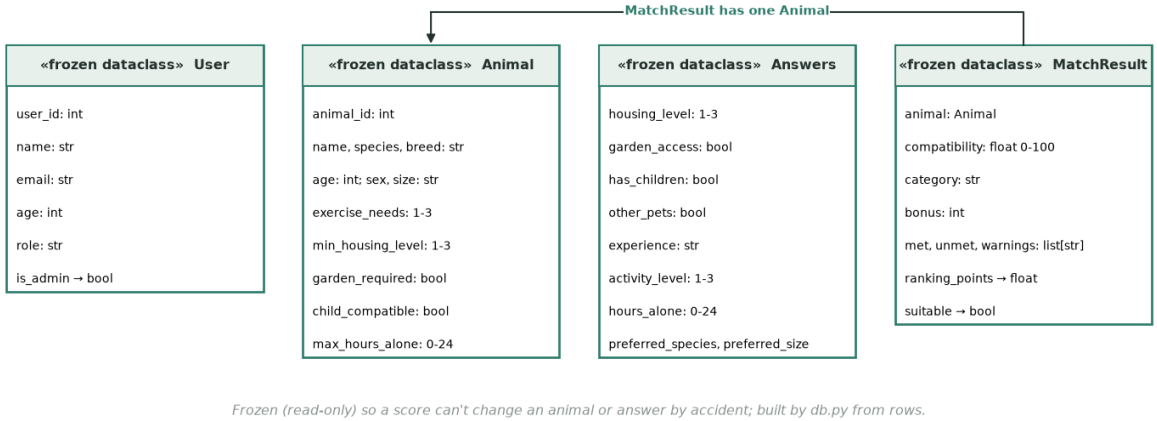


Diagram D4: the four data classes passed between the modules.

Structure	Type	Why this structure	Validation
User, Animal, Answers, MatchResult	Frozen dataclasses	Named fields instead of loose tuples; frozen so scoring cannot change an animal or answer by mistake	Only built from validated or stored values

WEIGHTS	Dictionary factor → points	The Design's weight table in one place; MAX_POINTS = sum of WEIGHTS	Test E1 checks 20/15/15/12/10
QUESTION_STEPS	Tuple of (title, question names)	The questionnaire plan as data: the page and the validator both use it	Test D1: every question once
errors	Dictionary field → message	Every validator returns one, so a page can show all problems at once	Empty dictionary means valid
session.user_id, session.q_step, session.q_answers	Streamlit session state	Who is logged in, and progress through the questionnaire	user_id re-checked against the database on every page
results	List of MatchResult, sorted	Ranked order is the output; a list keeps it	Every animal appears once

Validation

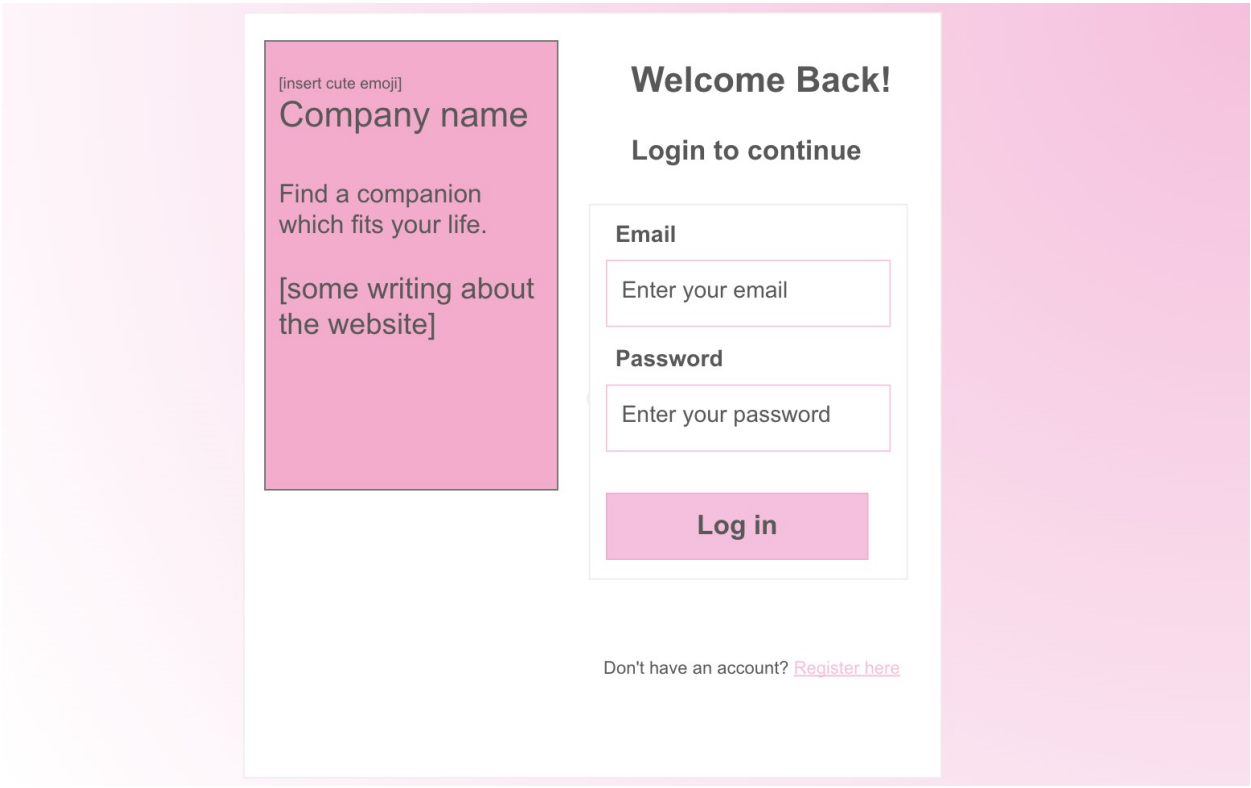
All user entered data should be validated before it is processed or stored in the SQLite database. Streamlit controls such as dropdowns radio buttons and number inputs will restrict the range of values users can enter. However, the backend will still validate the submitted values because interface controls alone should not be relied on.

Login + Adminitration

Login and Registration

User/Admin Authentication	Will allow users to register and existing users to login, and allow admins to access their page safely and securely. Registration will collect the users name, email, age, address and password. The interface should provide clear feedback when details are missing or forgotten from its validation system. It will also show an error message if login details are incorrect	The system will validate entered details, such as: - Username cannot be blank - Password minimum 8 characters - Required questions cannot be skipped - Age must be a valid number - Duplicate usernames not allowed The passwords will be hashed in order to be kept secure and safe. During login, enter details will be compared against records in the users table. A session will then be created when authentication is successful. For the admin side, if the login details are incorrect it will not

Interface Design



Branding and colour - A limited pink, white and dark grey palette will create a recognisable identity without being too distracting and bright. The system name will confirm that the user is on the right adoption website, and the dark text will keep information readable.

Whitespace and alignment - all the writing and textboxts are consistently aligned and have clear spacing. This separates sections and will lowers the risk of the user selecting wrong section. The amount white space reduces chance of user becomes overwhelmed.

Clear visuals - Large bold heading will welcome users and clearly identifies the page. It is followed by the task instruction. The difference in size and position will guide the user to page purpose to form clearly.

Mini introduction - the short statement will explain a bit about the system, this will give context before login and will reassure the first time users about the purpose of the website

Boxing inputs - the email, password and the login buttons are all within one section which shows they are all together very clear for the user making the task readable.

Two columns - The branding is separate from the login. This will allow the user to focus on one point at a time which will prevent them from becoming overwhelmed increases usability.

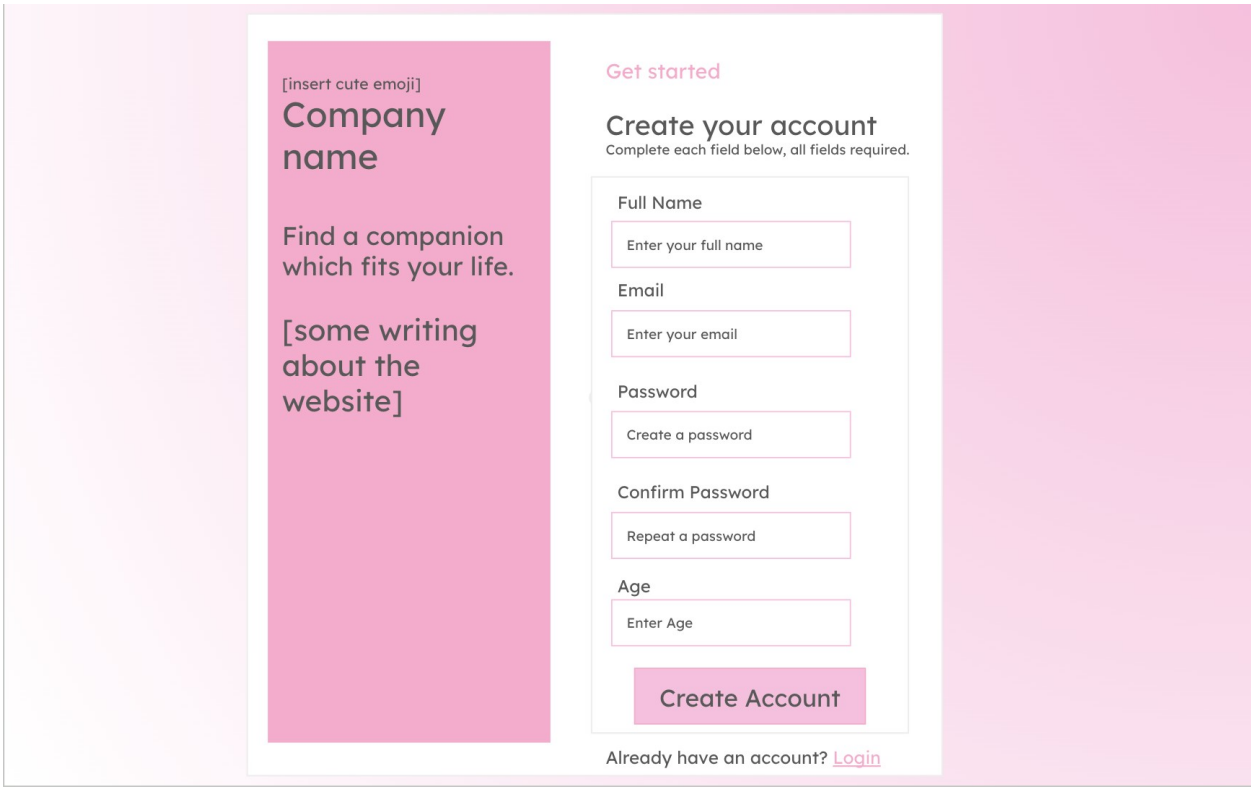
Labels and prompts - The email and password boxes are labelled with placeholder text to show the user where the information should be entered. Double labels will ensure the user will know where to enter data clearly.

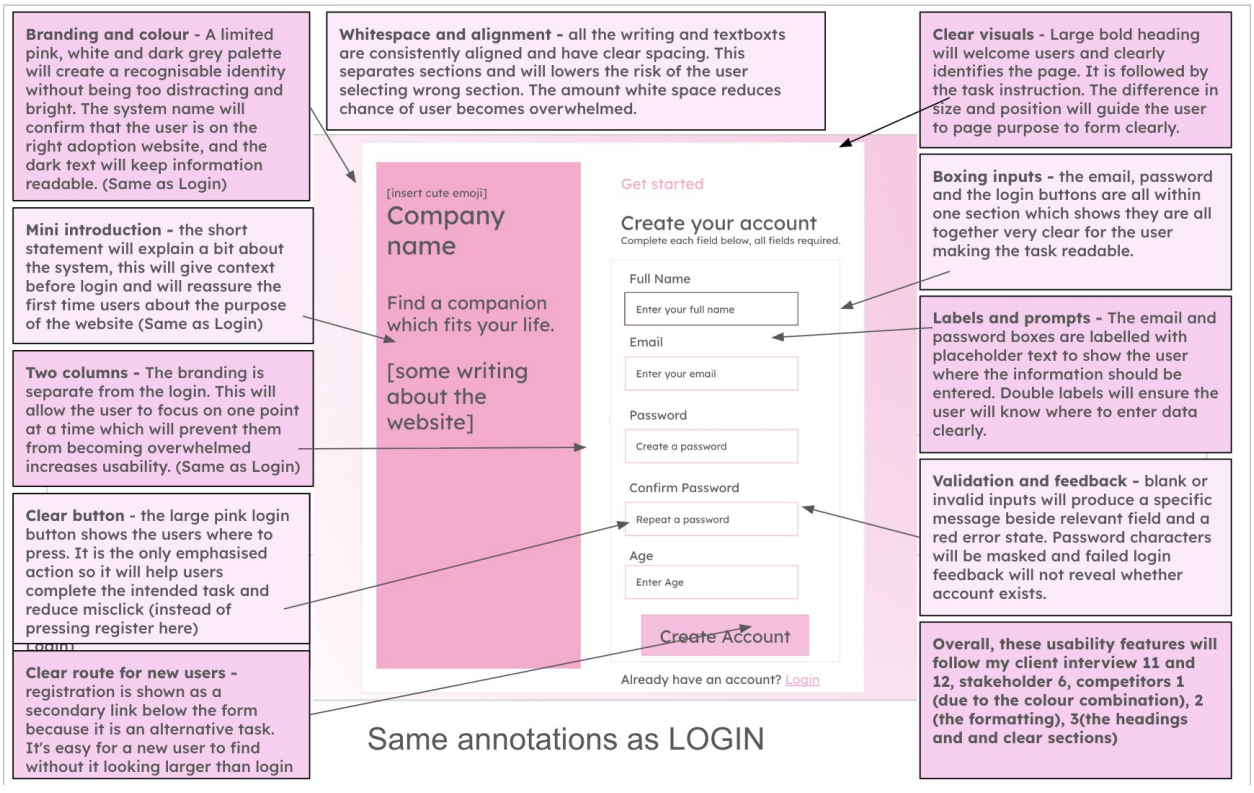
Clear button - the large pink login button shows the users where to press. It is the only emphasised action so it will help users complete the intended task and reduce misclick (instead of pressing register here)

Validation and feedback - blank or invalid inputs will produce a specific message beside relevant field and a red error state. Password characters will be masked and failed login feedback will not reveal whether account exists.

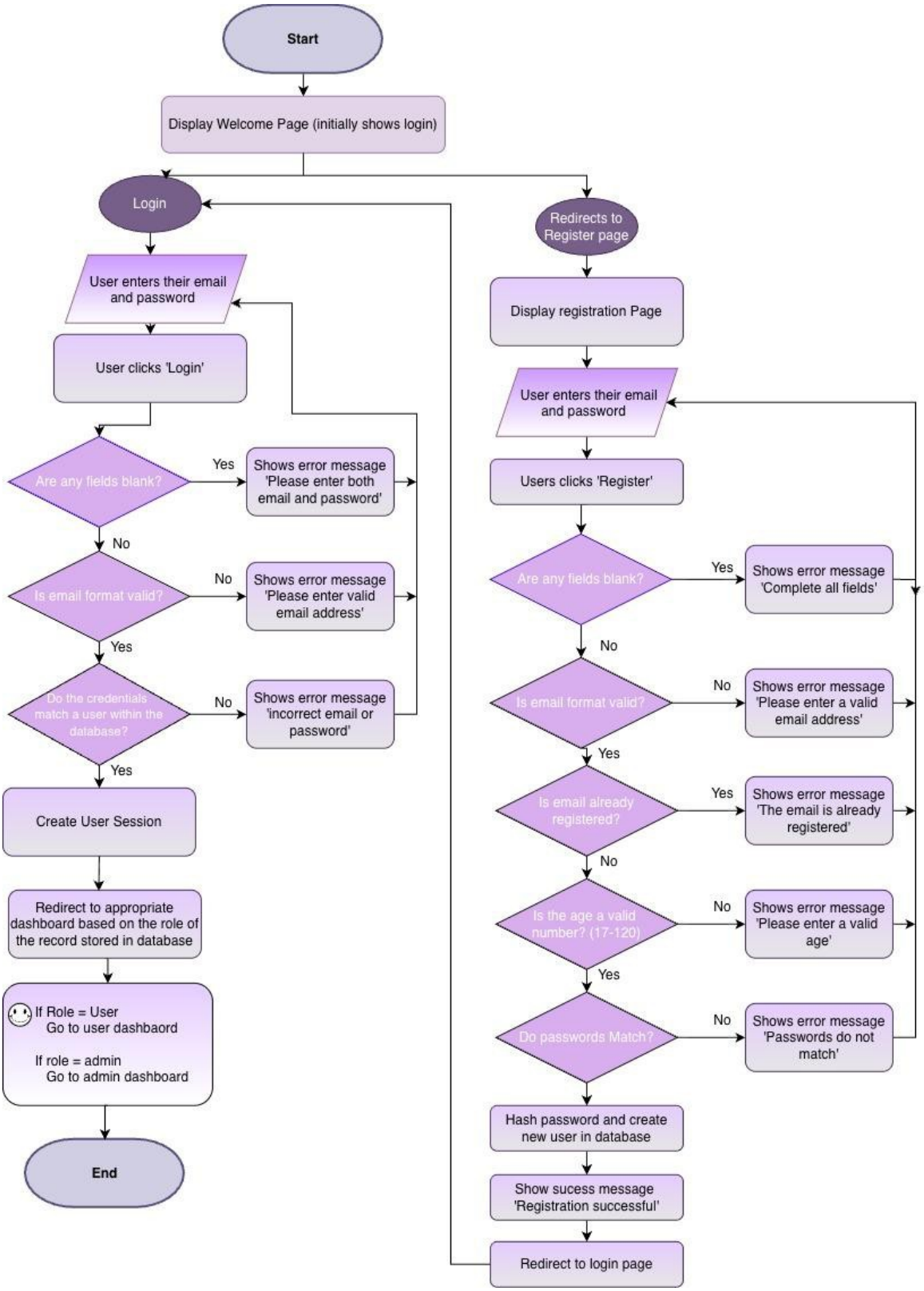
Clear route for new users - registration is shown as a secondary link below the form because it is an alternative task. It's easy for a new user to find without it looking larger than login

Overall, these usability features will follow my client interview 11 and 12, stakeholder 6, competitors 1 (due to the colour combination), 2 (the formatting), 3(the headings and and clear sections)


[insert cute emoji]
Company name
Find a companion which fits your life.
[some writing about the website]
Get started
Create your account
Complete each field below, all fields required.
Full Name
Enter your full name
Email
Enter your email
Password
Create a password
Confirm Password
Repeat a password
Age
Enter Age
Create Account
Already have an account? Login


Branding and colour - A limited pink, white and dark grey palette will create a recognisable identity without being too distracting and bright. The system name will confirm that the user is on the right adoption website, and the dark text will keep information readable. (Same as Login)
Whitespace and alignment - all the writing and textboxts are consistently aligned and have clear spacing. This separates sections and will lowers the risk of the user selecting wrong section. The amount white space reduces chance of user becomes overwhelmed.
Clear visuals - Large bold heading will welcome users and clearly identifies the page. It is followed by the task instruction. The difference in size and position will guide the user to page purpose to form clearly.
Mini introduction - the short statement will explain a bit about the system, this will give context before login and will reassure the first time users about the purpose of the website (Same as Login)
Boxing inputs - the email, password and the login buttons are all within one section which shows they are all together very clear for the user making the task readable.
Two columns - The branding is separate from the login. This will allow the user to focus on one point at a time which will prevent them from becoming overwhelmed increases usability. (Same as Login)
Labels and prompts - The email and password boxes are labelled with placeholder text to show the user where the information should be entered. Double labels will ensure the user will know where to enter data clearly.
Clear button - the large pink login button shows the users where to press. It is the only emphasised action so it will help users complete the intended task and reduce misclick (instead of pressing register here)
Validation and feedback - blank or invalid inputs will produce a specific message beside relevant field and a red error state. Password characters will be masked and failed login feedback will not reveal whether account exists.
Clear route for new users - registration is shown as a secondary link below the form because it is an alternative task. It's easy for a new user to find without it looking larger than login
Overall, these usability features will follow my client interview 11 and 12, stakeholder 6, competitors 1 (due to the colour combination), 2 (the formatting), 3(the headings and and clear sections)
[insert cute emoji]
Company name
Find a companion which fits your life.
[some writing about the website]
Get started
Create your account
Complete each field below, all fields required.
Full Name
Enter your full name
Email
Enter your email
Password
Create a password
Confirm Password
Repeat a password
Age
Enter Age
Create Account
Already have an account? Login
Same annotations as LOGIN

Algorithm (Flow chart, pseudocode)



Pseudocode

Model version addition

Pseudocode: registration, login, session and role check, logout

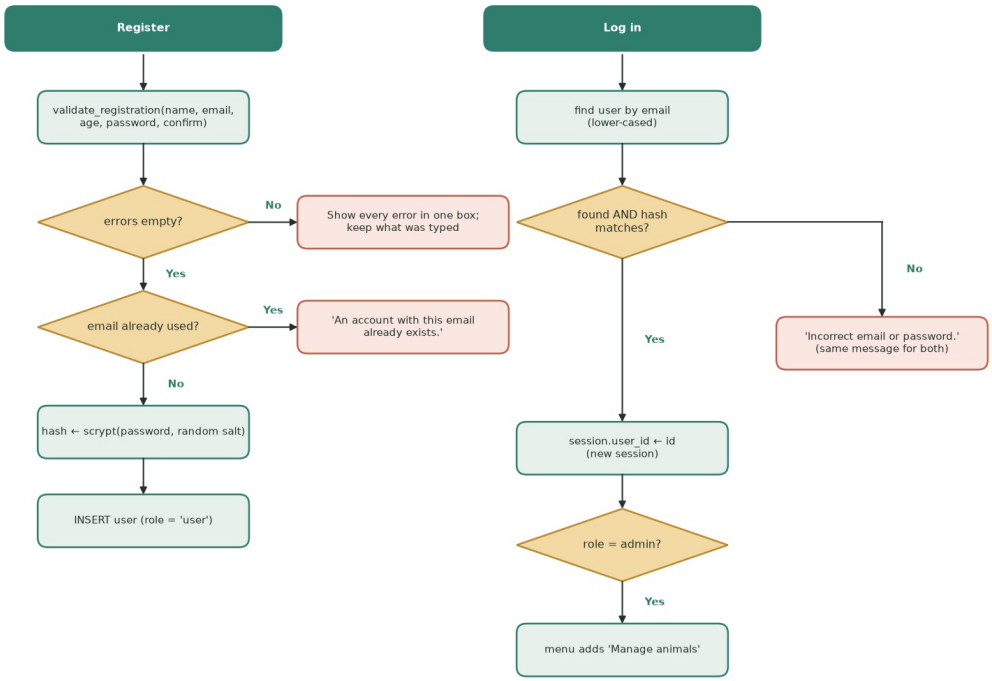


Diagram D5: registration and login flowchart (model version).

```
FUNCTION Register(user_name, email, age, password, confirm_password)
    errors ← empty dictionary                        // field name → message
    user_name ← TRIM(user_name)
    IF user_name = "" THEN
        errors["name"] ← "Enter a name."
    ELSE IF LENGTH(user_name) < 2 OR LENGTH(user_name) > 50 THEN
        errors["name"] ← "Name must be 2 to 50 characters."
    ELSE IF user_name has a character that is not a letter, space, hyphen or apostrophe THEN
        errors["name"] ← "Name can only use letters, spaces, hyphens and apostrophes."
    ENDIF
    email ← LOWER(TRIM(email))
    IF email = "" THEN
        errors["email"] ← "Enter an email address."
    ELSE IF email does not match  text "@" text "." letters  THEN
        errors["email"] ← "Enter a valid email address, like name@domain.com."
    ENDIF
    IF age is blank THEN
        errors["age"] ← "Age is required."
    ELSE IF age is not a whole number THEN
        errors["age"] ← "Age must be a whole number."
    ELSE IF age < 18 THEN
        errors["age"] ← "You must be 18 or over to adopt."
    ELSE IF age > 120 THEN
        errors["age"] ← "Age must be between 18 and 120."
    ENDIF
    IF TRIM(password) = "" THEN
        errors["password"] ← "Enter a password."
    ELSE IF LENGTH(password) < 8 THEN
        errors["password"] ← "Password must be at least 8 characters."
    ELSE IF password ≠ confirm_password THEN
        errors["confirm"] ← "The two passwords do not match."
    ENDIF
    IF errors is empty AND EmailExists(email) THEN       // SELECT 1 FROM users WHERE email =
?
        errors["email"] ← "An account with this email already exists."
```

```
    ENDIF
    IF errors is not empty THEN
        RETURN NULL, errors                              // page shows all messages in one
box
    ENDIF
    salt ← 16 random bytes
    password_hash ← "scrypt$16384$8$1$" + HEX(salt) + "$" + HEX(SCRYPT(password, salt))
    user_id ← INSERT INTO users (name, email, age, password_hash, role, created_at)
                VALUES (?, ?, ?, ?, 'user', NOW())        // parameterised
    RETURN user, empty dictionary
ENDFUNCTION
```

```
FUNCTION Login(login_email, login_password)
    login_email ← LOWER(TRIM(login_email))
    IF login_email = "" OR login_password = "" THEN
        RETURN NULL, "Enter your email address." / "Enter your password."
    ENDIF
    record ← SELECT * FROM users WHERE email = ?          // login_email bound, never joined
into SQL
    is_authenticated ← FALSE
    IF record ≠ NULL THEN
        salt, stored_digest ← SPLIT(record.password_hash)
        is_authenticated ← CONSTANT_TIME_EQUAL(SCRYPT(login_password, salt), stored_digest)
    ENDIF
    IF NOT is_authenticated THEN
        error_message ← "Incorrect email or password."    // same message: no hint which part
was wrong
        RETURN NULL, error_message
    ENDIF
    CLEAR session
    session.user_id ← record.user_id
    RETURN record, empty dictionary
ENDFUNCTION

FUNCTION CurrentUser()                                    // runs at the top of every page
    IF session.user_id is not set THEN
        RETURN NULL
    ENDIF
    RETURN SELECT * FROM users WHERE user_id = session.user_id
ENDFUNCTION                                               // user_role is re-read on every page

FUNCTION RequireStaff()                                   // the session/role check
    user ← CurrentUser()
    IF user = NULL THEN
        SHOW "Please log in or register to use this page." and a login link
        STOP page
    ELSE IF user.user_role ≠ "admin" THEN
        SHOW "This page is for shelter staff only."
        STOP page
    ENDIF
    RETURN user
ENDFUNCTION

PROCEDURE Logout()
    CLEAR session                                         // user_id and any half-finished
questionnaire
    GO TO Home
ENDPROCEDURE
```

How it fits the whole solution. Register and Login are the only ways to get a session. Every other page starts with CurrentUser() or RequireStaff(), so the role check written here protects the whole system. Uniqueness is checked only after the cheap checks pass, so the database is not queried for a form that is already invalid.

Database (Table structure)

Field	Data type	Key/validation
user_id	INTEGER	Primary key which will auto increment
name	TEXT	required
email	TEXT	Required and unique
age	INTEGER	required , valid range
password_hash	TEXT	required
user_role	TEXT	‘User’ or ‘admin’
date_created	DATE	required

Variables

Variable Name	Data Type	Purpose	Justification	E.g or Validation
user_name	String	Stores the name entered during registration	This will allow the system to identify and personalise the account. A string	Cannot be blank

			alphabetic characters and may	
email	String	Will store users email address	It will be used as a unique identifier for logging in and contacting the user. It will contain letters and	Must follow a valid email format and be unique
age	integer	Stores the users age	It will be used to ensure the user meets any minimum age requirements. An integer is suitable	Must be a valid positive number.
password	string	Temporarily stores the password entered by the password	It is needed to authenticate the user securely. Passwords usually contain letters, numbers and	Minimum of eight characters.
confirm_password	string	Stores the repeated password entered during registration	This allows the system to check that the user has mistyped their password	It will have to match password
password_hash	string	Stores the hashed version of the password	Passwords should not be stored as plain text. Storing the hash reduces the risk of passwords being	Generated using a secure hashing function
login_email	string	Stores the email entered on the login page	The variable is required so that the system can search for the corresponding	Cannot be blank
login_password	string	Temporarily stores the password entered during login	The entered password will be checked against the stored password hash	Cannot be blank
user_id	integer	Uniquely	An integer is efficient for use as	It will be retrieved

		identifies the logged in user	database primary key, it will allow records such as questionnaires and	from the database after login
user_role	String	Stores whether the account is an administrator or a user	This variable is required for access control, so that normal users cannot access the administrator pages Controls access to the administrator area, although normal	Will be stored as ‘user’ or ‘admin’
error_message	string	Stores the error message which will appear when the input or login details are invalid	My system will provide clear feedback and help the user understand what has gone wrong and how to correct it. A string will be	‘Incorrect email or password, please retry’
is_authenticated	booleen	Records whether login attempt was successful	A boolean data type is suitable as authentication only has two	True or False

Validation

1.Name

- Presence check: Field cannot be empty or contain only spaces
- Length check: name must be between 2 and 50 characters
- Whitespace check: spaces at beginning and end will be removed

2.Email address

- Presence check: Field cannot be empty
- or contain only spaces
- Format check: value must contain a @
- and domain
- Uniqueness check: a database query will check the address not already registered

3.Age

- Presence check: Field cannot
- be empty or contain only
- spaces Type check: data must be a whole number

4.Password

- Presence check: Field cannot be empty or contain only spaces
- Length check: password must be more than 8 characters

Registration

- Confirmation check: the confirmation field must exactly match original password -
  Security processing: the valid password will be hashed before it is inserted into the database and the original password will never be stored.

Login

- The password will be checked against the stored password hash rather than compared as plain text.

Registration outcome

- The account will be created if every field passes validation, individual messages will be shown if one or more fields are invalid.

Session check

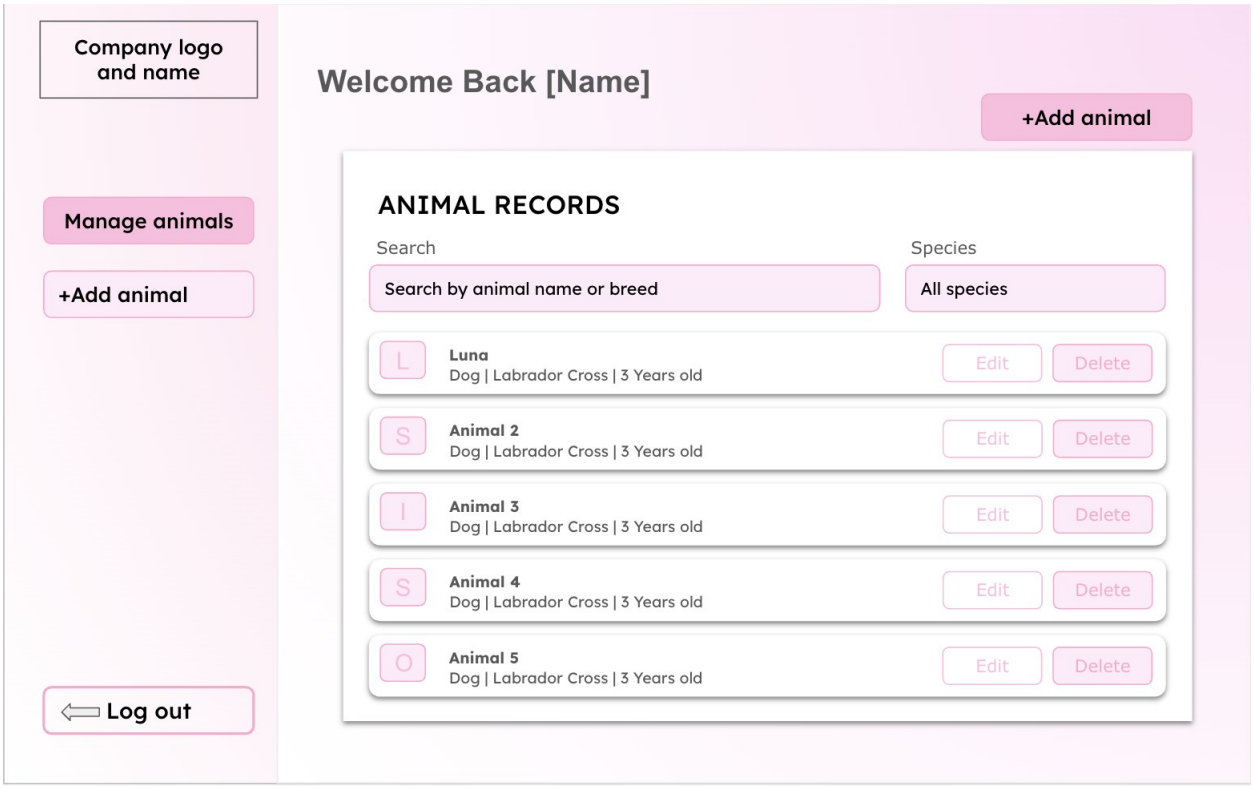
- A session will only be created after successful authentication. Invalid information will leave users on the login page and will not allow them to access the user or admin page.

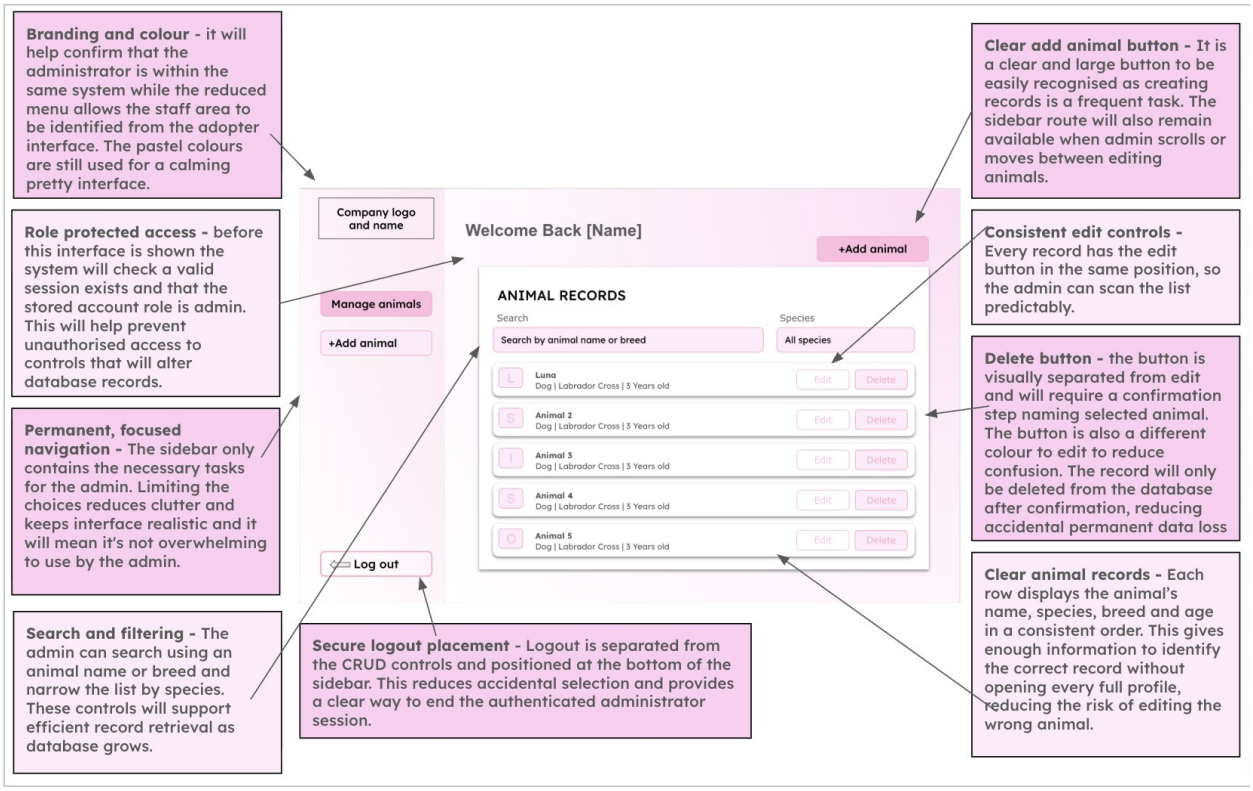
Admin Page

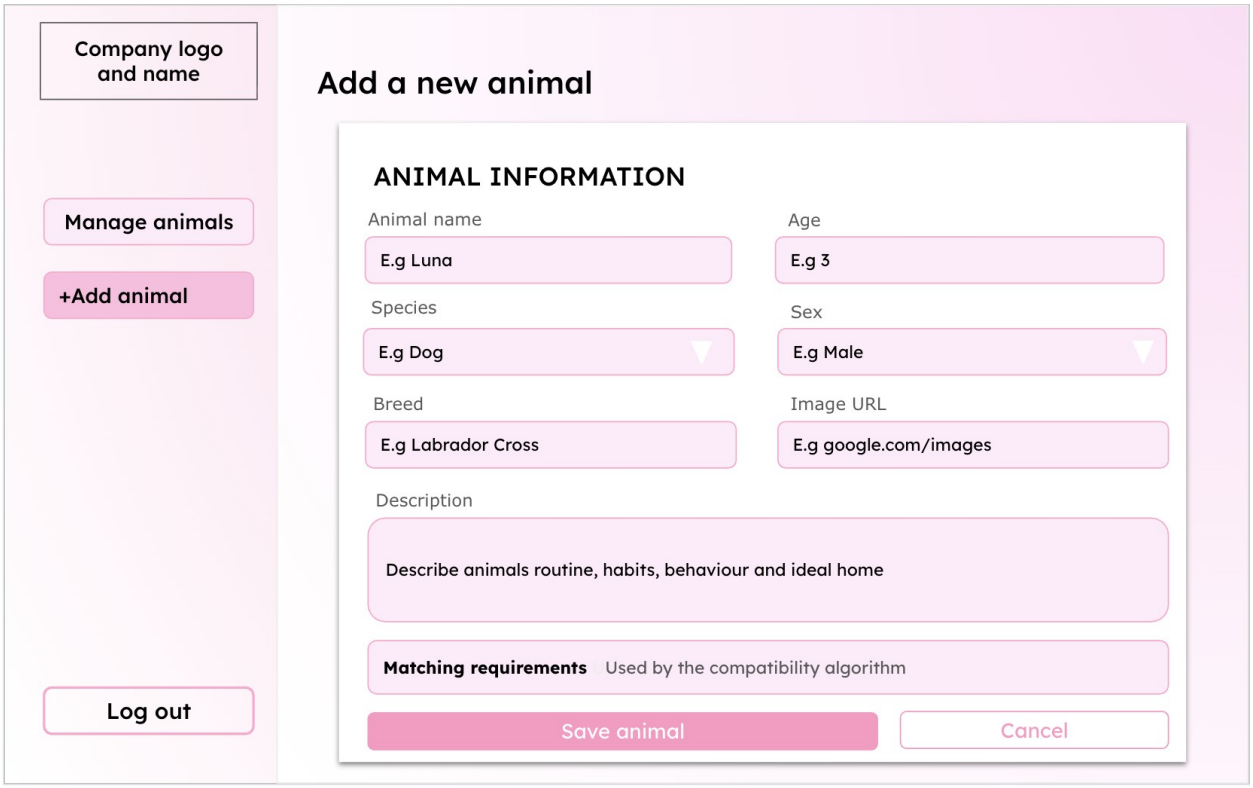
Admin Page

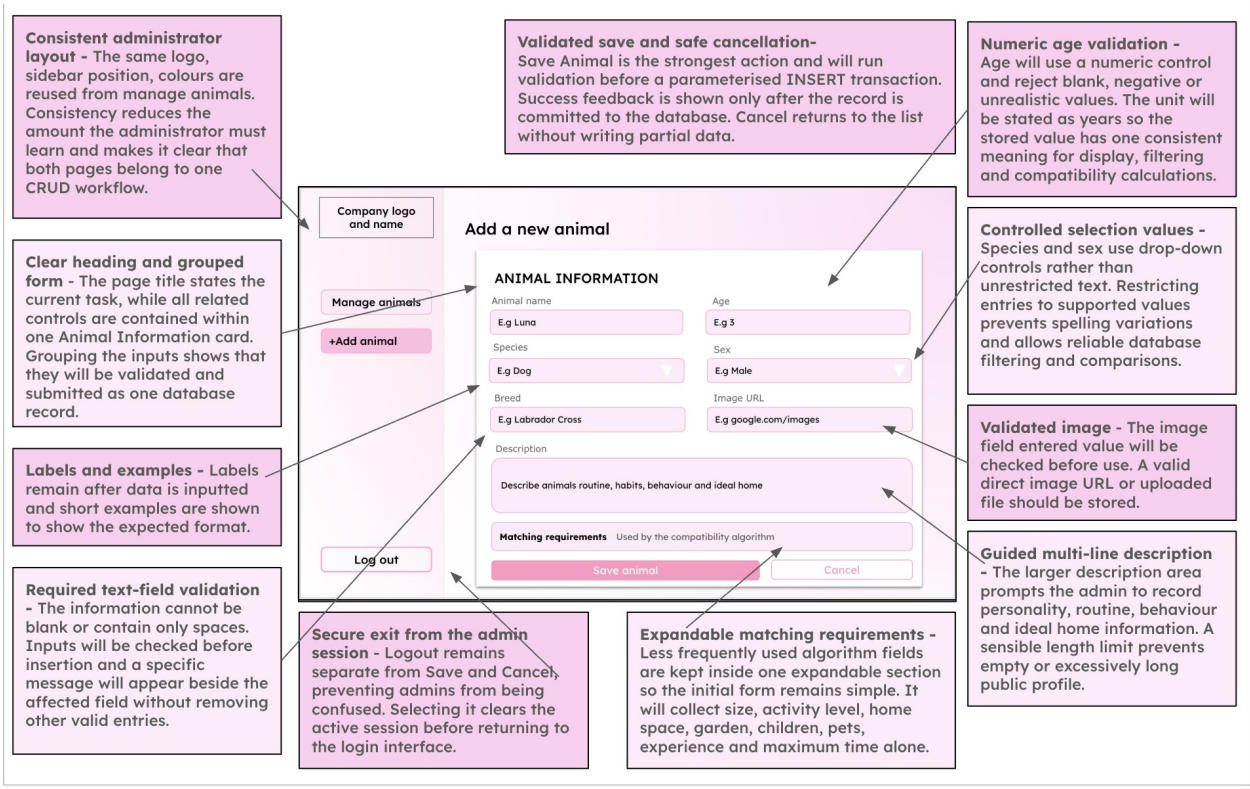
Administrator Menu	Provides staff with a dashboard to manage existing animals and their records. Admins will be clearly able to add new animals, edit existing details and remove records. To add animals it will store: Name, species, age, breed, sex, personality and exercise needs, children compatible, garden required, medical notes and an image. Forms will be clearly Labelled	The system will check that the loginned in account has the role before access is allowed. Parameterised SQL commands will insert, update and delete animal records. All inputs will be validated before changes are made to the database. Will also include a logout function, to securely log out so no physical access to the

Interface




Branding and colour - it will help confirm that the administrator is within the same system while the reduced menu allows the staff area to be identified from the adopter interface. The pastel colours are still used for a calming pretty interface.
Role protected access - before this interface is shown the system will check a valid session exists and that the stored account role is admin. This will help prevent unauthorised access to controls that will alter database records.
Permanent, focused navigation - The sidebar only contains the necessary tasks for the admin. Limiting the choices reduces clutter and keeps interface realistic and it will mean it's not overwhelming to use by the admin.
Search and filtering - The admin can search using an animal name or breed and narrow the list by species. These controls will support efficient record retrieval as database grows.
Company logo and name
Welcome Back [Name]
+Add animal
Manage animals
+Add animal
ANIMAL RECORDS
Search
Search by animal name or breed
Species
All species
Luna
Dog | Labrador Cross | 3 Years old
Animal 2
Dog | Labrador Cross | 3 Years old
Animal 3
Dog | Labrador Cross | 3 Years old
Animal 4
Dog | Labrador Cross | 3 Years old
Animal 5
Dog | Labrador Cross | 3 Years old
Edit
Delete
Log out
Secure logout placement - Logout is separated from the CRUD controls and positioned at the bottom of the sidebar. This reduces accidental selection and provides a clear way to end the authenticated administrator session.
Clear add animal button - It is a clear and large button to be easily recognised as creating records is a frequent task. The sidebar route will also remain available when admin scrolls or moves between editing animals.
Consistent edit controls - Every record has the edit button in the same position, so the admin can scan the list predictably.
Delete button - the button is visually separated from edit and will require a confirmation step naming selected animal. The button is also a different colour to edit to reduce confusion. The record will only be deleted from the database after confirmation, reducing accidental permanent data loss
Clear animal records - Each row displays the animal's name, species, breed and age in a consistent order. This gives enough information to identify the correct record without opening every full profile, reducing the risk of editing the wrong animal.


Company logo and name
Manage animals
+Add animal
Log out
Add a new animal
ANIMAL INFORMATION
Animal name
E.g Luna
Age
E.g 3
Species
E.g Dog
Sex
E.g Male
Breed
E.g Labrador Cross
Image URL
E.g google.com/images
Description
Describe animals routine, habits, behaviour and ideal home
Matching requirements
Used by the compatibility algorithm
Save animal
Cancel


Consistent administrator layout - The same logo, sidebar position, colours are reused from manage animals. Consistency reduces the amount the administrator must learn and makes it clear that both pages belong to one CRUD workflow.
Validated save and safe cancellation- Save Animal is the strongest action and will run validation before a parameterised INSERT transaction. Success feedback is shown only after the record is committed to the database. Cancel returns to the list without writing partial data.
Numeric age validation - Age will use a numeric control and reject blank, negative or unrealistic values. The unit will be stated as years so the stored value has one consistent meaning for display, filtering and compatibility calculations.
Company logo and name
Add a new animal
Manage animals
+Add animal
ANIMAL INFORMATION
Animal name
E.g Luna
Age
E.g 3
Species
E.g Dog
Sex
E.g Male
Breed
E.g Labrador Cross
Image URL
E.g google.com/images
Description
Describe animals routine, habits, behaviour and ideal home
Matching requirements
Used by the compatibility algorithm
Save animal
Cancel
Log out
Clear heading and grouped form - The page title states the current task, while all related controls are contained within one Animal Information card. Grouping the inputs shows that they will be validated and submitted as one database record.
Controlled selection values - Species and sex use drop-down controls rather than unrestricted text. Restricting entries to supported values prevents spelling variations and allows reliable database filtering and comparisons.
Labels and examples - Labels remain after data is inputted and short examples are shown to show the expected format.
Validated image - The image field entered value will be checked before use. A valid direct image URL or uploaded file should be stored.
Guided multi-line description - The larger description area prompts the admin to record personality, routine, behaviour and ideal home information. A sensible length limit prevents empty or excessively long public profile.
Required text-field validation - The information cannot be blank or contain only spaces. Inputs will be checked before insertion and a specific message will appear beside the affected field without removing other valid entries.
Secure exit from the admin session - Logout remains separate from Save and Cancel, preventing admins from being confused. Selecting it clears the active session before returning to the login interface.
Expandable matching requirements - Less frequently used algorithm fields are kept inside one expandable section so the initial form remains simple. It will collect size, activity level, home space, garden, children, pets, experience and maximum time alone.

Pseudocode

Model version addition

Pseudocode: staff add, edit and delete animal

```
FUNCTION ValidateAnimal(form)
    errors ← empty dictionary
    name ← TRIM(form.name)
    IF name = "" OR LENGTH(name) NOT IN 2..50 OR name has no letter
       OR name has a character other than letters, digits, spaces, ' and - THEN
        errors["name"] ← message
    ENDIF
    IF form.species NOT IN ["Dog", "Cat", "Rabbit"] THEN errors["species"] ← "Choose a
species."
    breed ← TRIM(form.breed)
    IF breed = "" OR LENGTH(breed) NOT IN 2..50 OR breed has no letter THEN errors["breed"] ←
message
    IF form.age is not a whole number OR form.age NOT IN 1..30 THEN errors["age"] ← message
    IF form.sex NOT IN ["Male", "Female", "Unknown"] THEN errors["sex"] ← "Choose a sex."
    IF form.size NOT IN ["Small", "Medium", "Large"] THEN errors["size"] ← "Choose a size."
    IF TRIM(form.personality) = "" OR LENGTH > 500 THEN errors["personality"] ← message
    IF form.exercise_needs NOT IN 1..3 THEN errors["exercise_needs"] ← message
    IF form.min_housing_level NOT IN 1..3 THEN errors["min_housing_level"] ← message
    IF form.garden_required NOT IN [Yes, No] THEN errors["garden_required"] ← message
    IF form.child_compatible NOT IN [Yes, No] THEN errors["child_compatible"] ← message
    IF form.max_hours_alone NOT IN 0..24 THEN errors["max_hours_alone"] ← message
    IF LENGTH(form.medical_notes) > 500 THEN errors["medical_notes"] ← message
    RETURN cleaned values, errors
ENDFUNCTION

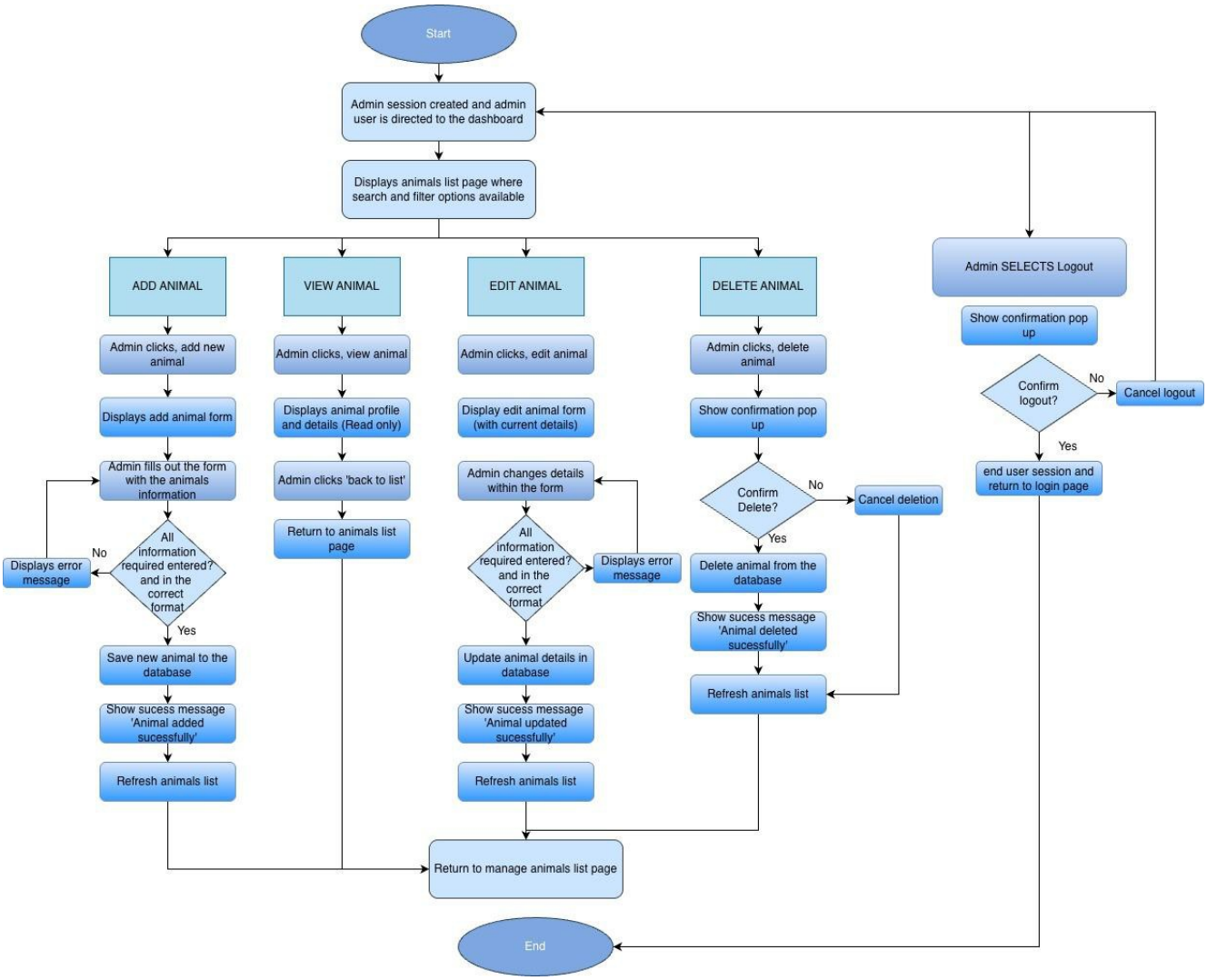
FUNCTION StoreUpload(upload)                              // optional photo
    IF upload = NULL THEN RETURN NULL
    IF EXTENSION(upload.name) NOT IN ["png", "jpg", "jpeg"] THEN RETURN "Images must be PNG
or JPG."
    IF upload.size > 2 MB THEN RETURN "Images must be 2 MB or smaller."
    safe_name ← RANDOM_HEX(32) + "." + EXTENSION(upload.name)   // never the user's file name
    WRITE upload TO "data/uploads/" + safe_name
    form.image_path ← "data/uploads/" + safe_name
    RETURN NULL
ENDFUNCTION

FUNCTION AddAnimal(actor, form)
    IF actor = NULL OR actor.user_role ≠ "admin" THEN RAISE AccessDenied   // checked again
here, not only on the page
    values, errors ← ValidateAnimal(form)
    IF errors is not empty THEN RETURN NULL, errors
    animal_id ← INSERT INTO animals (...14 fields..., date_added) VALUES (?, ?, ...)
    RETURN animal_id, empty dictionary
ENDFUNCTION

FUNCTION EditAnimal(actor, animal_id, form)
    IF actor.user_role ≠ "admin" THEN RAISE AccessDenied
    values, errors ← ValidateAnimal(form)
    IF errors is not empty THEN RETURN errors
    rows ← UPDATE animals SET name = ?, ... WHERE animal_id = ?
    IF rows = 0 THEN RETURN {"animal": "That animal no longer exists."}
    RETURN empty dictionary
ENDFUNCTION

FUNCTION DeleteAnimal(actor, animal_id, confirmed)
    IF actor.user_role ≠ "admin" THEN RAISE AccessDenied
    IF NOT confirmed THEN RETURN                          // Delete button disabled until the
box is ticked
    DELETE FROM animals WHERE animal_id = ?               // its stored match results go too
(CASCADE)
ENDFUNCTION
```

How it fits the whole solution. The staff page calls AddAnimal, EditAnimal and DeleteAnimal. Each one checks the role itself, so even if the page guard were bypassed nothing could be changed. ValidateAnimal is also used for the demo animals, so seed data obeys the same rules as staff input.



Animal Table

Field	Data type	Key/validation
animal_id	INTEGER	Primary key, autoincrement
name	TEXT	required
species	TEXT	Required - dog, cat or rabbit

breed	TEXT	Required
age	INTEGER	Required, cannot be zero
sex	TEXT	required- male, female or unknown
size	TEXT	Required - small medium or large
personality	TEXT	required
Exercise_needs	INTEGER	Required - 1 to 3
minimum_housing_level	INTEGER	Required - 1 to 3
garden_required	BOOLEAN	Required 0 or 1
child_compatible	BOOLEAN	Required 0 or 1
max_hours_left_alone	INTEGER	required
Medical_notes	TEXT	optional
image	TEXT	Required, an image path
date_added	DATE	required

Variables

Variable Name	Data Type	Purpose	Justification	Validation or e.g
animal_id	INTEGER	Uniquely identifies	A unique integer	Automatically

		each animal record	stops confusion between animals with similar names and allows the correct to be updated and deleted	generated
name	string	Stores the animals name	The name is required to identify the animal throughout the interface and allows the user to remember the animal with a specific name. A string is suitable as it allows letters spaces and	Can't be blank
species	string	Stores the type of animal	Species is required for searching, filtering and matching against user	dog, cat or rabbit
breed	string	Stores the animals breed	Breeds provides users with important information and may help describe likely size or care requirements	Can't be blank or might be 'unknown'
age	INTEGER	Stores the animals age	Age affects care needs. An integer is suitable as it allows the value to be validated sorted and compared	cannot be zero
sex	string	Stores the animals sex	This is relevant profile information and may be considered by the potential adopters	male, female
size	string	Stores the animals	Size affects space	small medium or large

		size and is used in filtering	requirements and is needed by the filtering and compatibility systems	
personality	string	Stores a description of the animals behaviour and personality	A text field will allow the admin to explain the animals personality and background in more detail to help attract possible adopters	Cant be blank but a character limit may be applied
Exercise_needs	INTEGER	Stores whether the animal has low, medium or high exercise needs	This can be compared with the adopters lifestyle to produce a more appropriate compatibility score. An integer is suitability for easier comparison	1 to 3 (low medium or high)
minimum_housing_level	INTEGER	Stores the amount of living space required	This can be compared with adopters lifestyle to aid the algorithm when creating the compatibility score	1 to 3
garden_required	boolean	Records whether the animal requires access to a garden	This may act as a compulsory suitability rule. A boolean value is useful as it is simple to input.	True or false
child_compatible	boolean	Records whether the animal is suitable for a household with children	There are only two options which make boolean values suitable. This is very important for the safety and welfare of the children	True or false

max_hours_left_alone	INTEGER	Stores value for which the animal can be left home alone for, and representing if more independent or clingy	This is important if a user works long hours and the animal requires a lot of attention it may cause neglect of the animal which is a key issue we are trying to prevent.	Cant be blank
Medical_notes	string/list	Stores any medical conditions, treatments or special requirements	Potential adopters need to understand any medical needs before making a decision. A string will allow multiple to be described.	May be blank or 'N/A'
image	string	Stores the location of the animals image	This allows the correct photo to be displayed without storing the full image directly in database	Must use an accepted file extension
form_action	string	Stores the operation the admin is performing	The same form may be used for adding and editing variables. This variable will allow the program to select the right query	'Add' or 'edit' or 'delete' or 'view'
delete_confirmation	boolean	Records whether the administrator has confirmed deletion	Deletion is permanent so confirmation reduces risk of animal being removed accidentally and not being considered for	True or false
record_found	boolean	Records whether the database has retrieved the animal record successfully	This prevents the program from attempting to edit or delete a record not	True or false

			present in the database	

Validation

1.Animal name

- Presence check: Cannot be empty
- Length check: Between 1 and 50 characters
- Format check: Letters, spaces, apostrophes,

a
l

- Presence check: Must be selected.
- Lookup check: Must be Dog, Cat or Rabbit.
- A drop-down menu will restrict the choices.

3.Breed

- Presence check: Cannot be empty
- Length check: For example, between 2 and 50
- Format check: Must not consist entirely of

4.Age

- Presence check: Must be provided
- Type check: Must be a whole number
- Range check: Cannot be negative and must be

age

5

- Lookup check: Must be Male, Female or Unknown
- A drop-down menu prevents invalid values

6.Size

- Lookup check: Must be Small, Medium or Large

7.Personality

- Presence check: A description must be provided
- Length check: For example, between 20 and 1,000 characters
- Whitespace check: A value consisting only of spaces will be rejected

8.Exercise need

- Presence check: An
- option must be

selected Range

9. Minimum housing requirement

- Presence check: Must be selected
- Range check: Must be between 1 and 3

10. Maximum hours alone

- Type check: Must be an integer
- Range check: Must be between 0 and 24

11. Garden required

- Type check: Must be stored as a Boolean value 0 or 1

12. Child compatibility

- Type check: Must be stored as a Boolean value 0 or 1

13. Medical notes

- This field may be optional
- Type check: it will have a maximum length
- Text will be stored as plain text rather than interpreted as code

14. Animal image

- Presence check : An image must be uploaded if every profile requires one
- File-type check : Only approved formats such as JPG, JPEG and PNG will be accepted
- File-size check : The file must not exceed the chosen maximum size
- Filename handling : A safe, unique filename will be generated to avoid overwriting existing images

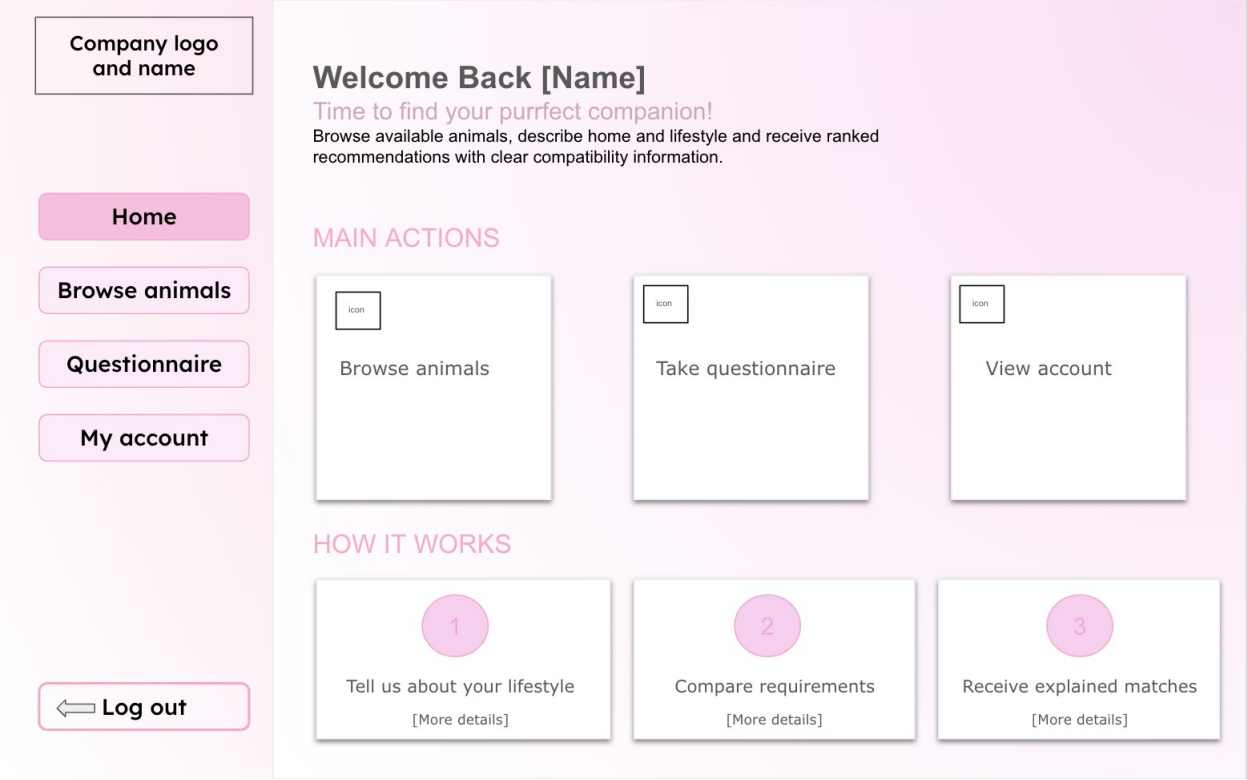
The animal record will only be inserted after all required fields pass validation. The SQL statement will be parameterised and committed only after successful validation. A success message will then confirm that the record has been created.

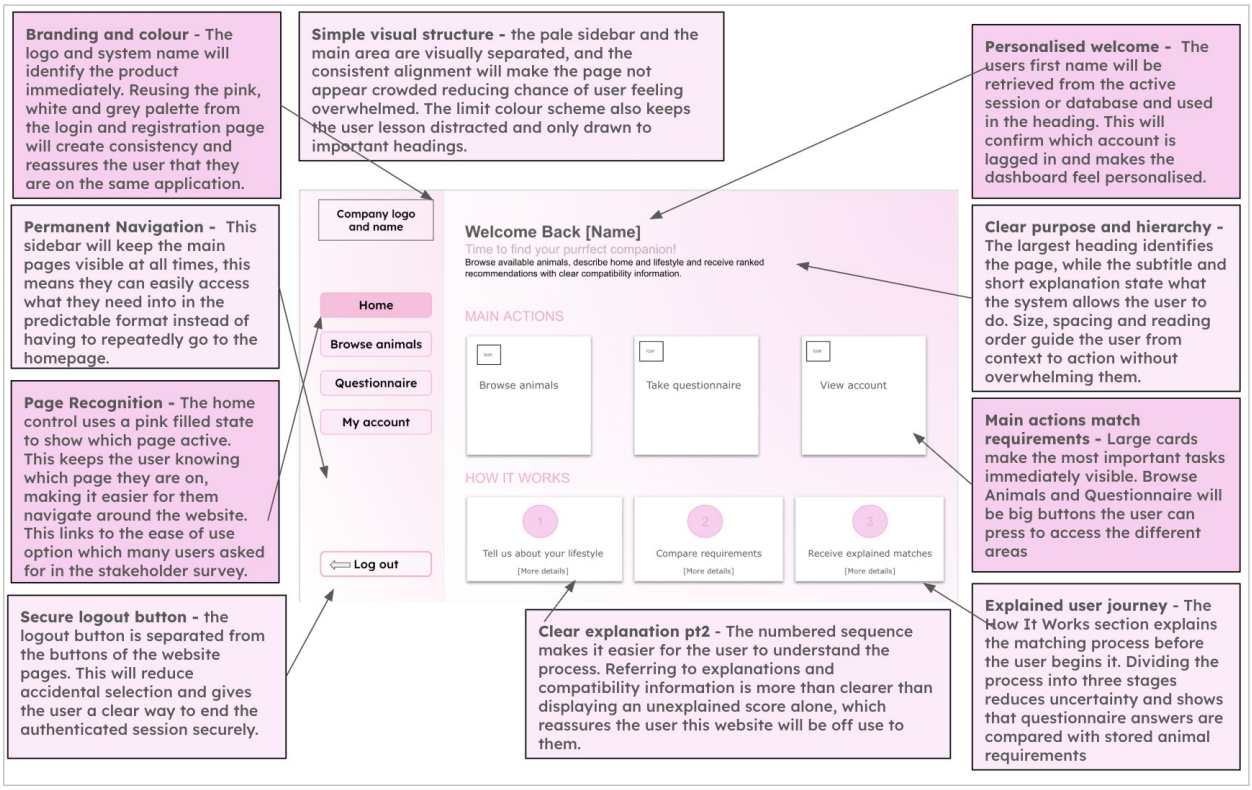
Main Menu/INterface

Main Menu

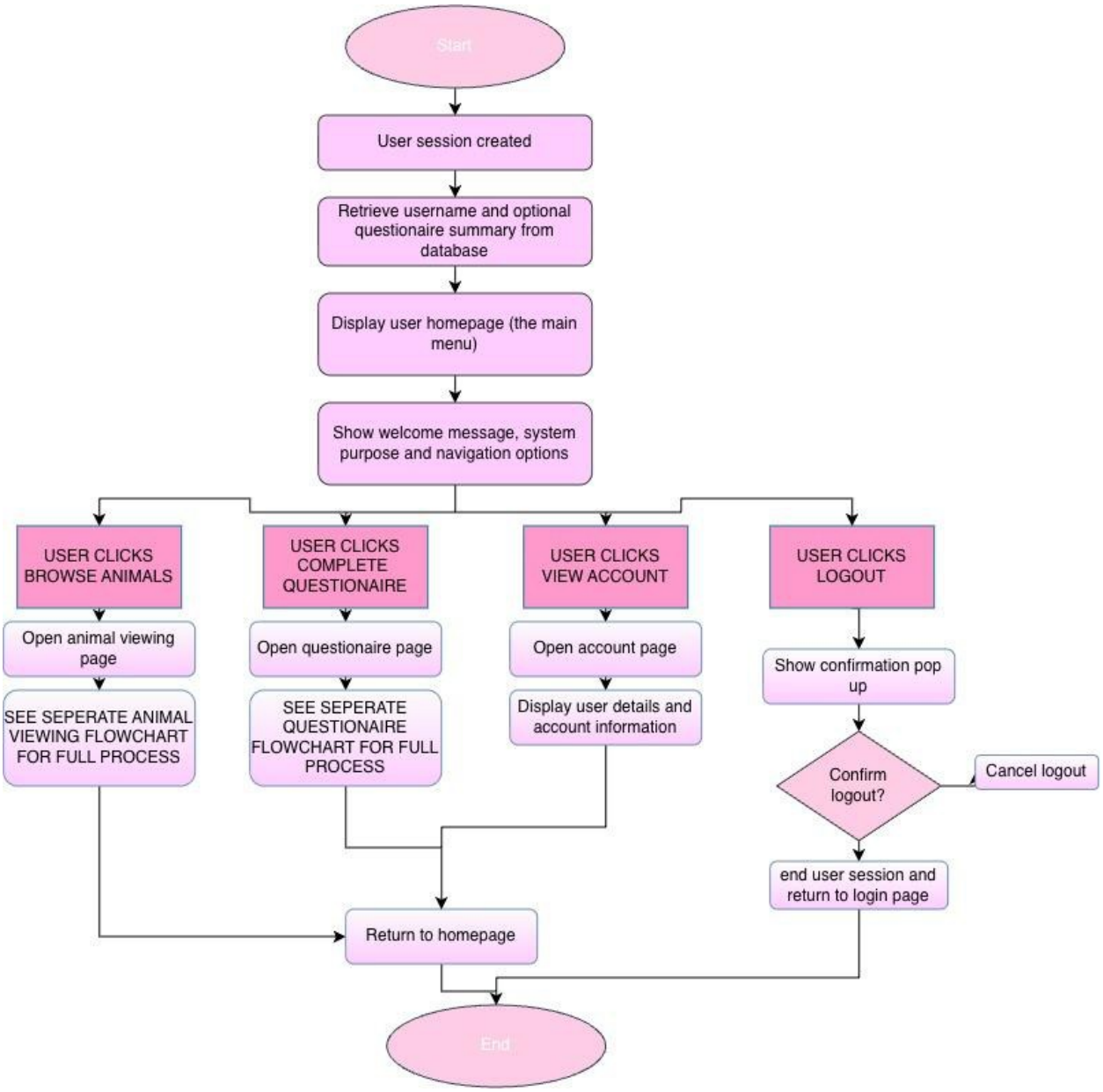
User Homepage/Main Menu	Will provide the main navigation area after the user logs in. It will allow the user to access the questionnaire, browse available animals and view their account. It will also explain the purpose of the adoption matching system. It will be clear and	The system will check the user's session to make sure they are logged in. It may retrieve the user's name and recent questionnaire information from the database so that personalised information can be displayed.

Interface Design





Algorithm (Flow chart, pseudocode)


Start
User session created
Retrieve username and optional questionaire summary from database
Display user homepage (the main menu)
Show welcome message, system purpose and navigation options
USER CLICKS BROWSE ANIMALS
USER CLICKS COMPLETE QUESTIONAIRE
USER CLICKS VIEW ACCOUNT
USER CLICKS LOGOUT
Open animal viewing page
Open questionaire page
Open account page
Show confirmation pop up
SEE SEPERATE ANIMAL VIEWING FLOWCHART FOR FULL PROCESS
SEE SEPERATE QUESTIONAIRE FLOWCHART FOR FULL PROCESS
Display user details and account information
Confirm logout?
Cancel logout
end user session and return to login page
Return to homepage
End

Pseudocode

Model version addition

Pseudocode: menu, home page and 'My matches' entry

```
// streamlit_app.py runs first on every page load
PROCEDURE BuildMenu()
    PrepareDatabaseOnce()                              // create tables and demo animals if
missing
    user ← CurrentUser()
    pages ← [Home, Animals, Animal profile (hidden), Questionnaire, My matches]
    IF user = NULL THEN
        APPEND "Log in" TO pages
    ELSE
        APPEND "My account" TO pages
    ENDIF
    IF user ≠ NULL AND user.user_role = "admin" THEN
        APPEND "Manage animals" TO pages               // the staff page does not exist for
anyone else
    ENDIF
    selected_page ← page chosen in the top menu (Home by default)
    RUN selected_page
ENDPROCEDURE

PROCEDURE HomePage()
    SHOW title, purpose and "Scores are guidance only. Shelter staff make the final
decision."
    IF "Start the questionnaire" clicked THEN GO TO Questionnaire
    animal_results ← SELECT * FROM animals
    FOR species IN ["Dog", "Cat", "Rabbit"]
        SHOW COUNT of animal_results with that species
    NEXT species
    IF CurrentUser() = NULL THEN SHOW link "Log in or register"
ENDPROCEDURE

PROCEDURE MyMatchesPage()
    user ← RequireLogin()
    latest ← SELECT newest response FROM questionnaire_responses WHERE user_id = ?
    IF latest = NULL THEN                              // questionnaire not completed yet
        SHOW "You haven't completed the questionnaire yet." and a link to it
        STOP
    ENDIF
    results ← stored match_results for latest, ORDER BY rank
    SHOW first MAX(4, 5) results (all of them if "Show every animal" is on)
ENDPROCEDURE
```

How it fits the whole solution. BuildMenu runs on every page load. Because the staff page is only added for admins, adopters never see a link they cannot use. The questionnaire-completed check sits on the My matches page: users who have not finished it are sent to it.

Variables

Variable Name	Data Type	Purpose	Justification	E.g or validation
current_user_id	INTEGER	Identifies user currently accessing dashboard	Ensures that the dashboard displays information belonging to the correct account	Retrieved from session
current_user_name	STRING	Stores the logged in users name	This will allow the interface to display personalised information and reassures user they are accessing correct account	'Welcome sienna'
animal_count	INTEGER	Stores the number of animals currently available	It will provide a useful summary of information on the dashboard. An integer is a suitable data type as the number of animals is a	Must be zero or larger
questionaire_completed	BOOLEAN	Records whether the user has completed the questionnaire	This variable allows the system to display recommendations or prompt the user to complete the questionnaire. Boolean is suitable as there is only two possible states	True or False
reccomended_animals	LIST	Stores the animals recommended to the users	Multiple animals will need to be displayed in ranked order so a list is a suitable data type	Ordered from lowest to highest compatibility

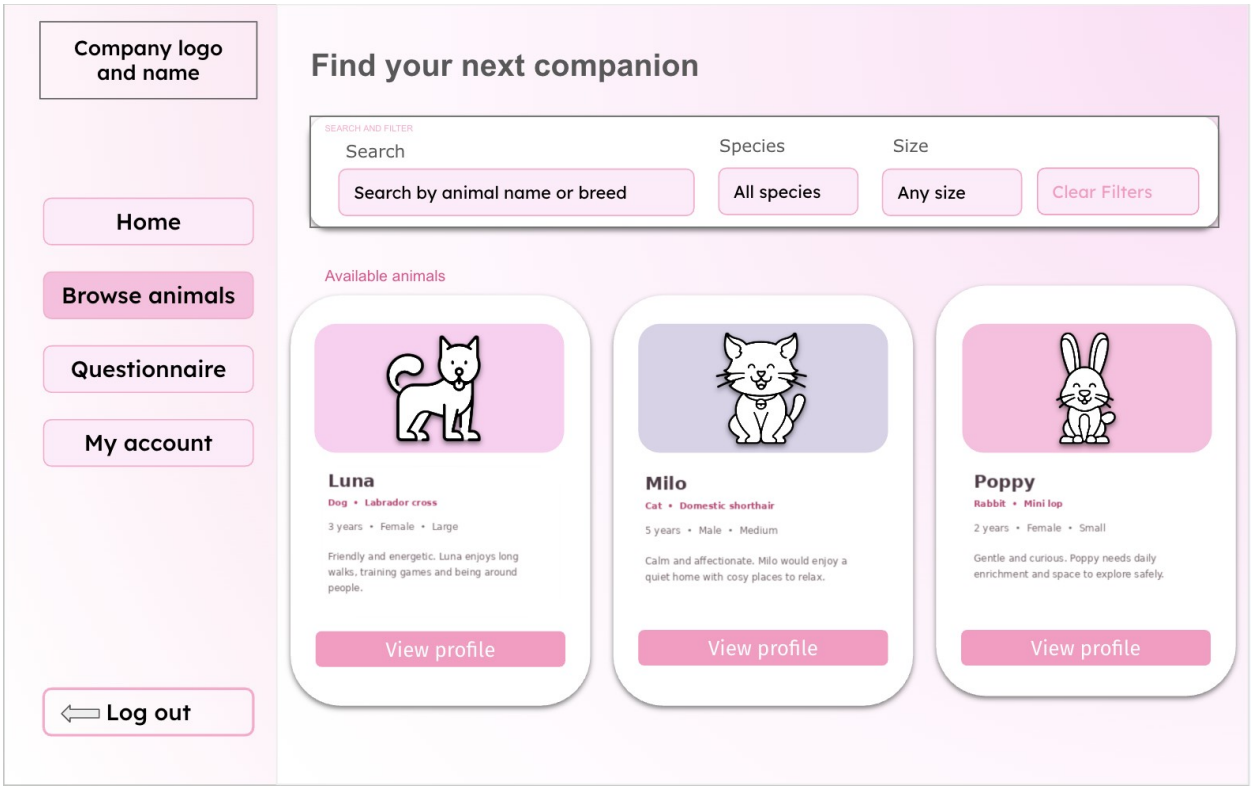
selected_page	STRING	Stores the page selected from navigation menu	This allows the program to choose which page or section should be selected	Animals or questionaire
notification_messa g e	STRING	Stores confirmation or status feedback	The user will need clear feedback after completing actions such as submitting questionnaire	‘Questionaire submitted successfully’

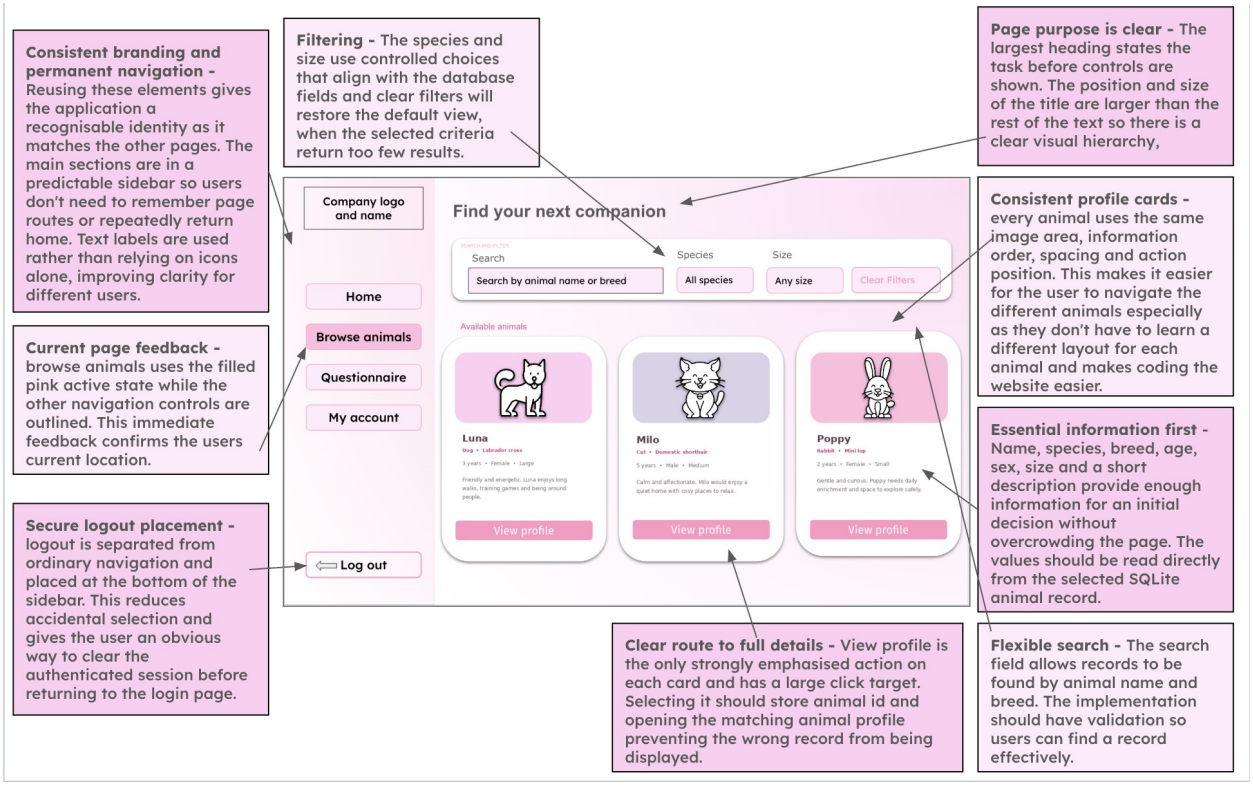
Animal Viewing Page

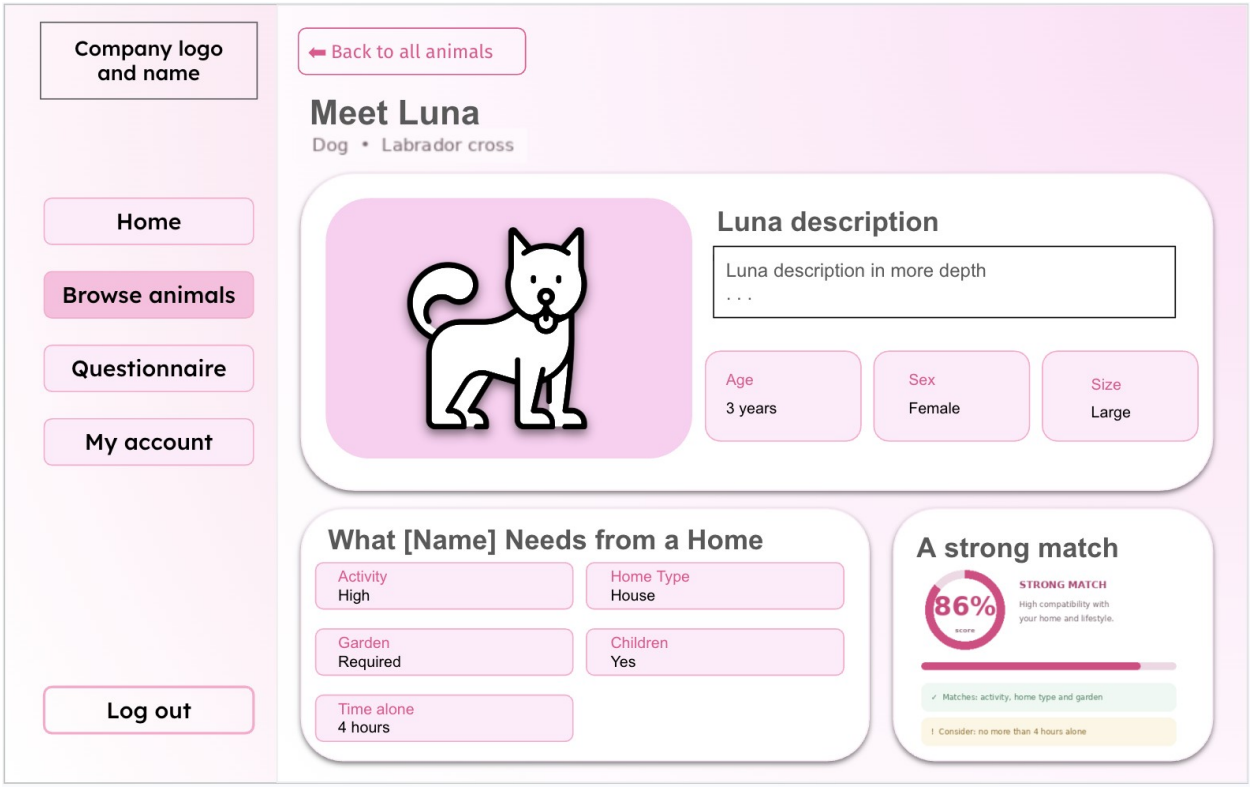
Animal Viewing Page

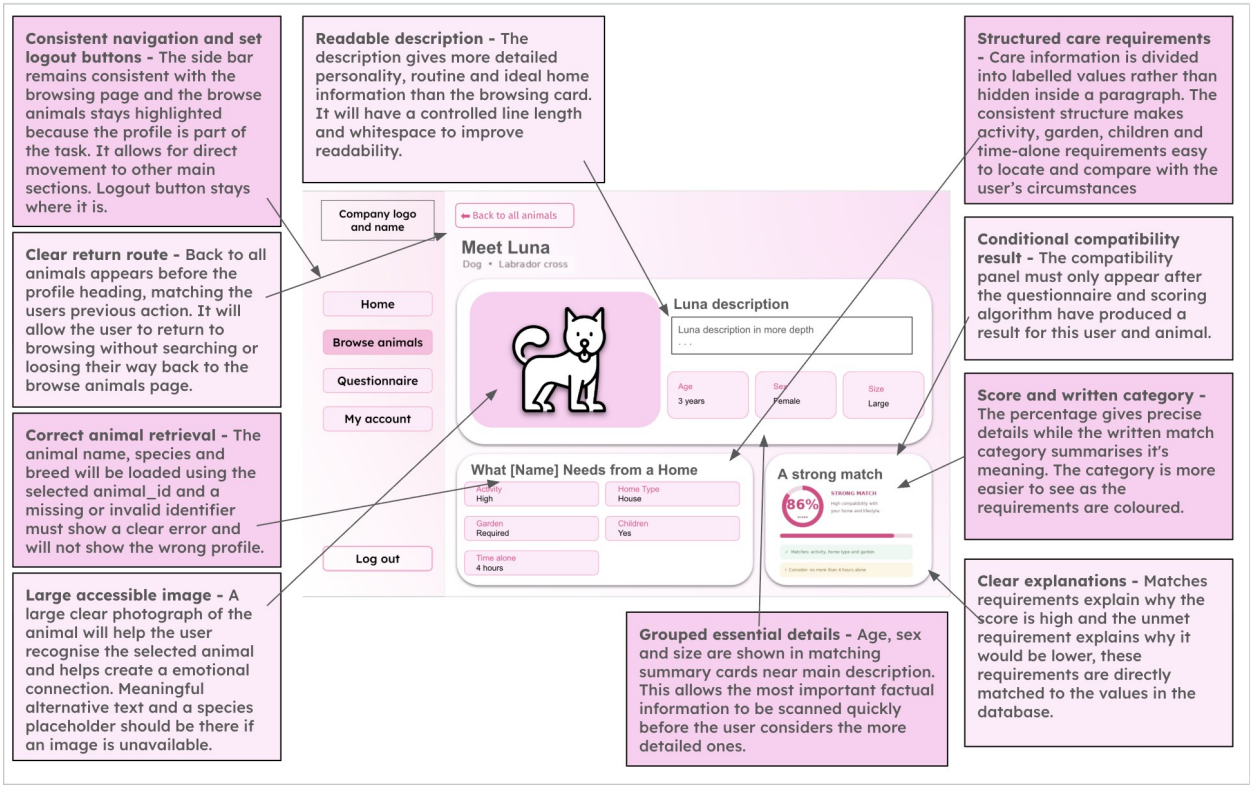
Animal Viewing	Allows users to browse available animals using clearly presented animal cards. Each card will show information such as the animal name, image, species, breed and age. Users will be able to select a card to view a more detailed animal profile.	Animal records will be retrieved from the animals table. Search and filter selections will be used to change the database query or filter the retrieved records. Selecting an animal will use its unique animal ID to load the correct profile.

Interface Design

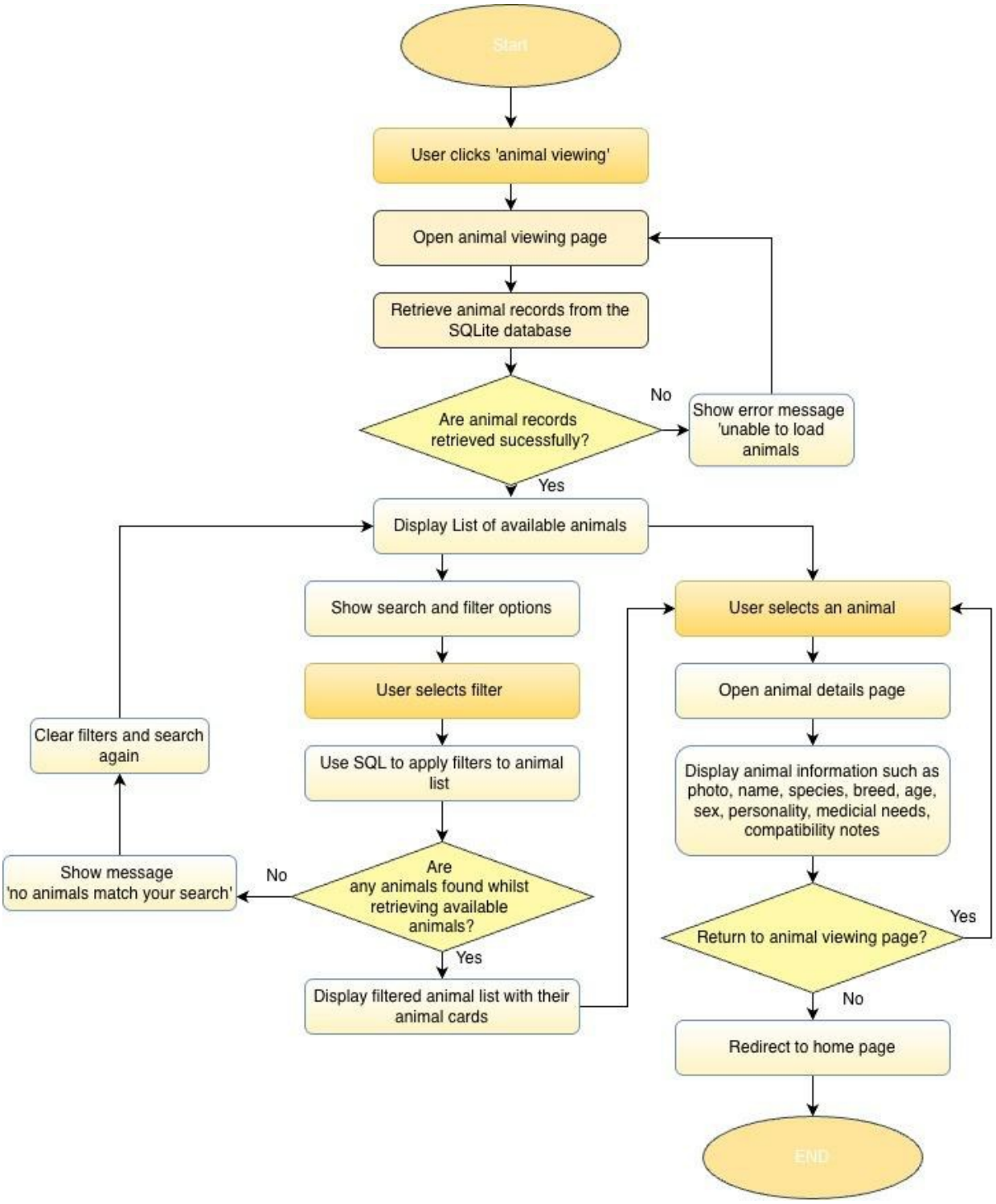



Consistent branding and permanent navigation - Reusing these elements gives the application a recognisable identity as it matches the other pages. The main sections are in a predictable sidebar so users don't need to remember page routes or repeatedly return home. Text labels are used rather than relying on icons alone, improving clarity for different users.
Filtering - The species and size use controlled choices that align with the database fields and clear filters will restore the default view, when the selected criteria return too few results.
Page purpose is clear - The largest heading states the task before controls are shown. The position and size of the title are larger than the rest of the text so there is a clear visual hierarchy,
Company logo and name
Find your next companion
Search
Species
Size
Search by animal name or breed
All species
Any size
Clear Filters
Home
Browse animals
Questionnaire
My account
Available animals
Luna
Milo
Poppy
View profile
View profile
View profile
Log out
Consistent profile cards - every animal uses the same image area, information order, spacing and action position. This makes it easier for the user to navigate the different animals especially as they don't have to learn a different layout for each animal and makes coding the website easier.
Current page feedback - browse animals uses the filled pink active state while the other navigation controls are outlined. This immediate feedback confirms the users current location.
Essential information first - Name, species, breed, age, sex, size and a short description provide enough information for an initial decision without overcrowding the page. The values should be read directly from the selected SQLite animal record.
Secure logout placement - logout is separated from ordinary navigation and placed at the bottom of the sidebar. This reduces accidental selection and gives the user an obvious way to clear the authenticated session before returning to the login page.
Clear route to full details - View profile is the only strongly emphasised action on each card and has a large click target. Selecting it should store animal id and opening the matching animal profile preventing the wrong record from being displayed.
Flexible search - The search field allows records to be found by animal name and breed. The implementation should have validation so users can find a record effectively.


Company logo and name
Back to all animals
Meet Luna
Dog • Labrador cross
Home
Browse animals
Questionnaire
My account
Log out
Luna description
Luna description in more depth
. . .
Age
3 years
Sex
Female
Size
Large
What [Name] Needs from a Home
Activity
High
Home Type
House
Garden
Required
Children
Yes
Time alone
4 hours
A strong match
86%
STRONG MATCH
High compatibility with your home and lifestyle.
✓ Matches: activity, home type and garden
! Consider: no more than 4 hours alone



Algorithm (Flow chart, pseudocode)


Start
User clicks 'animal viewing'
Open animal viewing page
Retrieve animal records from the SQLite database
Are animal records retrieved sucessfully?
No
Show error message 'unable to load animals
Yes
Display List of available animals
Show search and filter options
User selects filter
Use SQL to apply filters to animal list
Are any animals found whilst retrieving available animals?
No
Show message 'no animals match your search'
Clear filters and search again
Yes
Display filtered animal list with their animal cards
User selects an animal
Open animal details page
Display animal information such as photo, name, species, breed, age, sex, personality, medicial needs, compatibility notes
Return to animal viewing page?
Yes
No
Redirect to home page
END

Pseudocode

Model version addition

Pseudocode: browse, filter, search and open a profile

```
PROCEDURE AnimalViewingPage()
    species_filter ← drop-down: All species / Dog / Cat / Rabbit
    size_filter ← drop-down: All sizes / Small / Medium / Large
    search_term ← text box (at most 20 characters)
    animal_results ← ListAnimals(species_filter, size_filter, search_term)
    no_results ← LENGTH(animal_results) = 0
    IF no_results THEN SHOW "No animals match those filters."
    FOR EACH animal IN animal_results                    // 4 cards per row
        SHOW card: image, name, species, breed, age, "View profile" button
        IF "View profile" clicked THEN
            selected_animal_id ← animal.animal_id
            GO TO ProfilePage(id = selected_animal_id, back = "browse")
        ENDIF
    NEXT animal
ENDPROCEDURE

FUNCTION ListAnimals(species_filter, size_filter, search_term)
    sql ← "SELECT * FROM animals WHERE 1 = 1"
    params ← empty list
    IF species_filter chosen THEN sql ← sql + " AND species = ?"; APPEND species_filter TO
params
    IF size_filter chosen THEN sql ← sql + " AND size = ?"; APPEND size_filter TO params
    search_term ← TRIM(search_term)
    IF search_term ≠ "" THEN                             // empty search = show every animal
        sql ← sql + " AND (name LIKE ? OR breed LIKE ?)"
        APPEND "%" + search_term + "%" TO params (twice)  // the text is data, never part of
the SQL
    ENDIF
    RETURN EXECUTE(sql + " ORDER BY species, name", params)
ENDFUNCTION

PROCEDURE ProfilePage(id, back)
    SHOW link "Back to animals" (or "Back to my matches" if back = "results")
    IF id is not a whole number THEN animal ← NULL ELSE animal ← SELECT * FROM animals WHERE
animal_id = ?
    IF animal = NULL THEN SHOW "That animal could not be found. It may have been adopted.";
STOP
    SHOW image, name, species, breed, age, sex, size, personality, medical needs
    SHOW needs: exercise level, minimum home, garden, children, maximum hours alone
ENDPROCEDURE
```

How it fits the whole solution. The same ListAnimals function serves Home (counts), the animal list and the staff list. The profile is opened by animal_id, so two animals with the same name can never be confused.

Variables

Variable Name	Data Type	Purpose	Justification	E.g or validation
search_term	String	Stores text entered into the search bar	A string allows users to search using names, breeds or other information	Leading and trailing spaces should be removed
species_filter	String	Filters animals by species	This is required to reduce the displayed records to animals matching the users chosen	'Dog', 'cat', 'rabbit' are the specified filters
size_filter	String	Filters animals by size	Size may affect whether an animal is suitable for the users home so it is suitable for narrowing the	'Small', 'medium', large'
sort_option	String	Determines the order of results	Users may wish to order the results by suitability or animal information. The value determines which sorting	'Compatibility'
animal_results	list	Holds the records returned from the database	A list is suitable because a search may return multiple animal records. It can be iterated through to generate animal cards on the	May be empty is no matching results
selected_animal_id	integer	Identifies the animal whose profile is opened	Using the unique id ensures the correct animal	Must match an existing record

			even when animals have the same name	
compatibility_score	float	Displays or sorts by compatibility	A float is a more suitable data type than an integer as it allows the result to include decimal values, which will produce a more precise % than an integer.	Between 0 and 100
no_results	boolean	Feedback displayed if there is any animals which meet the filters	Without feedback an empty page could appear to be an error, so the message informs the user that they should change	'No animals match your search'

Validation

1.Search input

- Length check: the text will not exceed 20 characters
- Whitespace check: leading and trailing spaces will be removed
- Empty input handling: an empty search will be treated as a request to display all available animals rather than as an error

2.Filter inputs

Drop down menus or selection controls will be used for:

- Species
  - Dog
  - Cat
  - Rabbit
- Size
  - Small
  - Medium

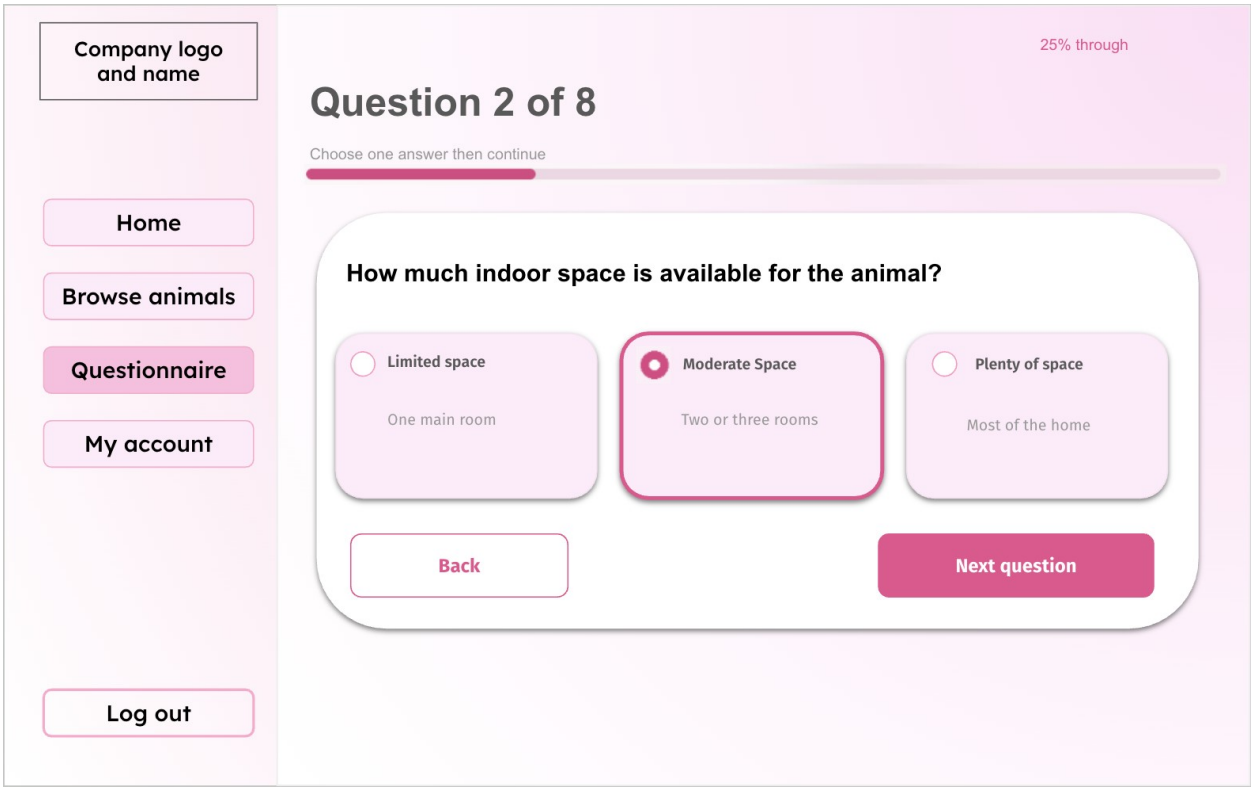
- Large

Compatibility algorithms

Compatibility algorithm and questionnaire

Questionnaire and Recommendations	Will use a form in order to allow users to answer questions about their home, lifestyle, experience and preferences. The questionnaire will have questions such as the housing type, access to garden, animal loneliness times, children, pets, experience, activity labels, what species and size preferred. These questions would not appear all at one time to not overwhelm the user and the input type would be drop downs or radio buttons. After submitting the questionnaire, the system will display	Responses will be validated and compared with the requirements of each available animal. A weighted compatibility algorithm will calculate a percentage score for every animal. The animals will then be sorted from highest to lowest score so the top recommendations can be displayed. The questionnaire responses and match results may also be stored within the database with the users ID

Interface Design

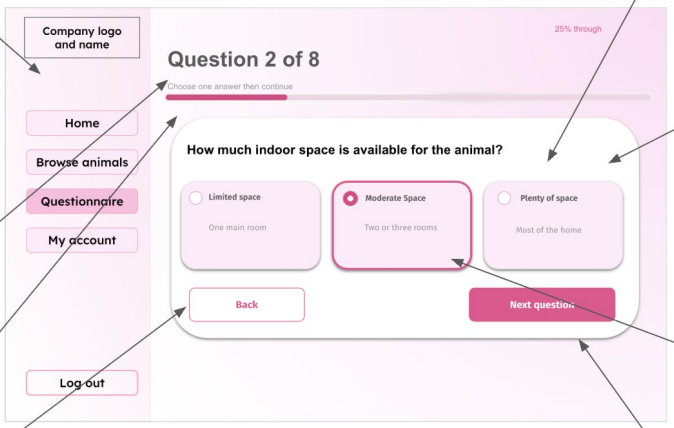


Consistent navigation and set logout buttons - The side bar remains consistent, and the questionnaire section is now highlighted, which is especially useful as this page is a multistep section. It allows for direct movement to other main sections and easier familiarisation with the website. Logout button stays where it is.

Clear question position - The large 'question 2 of 8' at the top of the page, tells the user exactly where they are in the sequence. Showing both the current step and total number makes task feel finite and allows the remaining effort to be estimated.

Visible progress feedback - the progress bar shows that the system has recorded movement through the questionnaire.

Safe backwards - Back is visually next to next question, previously selected answers should remain in session state allowing the user to review or change a response without losing later questionnaire data

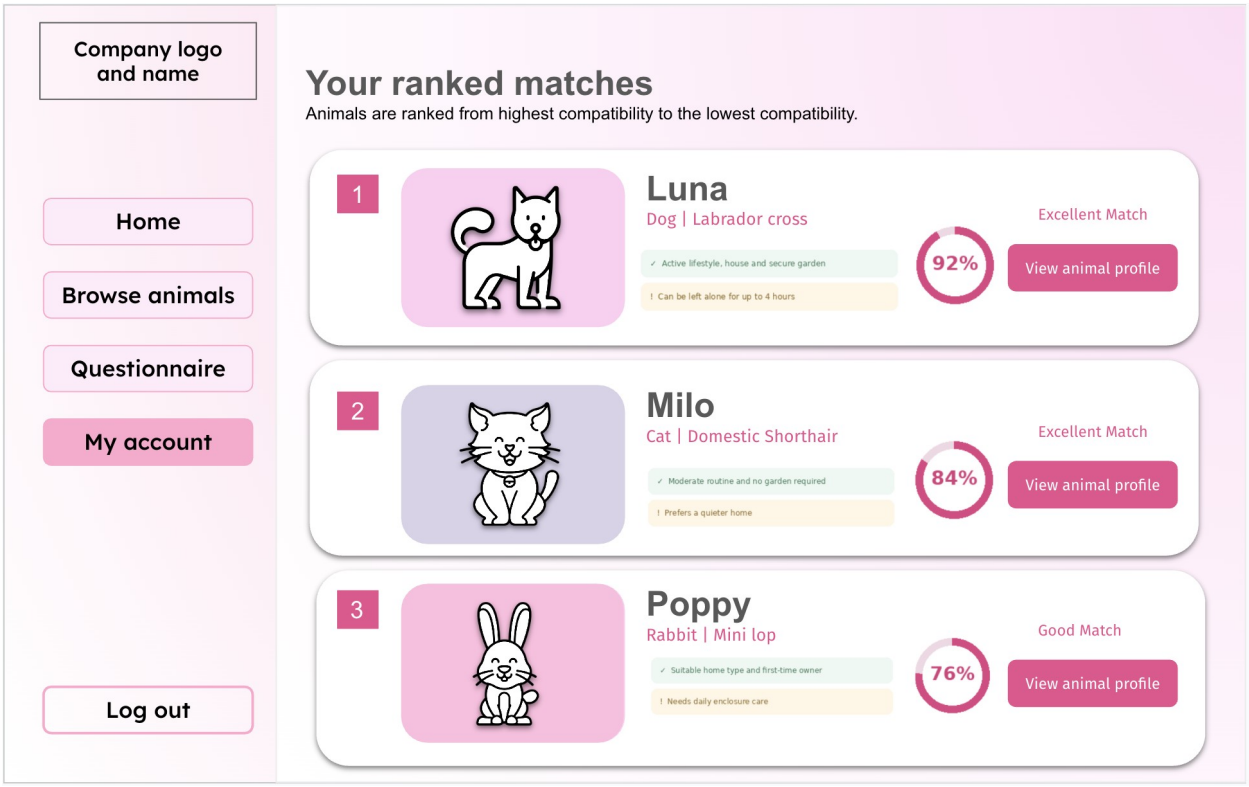


Clear options in their own boxes - Limited, moderate and plenty of space use a single choice control because only one answer should be stored. The room examples make the categories more objective and reduce different interpretation of vague words.

One question at a time - Displaying one question at a time removed unrelated actions and buttons and reduces cognitive load. The large central card focuses the attention on current decision and is easier to use. This links to the examples of competitors and their designs and the stakeholder survey and client with the website being easy to use.

Clear selected state - The chosen card uses a filled radio marker so selection will not change on a subtle colour change so it is very clear to the user what they have chosen as an option.

Validated progression- Next question should only progress if an answer is selected. A error message is shown if the user attempt to continue without responding.



Consistent navigation - The ranked-results page reuses the same sidebar, palette and card styling as the other user interfaces. This provides continuity after questionnaire submission and reassures the user that the results belong to this website

Clear ranking explanation - The heading and subtitle state that results are ordered from highest to lowest compatibility. This explains the purpose of the page before the user examines individual scores.

Unique rank numbers - The animals are labelled 1 2 3 are in order so they can easily see the clear order and ensure its the right order.

Consistent result cards - Each animal uses the same sequence of rank, image, identity, explanations, score and action. A repeated layout allows rapid comparison and ensures that there is no animal stands out more and differently.

Connected full profile - View animal profile stores selected animal_id and opens the corresponding full page. The detailed page then can show every care requirements without making the ranked result too long or difficult to compare.

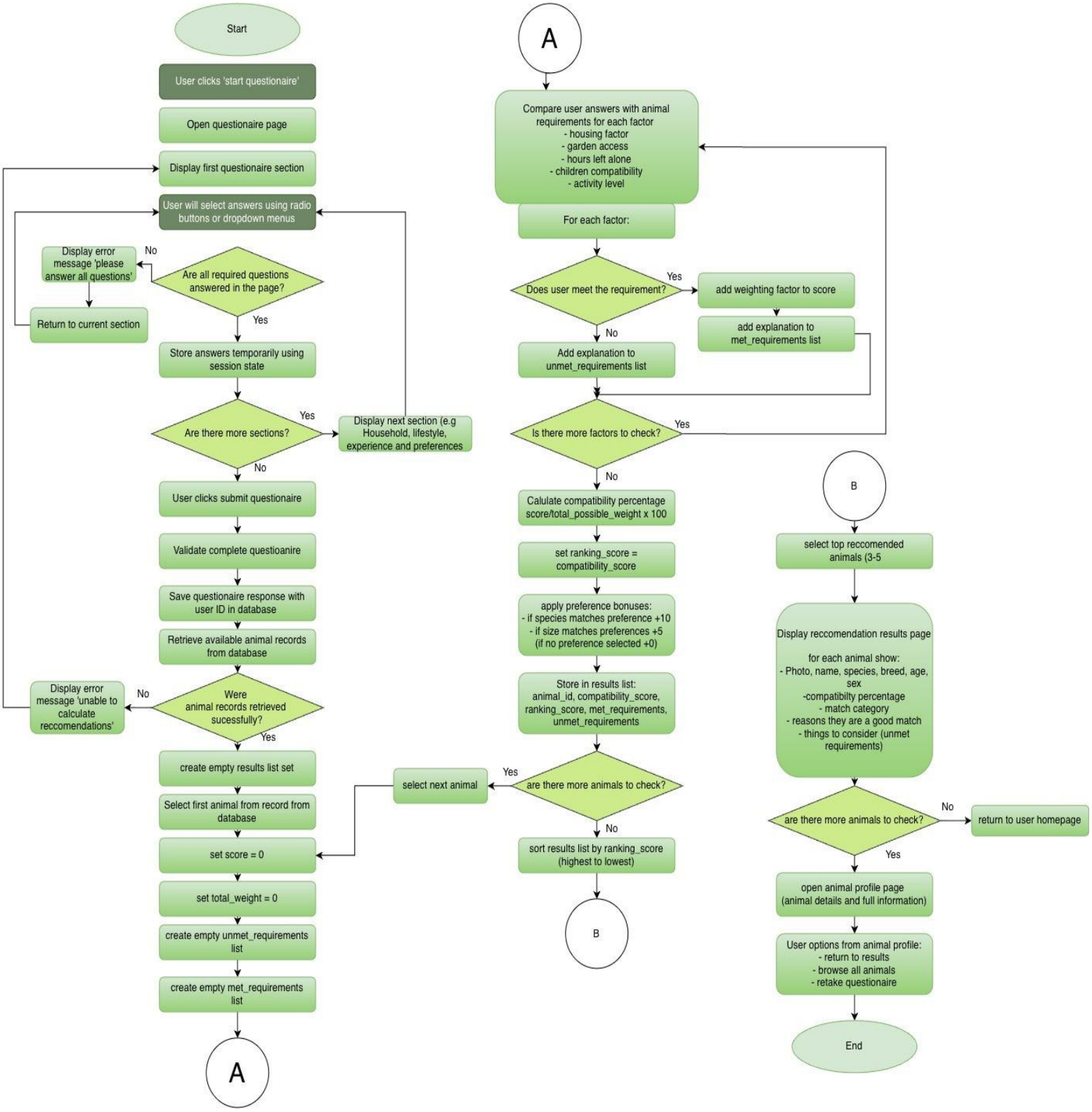
Animal identity remains clear - The image, name, species and breed make each result identifiable before the user interprets the score. These values should be retrieved from the available-animal records rather than copied into the results interface.

Sorted algorithm output - compatibility scores should be calculated for every available animal, stored against its animal_id and sorted in descending order before cards are displayed.

Consistent score bands - A quick way of seeing if its suitable instead of looking at the percentages and being upset if its a bit less than another animal.

Matched requirements explained (and unmatched) - The green explanation identifies the strongest reasons for the recommendation, such as activity, home space or garden access. This provides evidence for the score rather than asking the user to trust an unexplained percentage.

Algorithm (Flow chart, pseudocode)


Start
User clicks 'start questionaire'
Open questionaire page
Display first questionaire section
User will select answers using radio buttons or dropdown menus
Are all required questions answered in the page?
No
Display error message 'please answer all questions'
Return to current section
Yes
Store answers temporarily using session state
Are there more sections?
Yes
Display next section (e.g Household, lifestyle, experience and preferences
No
User clicks submit questionaire
Validate complete questioanire
Save questionaire response with user ID in database
Retrieve available animal records from database
Were animal records retrieved sucessfully?
No
Display error message 'unable to calculate reccomendations'
Yes
create empty results list set
Select first animal from record from database
set score = 0
set total_weight = 0
create empty unmet_requirements list
create empty met_requirements list
A
A
Compare user answers with animal requirements for each factor
- housing factor
- garden access
- hours left alone
- children compatibility
- activity level
For each factor:
Does user meet the requirement?
Yes
add weighting factor to score
add explanation to met_requirements list
No
Add explanation to unmet_requirements list
Is there more factors to check?
Yes
No
Calulate compatibility percentage score/total_possible_weight x 100
set ranking_score = compatibility_score
apply preference bonuses:
- if species matches preference +10
- if size matches preferences +5
(if no preference selected +0)
Store in results list:
animal_id, compatibility_score, ranking_score, met_requirements, unmet_requirements
are there more animals to check?
Yes
select next animal
No
sort results list by ranking_score (highest to lowest)
B
B
select top reccomended animals (3-5
Display reccomendation results page
for each animal show:
- Photo, name, species, breed, age, sex
-compatibilty percentage
- match category
- reasons they are a good match
- things to consider (unmet requirements)
are there more animals to check?
No
return to user homepage
Yes
open animal profile page (animal details and full information)
User options from animal profile:
- return to results
- browse all animals
- retake questionaire
End

Pseudocode

Model version addition

Pseudocode: questionnaire, compatibility score and ranking

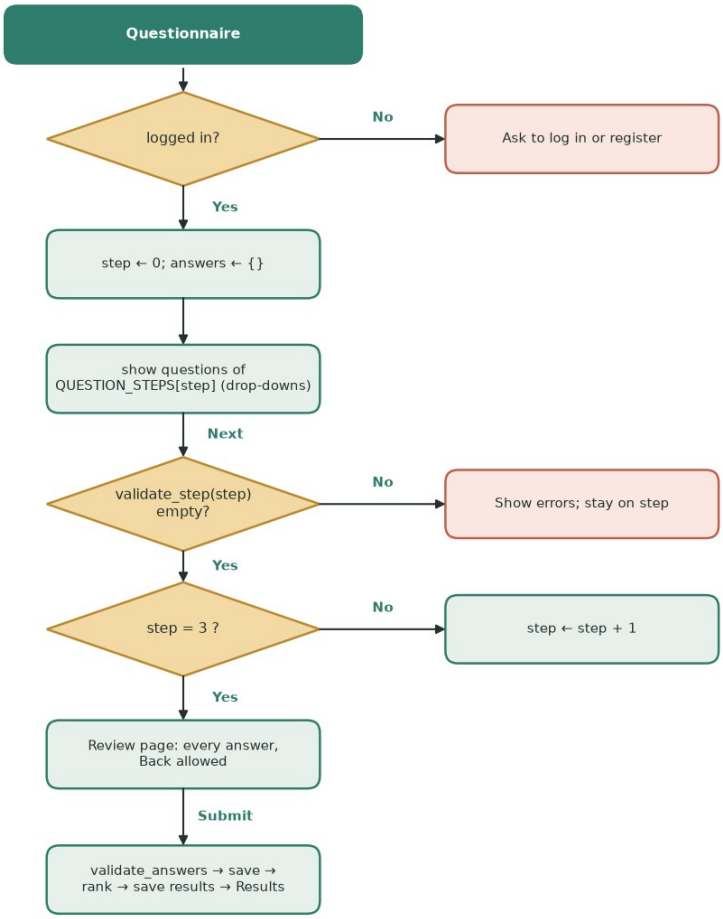


Diagram D6: questionnaire flowchart (model version).

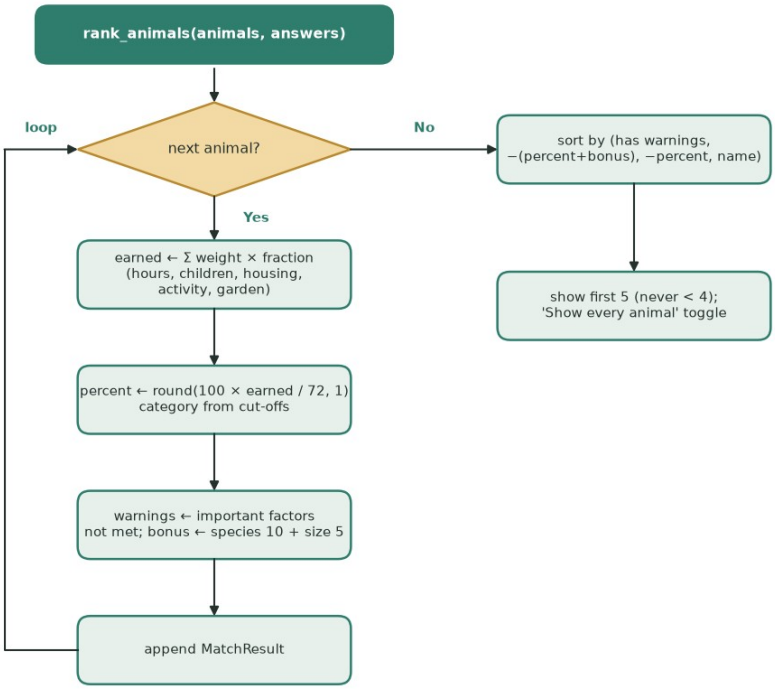


Diagram D7: scoring and ranking flowchart (model version).

```
CONSTANT QUESTION_STEPS ← [
    ("Your home",        [housing_level, garden_access]),
    ("Your household",   [has_children, other_pets]),
    ("Your lifestyle",   [activity_level, hours_alone, experience]),
    ("Your preferences", [preferred_species, preferred_size]) ]

PROCEDURE QuestionnairePage()
    user ← RequireLogin()
    step ← session.q_step (0 if not set)
    answers ← session.q_answers (empty if not set)
    IF step < 4 THEN
        SHOW progress "Step (step + 1) of 5" and the questions of QUESTION_STEPS[step] as
drop-downs
        IF "Back" clicked AND step > 0 THEN session.q_step ← step − 1
        IF "Next" clicked THEN
            errors ← ValidateStep(step, answers)
            IF errors is empty THEN session.q_step ← step + 1 ELSE SHOW errors
        ENDIF
    ELSE
        SHOW every question with its answer        // review step
        IF "Change answers" clicked THEN session.q_step ← 0
        IF "Submit and see my matches" clicked THEN SubmitQuestionnaire(user, answers)
    ENDIF
ENDPROCEDURE

FUNCTION ValidateStep(step, answers)
    errors ← empty dictionary
    FOR EACH question IN QUESTION_STEPS[step]
```

```
        CHECK answers[question] with that question's rule:
            housing_level, activity_level: one of 1, 2, 3
            garden_access, has_children, other_pets: Yes or No
            hours_alone: whole number 0 to 24
            experience: one of the three listed levels
            preferred_species: Dog, Cat, Rabbit or No preference
            preferred_size: Small, Medium, Large or No preference
        IF it fails THEN errors[question] ← that question's message
    NEXT question
    RETURN errors
ENDFUNCTION

FUNCTION ValidateAnswers(answers)                   // all nine questions again before saving
    errors ← ValidateStep(0) ∪ ValidateStep(1) ∪ ValidateStep(2) ∪ ValidateStep(3)
    IF errors is empty THEN RETURN Answers(answers), errors ELSE RETURN NULL, errors
ENDFUNCTION
```

```
CONSTANT WEIGHTS ← {hours_alone: 20, children: 15, housing: 15, activity: 12, garden:
10}   // total 72
CONSTANT IMPORTANT ← {hours_alone, children, housing}    // unmet → warning

FUNCTION ScoreAnimal(animal, answers)
    total_score ← 0
    met ← empty list; unmet ← empty list; warnings ← empty list
    fraction["hours_alone"] ← 1 IF answers.hours_alone ≤ animal.max_hours_alone ELSE 0
    fraction["children"]    ← 1 IF animal.child_compatible OR NOT answers.has_children ELSE 0
    fraction["housing"]     ← 1 IF answers.housing_level ≥ animal.min_housing_level ELSE 0
    gap ← MAX(0, animal.exercise_needs − answers.activity_level)
    fraction["activity"]    ← MAX(0, 1 − gap / 2)          // 1, 0.5 or 0
    fraction["garden"]      ← 1 IF answers.garden_access OR NOT animal.garden_required ELSE 0
    FOR EACH factor IN WEIGHTS
        total_score ← total_score + WEIGHTS[factor] × fraction[factor]
        IF fraction[factor] = 1 THEN
            APPEND reason TO met
        ELSE
            APPEND reason TO unmet
            IF factor IN IMPORTANT THEN APPEND reason TO warnings
        ENDIF
    NEXT factor
    compatibility_score ← ROUND(100 × total_score / 72, 1)
    IF compatibility_score ≥ 80 THEN match_category ← "Strong"
    ELSE IF compatibility_score ≥ 60 THEN match_category ← "Good"
    ELSE IF compatibility_score ≥ 40 THEN match_category ← "Possible"
    ELSE match_category ← "Low"
    ENDIF
    bonus ← 0                                           // preferences order the list but
never change the %
    IF answers.preferred_species = animal.species THEN bonus ← bonus + 10
    IF answers.preferred_size = animal.size THEN bonus ← bonus + 5
    RETURN compatibility_score, match_category, bonus, met, unmet, warnings
ENDFUNCTION

FUNCTION RankAnimals(animal_results, answers)
    results ← empty list
    FOR EACH animal IN animal_results
        APPEND ScoreAnimal(animal, answers) TO results
    NEXT animal
    SORT results BY, in order:
        1. animals with no warnings first
        2. (compatibility_score + bonus), highest first
        3. compatibility_score, highest first
        4. name A-Z, then animal_id                     // ties always come out in the same
order
    RETURN results
ENDFUNCTION

FUNCTION SubmitQuestionnaire(user, raw_answers)
    answers, errors ← ValidateAnswers(raw_answers)
    IF errors is not empty THEN RETURN errors           // nothing is saved
    response_id ← INSERT INTO questionnaire_responses (...) VALUES (?, ...)
    results ← RankAnimals(SELECT * FROM animals, answers)
    DELETE FROM match_results WHERE response_id = ?
    calculated_at ← NOW()
    FOR rank ← 1 TO LENGTH(results)
        INSERT INTO match_results (response_id, animal_id, rank, compatibility, category,
```

```
bonus,
                                    met, unmet, warnings, calculated_at) VALUES (?, ...)
    NEXT rank
    GO TO My matches
ENDFUNCTION
```

How it fits the whole solution. ValidateStep stops gaps step by step; ValidateAnswers checks the whole set again before anything is stored. SubmitQuestionnaire then joins the parts: it stores the response, calls RankAnimals (which calls ScoreAnimal for every animal) and stores the ranked results that the My matches page shows.

Validation - questionnaire

1.Housing type or housing level

- Presence check: The user must select an answer
- Lookup check: The answer must be one of the
- A radio button or drop-down will prevent

1 = Flat or limited space
2 = Small or medium house
3 = Large house

2.Garden access

- Presence check: Yes or No must be selected
- Boolean check: The stored result must be 1 or 0

3.Hours the animal will be left alone

- Presence check : a value must be provided
- Type check : a whole number
- Range check : between 0 and 24 hours

4.Children in the home

- Presence check: Yes or No must be selected
- Boolean check: The stored result must be 1 or 0

5.Preferred activity level

- Presence check: an option must be selected
- Boolean check: The stored result must be 1, 2 or 3

After

6.Preferred species

- Presence check: The user must select an answer
- Lookup check: The answer must be a dog, cat, rabbit or no preference

7.Preferred size

- Presence check: The user must select an answer
- Lookup check: answer must be small, medium, large or no preference

Validation is necessary within this compatibility algorithm to ensure that the recommendations produced are reliable and that invalid data does not cause incorrect results.

Questionnaire Response table

Field	Data type	Key/validation
response_id	INTEGER	Primary key, auto increment
user_id	INTEGER	Foreign key linked to Users.userid
housing_level	INTEGER	Required range 1-3
garden_access	BOOLEAN	Required 0 or 1
hours_left_alone	INTEGER	Required valid range
has_children	BOOLEAN	Required 0 or 1
preferred_activity_level	INTEGER	Required range 1 - 3
preffered_species	TEXT	Required dog, cat, rabbit or no preference
preferred_size	TEXT	Required small, medium, large or no preference
completed_at	DATE	required

Match Results Table

Field	Data type	Key/validation
match_id	INTEGER	Primary key autoincrement
response_id	INTEGER	Foreign key linked to QuestionaireReponses .response_id
animal_id	INTEGER	Foreign key linked to animals.animal_id
compatibility_score	REAL	Required range 0 to 100
match_category	TEXT	Required strong, good, possible or low
unmet_requirements	TEXT	optional

Calculated_at	DATE	required

Adoption Questionnaire Variables

Variable Name	Data Type	Purpose	Justification	Validation or e.g
housing_level	integer	Stores amount of living space available	The available space may affect whether or not the home is suitable for different sizes and exercise needs	Required range 1-3
garden_access	boolean	Records whether user has access to a garden	Some animals may require regular access to an outdoor space. A boolean value is suitable as the answer has two possible	0 or 1
hours_left_alone	integer	Stores how long the animal would normally be left alone	Time availability can be numerically compared with the animals required attention and	Required valid range such as 0-24
has_children	boolean	Records whether children live in the household	Some animals may not be suitable for homes with children making this an important safety	0 or 1
preferred_activity_level	integer	Stores the users preferred animal activity	Matching activity level helps ensure the adopter can meet	range 1 - 3

		level	the animals exercise requirements	
preffered_species	string	Stores the users preferred species	This ensures that the recommendations remain relevant to the type of animal the user is interested in	Required ‘dog’, ‘cat’, ‘rabbit’ or ‘no preference’
preferred_size	string	Stores the users preferred size	The user may have practical or personal reasons for preferring a particular size.	‘small’, ‘medium’, ‘large’ or ‘no preference’

Compatibility scoring algorithm variable names

Variable Name	Data Type	Purpose	Justification	Validation or e.g
animal_id	integer	Stores the unique identifier of the animal being assessed	It ensures that each calculated score is linked to the correct animal, even when animals have the	Must match an existing record in the animals table e.g 12
maximum_score	integer	Stores the highest possible number of points that can be awarded by the compatibility algorithm	This is required so that the user's raw score can be converted into a percentage. An integer is suitable because the algorithm awards whole number points for each matching factor.	Must be greater than 0

total_score	integer	Stores the total compatibility points awarded for the animal currently being assessed	The value begins at zero and increases whenever the user's answers from the questionnaire match an animal's requirements. It must be reset before the algorithm assesses the next animal.	Begins at 0 and cannot exceed maximum_score
animal_scores	list	Stores the ID and compatibility score for all available animals	This will allow the results to be sorted from highest to lowest compatibility. A list is suitable because the algorithm processes multiple animals	[{"animal_id": 3, "score": 92.5}, {"animal_id": 7, "score": 78.0}]
compatibility_score	float	Stores the calculated compatibility percentage	A percentage is easier for users to understand and compare than a raw points total, a float will allow decimal results to be stored.	Must be between 0-100 and can have decimals
match_category	string	Stores an understandable category for the calculated score	A category makes the numerical result easier for the user to interpret. It is selected according to predefined percentage ranges.	strong, good, possible or low
unmet_requirements	list	Stores explanations of requirements the user did not meet	There may be several reasons why an animal is not fully suitable so a list will allow multiple explanations to be stored and displayed	Example: "Requires a secure garden", "Not suitable with cats". It may be empty when all essential requirements are met.

Questionnaire Plan

Section 1: House environment
- Amount of living space
- Garden access?

Section 2:
- Children in the home?

Section 3:
- Preferred activity level / time available for exercise and care
- Hours the animal would be left alone

Section 4:
- Preferred species
- Preferred animal size

Section 5:
- Submit

Where it links

- Amount of living space → minimum housing level
- Garden access → garden required
- Children in the home → child compatible
- Preferred activity level → exercise need
- Hours left alone →

First Matching part

Factor	Suggested weight (can be changed)	Justification
Hours Left alone	20	Strongly will affect whether the adopter can meet the animals needs

Child compatibility	15	Important for safety and suitability
Housing space	15	Ensures the home provides enough room
Activity	12	Matches exercise needs with users lifestyle and what they can provide
Garden access	10	Important for animals requiring outdoor access, but can even use a park so not as important factor

- Calculate the suitability score
- Remove animals with low
- suitability
- Prioritise animals matching
- users preferred species
- Prioritise animals matching users preferred size
- Sort remaining animals by

Factor	Suggested weight
Species	10
Size	5

However, these weights will not add to the compatibility percentage as their preferences do not reflect animal suitability. It will influence the order in which suitable animals are displayed.

This will prevent an unsuitable animal from receiving a misleading high compatibility percentage as it matches the users preferences, but still ensuring preferred animals are presented as users top choices as this will make them more inclined to adopt. If the user picks ‘no preference’ no animal will receive a bonus.

Model version addition

Compatibility score, exactly

- Each factor gives a fraction: 1 if met, 0 if not. Activity can also give 0.5: fraction = max(0, 1 − gap ÷ 2), where gap = the animal's exercise level minus the adopter's activity level (never below 0). Being more active than needed is not penalised.
- points = 20 × hours + 15 × children + 15 × housing + 12 × activity + 10 × garden (maximum 72).
- **compatibility % = round(100 × points ÷ 72, 1)**.
- Hard rules (children, hours alone, space) **lose their points and give a warning**; they do not remove the animal. Animals with a warning are listed after every animal without one. Garden and activity lose points only.
- Categories: **Strong** 80 % or more; **Good** 60 to 79.9 %; **Possible** 40 to 59.9 %; **Low** below 40 %.
- Preferences (species 10, size 5) are added only to the sorting points, never to the percentage, as the Design says.

Worked example. Adopter: lives in a flat (housing 1), no garden, has children, can offer activity level 2, leaves an animal 6 hours a day, prefers dogs, no size preference.

Factor (weight)	Milo, cat: needs housing 1, no garden, child-safe, exercise 1, up to 8 h alone	Oscar, dog: needs housing 1, no garden, child-safe, exercise 2, up to 4 h alone
Hours alone (20)	6 ≤ 8: met → 20	6 > 4: not met → 0, **warning**
Children (15)	child-safe: met → 15	child-safe: met → 15
Housing (15)	1 ≥ 1: met → 15	1 ≥ 1: met → 15
Activity (12)	gap max(0, 1 − 2) = 0 → 12	gap max(0, 2 − 2) = 0 → 12
Garden (10)	not needed: met → 10	not needed: met → 10
Points	72	52
Percentage	100 × 72 ÷ 72 = 100.0 % (Strong)	100 × 52 ÷ 72 = 72.2 % (Good)
Preference bonus	cat ≠ dog → 0	dog = dog → 10
Sorting points	100.0	82.2, but has a warning

Result: Milo is listed above Oscar, and Oscar comes after every animal without a warning even though his sorting points (82.2) are higher than some of theirs. Running the real code on the same answers gives the same numbers (Milo 100.0, Oscar 72.2 with one warning); this example is reused as test data (E2, E7, E9).

Model version addition

Usability features

Feature	Where it appears	Why it is included
Calm colour scheme from one theme file (green, cream, dark text)	Every page	Survey and examples asked for a calm, clean look (S6, E1-E3). One theme file means every page matches (criterion 1.4).
The same top menu on every page, showing only the pages the user may open	Every page	All three example sites keep one menu (E1-E3, S6). Hiding the staff page from adopters avoids a confusing 'no access' link (Q12).
Questionnaire in four short steps plus a review step, with a progress bar	Questionnaire	E3 and S6: long forms put people off. The progress bar shows how much is left; the review step lets users check before submitting.
Drop-downs for every answer (no typing)	Questionnaire, filters, staff form	E3, S6: quicker on a tablet, and the answer can only be one of the allowed values, so fewer errors (Q15).
All errors for a form shown together in one red box, in plain words	Register, log in, questionnaire, staff form	Q12, S6: users should see everything to fix at once instead of one message per attempt.
Compact animal cards (photo, name, species, breed, age, 'View profile')	Animals, Home	Q2, E1, E2: the examples use cards; the card holds the five facts adopters said they check first.
Score bar, category word and met / not met list on each match	My matches	Q3, E3: the client wants adopters to see why an animal suits them, not just a number.
Orange warning when an important need (time alone, children, space) is not met	My matches	Q3: welfare first. The warning cannot be missed and these animals are listed after the safer matches.
'Guidance only: staff make the final decision' note	Home, My matches	Q3: sets the right expectation.
Delete needs a tick-box confirmation; the button is disabled until then	Staff area	Q5: deleting is permanent, so a slip of the mouse must not remove an animal.
Back links that return to where the user came from	Animal profile	S6, E1-E3: a profile opened from the results goes back to the results, not to the start.

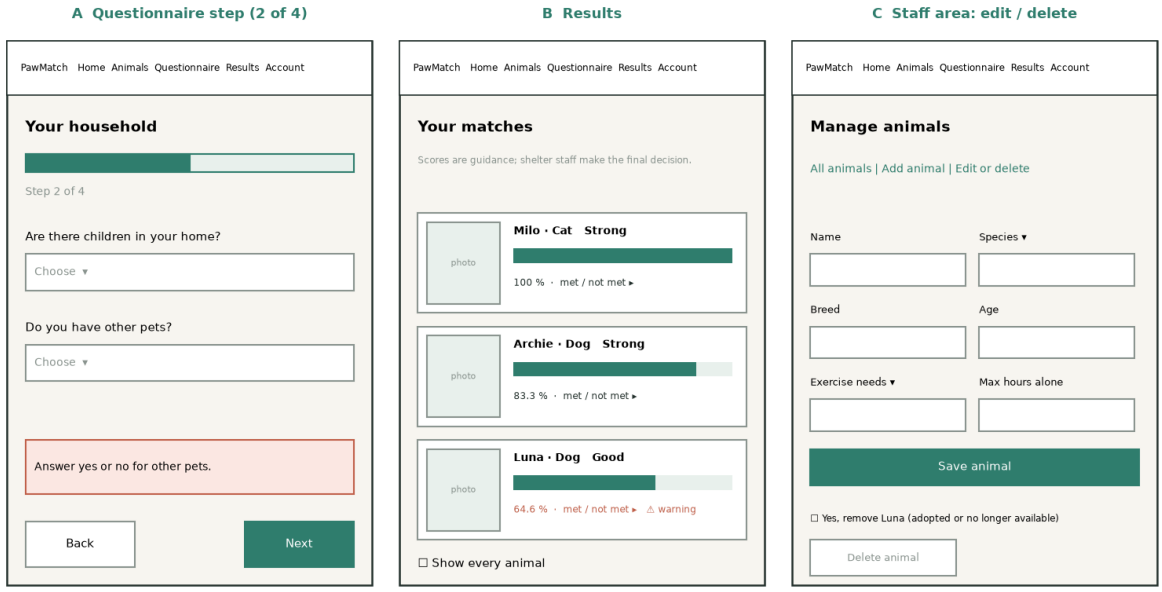


Diagram D8: wireframes for the questionnaire step, results and staff edit screens (model version).

Development Test plan

Development Test Plan

Model version addition

Development test plan

Five components, 8 to 11 tests each, mixing normal, boundary and invalid data. Each test is chosen to prove a rule from the Design works on both sides of its edge. These numbers are used in the Development section.

Test	What is tested and why	Input data	Expected result	Type
A1	Valid sign-up: the main path must work	Sam Lee, sam@x.com, 30, 'longenough' twice	Account stored, logged in	Normal
A2	Blank fields: presence checks	name ' ', email '', age '', password ''	One message per field	Invalid
A3	Email format	no-at-sign.com, a@b, two@@x.com	'Enter a valid email address'	Invalid
A4	Age boundary: 18+ rule	17 / 18 / 120 / 121	17 and 121 rejected; 18 and 120 accepted	Boundary
A5	Age type check	2.5, abc	'Age must be a whole number.'	Invalid
A6	Password boundary (at least 8)	7 characters / 8 characters	7 rejected, 8 accepted	Boundary
A7	Duplicate email, any case	SAM@x.com after sam@x.com	'already exists', nothing stored	Invalid
A8	Login with right details; email case ignored	' SAM@X.COM ' + password	Logged in	Normal
A9	Wrong password and unknown email	right email + wrong password; unknown email	Same message for both	Invalid
A10	Password stored only as a salted hash	same password twice	Two different hashes; no plain text	Normal
A11	Name format from the Design variables table	Mary-Jane, Siobhan O'Neill / 12345, Sam3, Sam!	Letters, spaces, hyphens, apostrophes only	Invalid
B1	Every animal shown as a card	13 demo animals	13 cards	Normal
B2	Species and size filters	Rabbit; Small	Only matching animals	Normal
B3	Search text is data, not SQL	' OR 1=1 --	No animals, no error	Invalid
B4	Empty search shows everything	''	All animals	Boundary
B5	Search length limit	21 characters typed	Box stops at 20	Boundary
B6	Profile loads from the database by id	id of Luna	Luna's details	Normal
B7	Profile with a missing or bad id	id=999, id=abc	'could not be found'	Invalid
B8	Back link returns to where the user came from	open from results	'Back to my matches'	Normal
C1	Adopter calls add, edit or delete	adopter account	Refused (AccessDenied)	Invalid
C2	Add a complete animal	every field valid	Stored; listed	Normal
C3	Animal age range	0 / 1 / 30 / 31	0 and 31 rejected	Boundary
C4	Hours alone range	-1 / 0 / 24 / 25	-1 and 25 rejected	Boundary
C5	Edit and save	new breed	Breed changed on the profile	Normal
C6	Edit an animal deleted meanwhile	stale id	'no longer exists'	Invalid
C7	Delete needs confirmation	box not ticked	Button disabled	Invalid
C8	Staff page hidden from adopters	adopter opens the page by name	No such page	Invalid
C9	Photo type and size	a.gif; 2 MB + 1 byte / exactly 2 MB	Rejected / accepted	Boundary
C10	Database repeats the rules	direct INSERT with species 'Hamster'	Database refuses it	Invalid
C11	Breed length and content (Design)	Mixed / X, ###	X and ### rejected	Boundary
D1	Each question in exactly one step	the four steps	9 questions, none twice	Normal
D2	A blank answer blocks Next	step 1 with one blank	Error, stays on step	Invalid
D3	A step only checks its own questions	step 1 answered, rest blank	Next allowed	Normal
D4	Hours alone range	-1 / 0 / 24 / 25	-1 and 25 rejected	Boundary
D5	Answer outside the list	species 'Hamster'	Rejected	Invalid
D6	Incomplete set cannot be saved	8 of 9 answers	Nothing stored	Invalid
D7	Must be logged in	no session	'Please log in'	Invalid
D8	Full valid set	9 valid answers	Saved; results page opens	Normal
E1	Weights equal the Design	WEIGHTS	20, 15, 15, 12, 10 = 72	Normal
E2	Perfect match	every need met	100 %, Strong	Boundary
E3	Nothing met	every need failed	0 %, Low	Boundary
E4	Each factor loses its own weight	fail one factor at a time	−20, −15, −15, −12, −10 points	Normal
E5	Exactly at a limit counts as met	hours = max hours; housing = minimum	Met	Boundary
E6	Category cut-offs	79.9 / 80 / 59.9 / 60 / 39.9 / 40	Possible/Strong/Possible/Good/ Low/Possible	Boundary
E7	Warnings only for important factors	fail garden vs fail children	Warning only for children	Normal
E8	Preferences change order, not %	preferred species	Same %, higher place	Normal
E9	Unsuitable animals rank after suitable ones	high score with warning vs lower without	Lower one first	Normal
E10	At least 4 results shown	13 animals	5 shown, toggle for all	Normal
E11	Results record when they were calculated (Design table)	any submission	calculated_at stored	Normal

Post-development test plan

One test per success criterion, using the measure from the Analysis, plus timing (T), robustness (R) and usability tests (U). The usability tests need real people from the stakeholder groups (adopters from the survey group and the shelter client).

Test	Criterion	Measure / expected result
P1.1	Pages must use a calm colour scheme, clear headings and identifiable buttons	One theme file sets every colour; body text contrast at least 4.5 : 1 (WCAG AA); every action is a labelled button
P1.2	The interface must remain readable on laptop and tablet sized screens	At 1280 px and 820 px (tablet) wide: no sideways scrolling and no text cut off
P1.3	The system must include a navigation menu to access home page, animal preview page, profil…	The top menu holds Home, Animals, Questionnaire, My matches and Log in / My account on 7 of 7 pages
P1.4	The menu and interface style must remain consistent across all the pages	0 pages with their own colours or CSS
P2.1	Should explain the purpose of the animal adoption matching system, especially including th…	The home page states the purpose and the word 'suitability'
P2.2	Should have a clear button or navigation option that directs users to complete the adoptio…	1 click from Home to the questionnaire
P3.1	The page should display every available animal using compact animal cards	Number of cards = number of animal records (13 of 13; 500 of 500 in the load test)
P3.2	Each card should show the species, name, breed, age and image	5 of 5 facts on every card: species, name, breed, age, image
P3.3	The system must support at least 3 animal species including dogs, cat and rabbits	At least 3 species stored and shown
P3.4	Users must be able to open a full animal profile from the preview page	1 click from a card to its profile
P4.1	Each profile must display the animals name, species, breed, age and sex	5 of 5 shown: name, species, breed, age, sex
P4.2	Each profile also must include personality, medical needs, exercise needs and suitability …	Personality, medical needs, exercise needs and suitability all shown
P4.3	Animal profile information must be retrieved from the database	Profile is loaded by animal_id from SQLite; editing the record changes the page
P4.4	Users must be able to return to the animal preview page or results page	1 click back to the list or to My matches
P5.1	Users must be able to create an account, using name, email and password and it can include…	A valid sign-up stores 1 user row with a hashed password
P5.2	The system must reject registration if required fields are empty and the email format is i…	100 % of the blank and invalid cases A2-A6 and A11 are rejected with a message
P5.3	Registered users must be able to login and logout using their correct account details	Correct details log in; Log out clears the session
P5.4	The admin login details should allow admins to access their own section with the CRUD inte…	A staff account sees 'Manage animals'; an adopter sees no staff link
P5.5	Incorrect Login details must display an error message and prevent access to the account	Wrong password and unknown email give the same message and no session
P6.1	The questionnaire must collect information about the user's home, lifestyle, garden access…	9 questions cover home, garden, children, other pets, activity, hours alone, experience, species, size
P6.2	The questionnaire must be split into clear sections or individual questions rather than ap…	4 steps and a review step; 4 of 5 testers rate the layout 8/10 or more for clarity
P6.3	The questionnaire must use suitable input methods such as dropdowns, radio buttons or sele…	0 free-text boxes in the questionnaire
P6.4	The system must prevent the questionnaire from being submitted if important questions are …	A blank answer blocks Next; an incomplete set saves 0 rows
P7.1	The system must compare adopter's questionnaire with each animal's requirements	Every animal gets a result (N results for N animals)
P7.2	The system must calculate a numerical compatibility score for each animal	A percentage from 0 to 100 (1 decimal place); 500 animals scored and stored in under 2 seconds
P7.3	The top 4 animals must be displayed from highest compatibility score to lowest at a minimu…	At least 4 shown, best first
P7.4	The results page must show which key requirements the user meets and does not meet for eac…	Every result lists what is met and what is not
P7.5	The system must display a warning if the user does not meet important requirements for an …	An unmet hours-alone, children or space requirement shows a warning
P7.6	The results page must explain that the compatibility scores are guidance, and the adoption…	The results page says scores are guidance and staff decide
P8.1	This page should be accessed separately with a separate admin login	The staff page is reached only through a staff login
P8.2	Admin users must be able to add a new animal record with all	A complete add form stores a record; any invalid field stores nothing

	the information required	
P8.3	Admin users must be able to edit existing animal records and save the updated information …	An edit is saved and shown on the profile
P8.4	Admin users must be able to delete animal records when animals adopted or no longer availa…	Delete only after the confirmation box is ticked; the record and its results are removed
P8.5	Normal users must not be able to access the admin management page	An adopter is refused every staff action, even if the page is bypassed
P9.1	The database must store animal records, user account details, admin login details and ques…	Stores users, animals, responses and results; at least 500 animals and 100 users
P9.2	The database must allow animal records to be added, viewed, edited and deleted through the…	Animals can be added, viewed, edited and deleted
P9.3	Questionnaire answers must be stored or processed so that they can be used by the compatib…	Answers are saved and used for the score
P9.4	Data must be retrieved correctly so animal profiles and recommendations display accurate i…	Values read back equal the values stored

Test	What	Who	Expected
T1	Theme and contrast	Developer	Text contrast at least 4.5 : 1
T2	Width check	Developer, headless browser	No sideways scrolling at 1280 and 820 px
T3	Menu on every page	AppTest	Same menu on 7 of 7 pages
T4	Scoring time	Developer	500 animals scored and stored in under 2 s
T5	Capacity	Developer	500 animals and 100 users stored and listed
U1	Usability: register and log in without help	Adopter (survey group), first visit	Done in under 2 minutes; rates it 8/10+
U2	Usability: complete the questionnaire	Adopter	No help needed; rates clarity 8/10+
U3	Usability: explain your top match and any warning	Adopter	Can say why the top animal suits them
U4	Usability: add, then edit, then delete an animal	Shelter staff (client)	All three done without help
U5	Usability: use the site on a tablet	Adopter with a tablet	No zooming or sideways scrolling

3 Development

Model version addition

Development was done in five stages, each saved as a commit on the opus-rebuild branch, so every prototype can be restored. The stages follow the hierarchy chart: rules first, then storage, then pages, then a whole-system run, then a re-check against the Design. Each module was written with its tests, and the full test suite was run at the end of every stage; the output of each run is kept in docs/evidence.

Stage	Commit (prototype)	What it added	Tests after
1	ffd0ad5	models.py, validation.py, matching.py	73 passed
2	10e84e7	db.py, auth.py, services.py, seed.py, animal photos	103 passed
3	b76527c	streamlit_app.py, webui/ pages, page tests, screenshot script	117 passed
4	213e208	README, write-up, comparison page, 16 screenshots	117 passed
5	fdef5a0	Design re-check: name and breed rules, calculated_at	134 passed

Reviews: no client or stakeholder was available for this model version, so each review below is a developer review against the Design and success criteria, and says so. In a real project each one would also be shown to the client.

Stage 1: Data model, validation and matching rules

Aim

Builds the parts with no screens: the data classes (Design: variables tables), every validation rule (Design: the Validation lists for login, registration, animals and the questionnaire) and the compatibility algorithm (Design: Compatibility algorithm and questionnaire, First Matching part). These come first because every later stage uses them, and they can be fully tested without a database or a browser. Commit ffd0ad5; components A, C, D and E of the hierarchy chart.

Code

Weights and normalisation (pawmatch/matching.py). The weights are the Design's table. MAX_POINTS is worked out from them, so changing a weight cannot break the percentage.

```
WEIGHTS: dict[str, int] = {
    "hours_alone": 20,
    "children": 15,
    "housing": 15,
    "activity": 12,
    "garden": 10,
}
MAX_POINTS = sum(WEIGHTS.values())  # 72
PREFERENCE_BONUS: dict[str, int] = {"species": 10, "size": 5}

# Unmet requirements on these factors are welfare risks, so they raise a warning.
IMPORTANT_FACTORS = frozenset({"hours_alone", "children", "housing"})

CATEGORY_THRESHOLDS: tuple[tuple[float, str], ...] = (
    (80.0, "Strong"),
    (60.0, "Good"),
    (40.0, "Possible"),
    (0.0, "Low"),
)
MIN_RESULTS_SHOWN = 4
```

Scoring one animal. Each factor gives a fraction (1, 0.5 or 0) times its weight. Unmet important factors also go into warnings. The percentage is rounded to one decimal place.

```
def score_animal(animal: Animal, answers: Answers) -> MatchResult:
    """Score one animal: percentage, category, met/unmet lists and warnings."""
    earned = 0.0
    met: list[str] = []
    unmet: list[str] = []
    warnings: list[str] = []
```

```
    for factor, fraction, reason in _factor_checks(animal, answers):
        earned += WEIGHTS[factor] * fraction
        if fraction == 1.0:
            met.append(reason)
        else:
            unmet.append(reason)
            if factor in IMPORTANT_FACTORS:
                warnings.append(reason)
    compatibility = round(100 * earned / MAX_POINTS, 1)
    return MatchResult(
        animal=animal,
        compatibility=compatibility,
        category=category_for(compatibility),
        bonus=preference_bonus(animal, answers),
        met=met,
        unmet=unmet,
        warnings=warnings,
    )
```

Ranking: warnings last, then points. Sorting by a tuple applies the rules in order: warnings, points with bonus, percentage, then name and id, so ties always come out the same way.

```
def rank_key(result: MatchResult) -> tuple[int, float, float, str, int]:
    """Suitable animals first, then ranking points, compatibility, name and id (all
deterministic)."""
    return (
        0 if result.suitable else 1,
        -result.ranking_points,
        -result.compatibility,
        result.animal.name.lower(),
        result.animal.animal_id,
    )

def rank_animals(animals: Iterable[Animal], answers: Answers) -> list[MatchResult]:
    """Score every animal and return them best first."""
    return sorted((score_animal(a, answers) for a in animals), key=rank_key)
```

One validator per questionnaire step (pawmatch/validation.py). Each step validates only its own questions, so users get messages for the step they are on.

```
def validate_step(step: int, raw: dict[str, Any]) -> Errors:
    """Validate only the questions on one step, so users can't move on with gaps."""
    errors: Errors = {}
    for name in QUESTION_STEPS[step][1]:
        QUESTION_VALIDATORS[name](raw.get(name), errors)
    return errors
```

Prototype evidence. Ev 1: the stage 1 test run (docs/evidence/stage1-pytest.txt):

```
tests/test_validation.py::test_steps_cover_every_question_once PASSED    [100%]

============================== 73 passed in 0.04s ==============================
```

Testing

Test no.	Description	Input	Expected	Actual	Pass / Fail	Evidence (figure)
A2-A6	Registration validation	see test plan	Rejected with the right message	As expected	Pass	test_registration_* (Ev 1)
C3, C4	Animal ranges	0/1/30/31 and -1/0/24/25	Ends rejected	As expected	Pass	test_invalid_animal_fields_are_rejected
D1-D5	Questionnaire steps and rules	see test plan	As planned	As expected	Pass	test_steps_cover_every_question_once, test_step_validation_only_checks_that_step
E1-E9	Scoring and ranking	see test plan	As planned	As expected	Pass	test_matching.py, 29 cases
A11	Name format	12345	Rejected	Accepted (not	Fail (found in	Ev 5a

				yet checked)	stage 5)	
C11	Breed length and content	X, ###	Rejected	Accepted	Fail (found in stage 5)	Ev 5a

Failed test and remedial action

Test no.	What happened	Cause	Fix made	Re-test result
A11, C11	'12345' was accepted as a person's name, and 'X' and '###' as breeds. Found when the Design was re-checked in stage 5.	The validator only checked presence and length; the Design's format rules (letters, spaces and hyphens; breed 2-50 and not only symbols) were missed.	Added a letters-only rule for people (digits allowed for animal names such as 'R2', as the Design allows) and a 2-50, must-contain-letters rule for breeds. Tests written first from the Design wording.	Pass: 134 passed (Ev 5b)

Review

Developer review against the Design and success criteria, after the stage 1 tests passed (no client or stakeholder was shown this stage, as it has no screens). Checked: weights 20/15/15/12/10 match the Design table (test E1); preferences do not change the percentage, as the Design says; every validation list has at least one test. Open question found: the Design does not say whether failing children or garden makes an animal unsuitable or only loses points.

Changes made after review

Decided and wrote down the rule: every unmet factor loses its points; unmet hours alone, children or space also gives a warning, and those animals are listed after all animals without warnings (tests E7 and E9). Reason: a high score must never put an unsafe match at the top (Q3). Added a fixed tie-break (name, then id) so equal scores always appear in the same order.

Stage 2: Database, login and staff-only actions

Aim

Builds the SQLite database (Design: Database structure and the four table designs), registration and login with hashed passwords (Design: Login and Registration), the role check and the add / edit / delete services (Design: Admin Page), and the demo animals. Commit 10e84e7; components A, C and E (storing results).

Code

Salted scrypt hashing and constant-time check (pawmatch/auth.py). A new 16-byte salt for every password; the parameters are stored with the hash so they can be raised later. compare_digest takes the same time whether the first or last character differs.

```
def hash_password(password: str) -> str:
    """Return 'scrypt$n$r$p$salt$hash' with a fresh 16-byte salt."""
    salt = secrets.token_bytes(16)
    digest = hashlib.scrypt(password.encode(), salt=salt, n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P)
    return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${salt.hex()}${digest.hex()}"

def verify_password(password: str, stored: str) -> bool:
    """Recompute the hash with the stored salt and compare in constant time."""
    try:
        scheme, n, r, p, salt_hex, digest_hex = stored.split("$")
    except ValueError:
        return False
    if scheme != "scrypt":
        return False
    digest = hashlib.scrypt(password.encode(), salt=bytes.fromhex(salt_hex), n=int(n), r=int(r),
```

```
p=int(p))
    return hmac.compare_digest(digest.hex(), digest_hex)
```

Login: one message for every failure. Validation, then lookup, then hash check. Any failure returns the same message.

```
def login(conn: sqlite3.Connection, email: Any, password: Any) -> tuple[User | None, Errors]:
    """Return the user for correct details; one generic message otherwise (no account probing)."""
    cleaned, errors = validate_login(email, password)
    if errors:
        return None, errors
    found = db.find_user_with_hash(conn, cleaned["email"])
    if found is None or not verify_password(cleaned["password"], found[1]):
        return None, {"login": LOGIN_FAILED}
    return found[0], {}
```

Role checked inside the service. require_admin raises AccessDenied before anything is validated or stored.

```
def add_animal(conn: sqlite3.Connection, actor: User | None, raw: dict[str, Any]) -> tuple[int |
None, Errors]:
    require_admin(actor)
    values, errors = validate_animal(raw)
    if errors:
        return None, errors
    return db.insert_animal(conn, values), {}

def remove_animal(conn: sqlite3.Connection, actor: User | None, animal_id: int) -> bool:
    require_admin(actor)
    return db.delete_animal(conn, animal_id)
```

Browse query: values bound, never joined into SQL (pawmatch/db.py). Filters add '?' placeholders; the user's text only ever goes into params.

```
def list_animals(
    conn: sqlite3.Connection, species: str | None = None, size: str | None = None, search: str = ""
) -> list[Animal]:
    """Browse query with optional filters; the search term is bound, never concatenated."""
    sql = "SELECT * FROM animals WHERE 1 = 1"
    params: list[Any] = []
    if species:
        sql += " AND species = ?"
        params.append(species)
    if size:
        sql += " AND size = ?"
        params.append(size)
    if search.strip():
        sql += " AND (name LIKE ? OR breed LIKE ?)"
        term = f"%{search.strip()}%"
        params += [term, term]
    sql += " ORDER BY species, name"
    return [_animal(r) for r in conn.execute(sql, params)]
```

Prototype evidence. Ev 2: the stage 2 test run (docs/evidence/stage2-pytest.txt):

```
tests/test_validation.py::test_steps_cover_every_question_once PASSED    [100%]

============================== 103 passed in 1.33s ==============================
```

Testing

Test no.	Description	Input	Expected	Actual	Pass / Fail	Evidence (figure)
A1, A7	Register; duplicate email in another case	sam@x.com then SAM@x.com	Second refused	As expected	Pass	test_register_rejects_duplicate_email_case_insensitively
A8, A9	Login right / wrong	see test plan	Logged in / same message	As expected	Pass	test_login_success..., test_wrong_password_and_unknown_email_give_

						the_same_message
A10	Salted hash	same password twice	Different hashes	As expected	Pass	test_hash_is_salted_and_not_plain_text
B3	SQL injection in search	' OR 1=1 --	No rows, no error	As expected	Pass	test_search_text_is_treated_as_data_not_sql
C1	Adopter calls staff actions	adopter account	AccessDenied	As expected	Pass	test_non_admins_are_refused_every_change
C10	Database CHECKs	species 'Hamster'	IntegrityError	As expected	Pass	test_database_constraints_back_up_validation
E11	calculated_at stored	any submission	Timestamp stored	No such column	Fail (found in stage 5)	Ev 5a

Failed test and remedial action

Test no.	What happened	Cause	Fix made	Re-test result
E11	Match results had no calculated_at column. Found when the Design's Match Results table was re-checked in stage 5.	The table was built from the scoring needs (rank, score, reasons) and the Design's calculated_at field was left out.	Added the column, one timestamp per ranking, and an in-place upgrade (ALTER TABLE) so databases made before the fix keep working; a second test builds an old database and checks it is upgraded.	Pass: 134 passed (Ev 5b)

Review

Developer review against the Design and criteria 5 and 8-9. Checked: passwords never stored or logged in plain text; every SQL statement uses ? placeholders (search included, test B3); deleting an animal also removes its stored results. Found two risks: a login message that says 'no such email' would let anyone test which emails have accounts; and hiding the staff page alone would not stop a determined user.

Changes made after review

One message for any failed login (A9). The role is checked inside every add, edit and delete service as well as on the page (C1), so even code that bypasses the page is refused. Staff accounts can only be created from the command line (python -m pawmatch.seed --admin), so nobody can register as staff.

Stage 3: The pages (home, animals, profile, questionnaire, results, staff area)

Aim

Builds every screen from the Design's interface designs: Home and menu (Design: Main Menu), the animal list and profile (Design: Animal Viewing Page), the four-step questionnaire and results (Design: Compatibility algorithm and questionnaire, Questionnaire Plan) and the staff page (Design: Admin Page). Pages only call the stage 1-2 code. Also adds headless page tests. Commit b76527c; all five components.

Code

Role-aware menu (streamlit_app.py). The staff page is only added to the menu for admin users, so for anyone else its address does not exist.

```
pages = [
    st.Page(PAGES["home"], title="Home", icon=":material/home:", default=True),
```

```
    st.Page(PAGES["browse"], title="Animals", icon=":material/pets:"),
    st.Page(PAGES["profile"], title="Animal profile", visibility="hidden"),
    st.Page(PAGES["questionnaire"], title="Questionnaire", icon=":material/checklist:"),
    st.Page(PAGES["results"], title="My matches", icon=":material/favorite:"),
    st.Page(PAGES["account"], title="My account" if user else "Log in", icon=":material/person:"),
]
if user and user.is_admin:  # staff page is not even routable for anyone else
    pages.append(st.Page(PAGES["admin"], title="Manage animals",
icon=":material/admin_panel_settings:"))

st.navigation(pages, position="top").run()
```

Questionnaire step logic (webui/views/questionnaire.py). The step number and answers live in the session, so Back keeps earlier answers. validate_step runs before moving on.

```
user = require_login()
step: int = st.session_state.setdefault("q_step", 0)
answers: dict = st.session_state.setdefault("q_answers", {})
last = len(QUESTION_STEPS)  # the review step comes after the question steps

st.title("Adoption questionnaire")
st.progress(step / last, text=f"Step {min(step + 1, last + 1)} of {last + 1}")

if step < last:
    title, names = QUESTION_STEPS[step]
    st.subheader(title)
    for name in names:
        label, options, fmt = QUESTIONS[name]
        saved = answers.get(name)
        answers[name] = st.selectbox(
            label, options, format_func=fmt, placeholder="Choose an answer",
            index=options.index(saved) if saved in options else None, key=f"q-{name}",
        )
    back, nxt, _ = st.columns([1, 1, 4])
    if step > 0 and back.button("Back", icon=":material/arrow_back:"):
        st.session_state["q_step"] = step - 1
        st.rerun()
    if nxt.button("Next", type="primary", icon=":material/arrow_forward:"):
        errors = validate_step(step, answers)
        show_errors(errors)
        if not errors:
            st.session_state["q_step"] = step + 1
            st.rerun()
else:
    st.subheader("Check your answers")
```

Page guards (webui/state.py). Every protected page starts with one of these; require_staff builds on require_login.

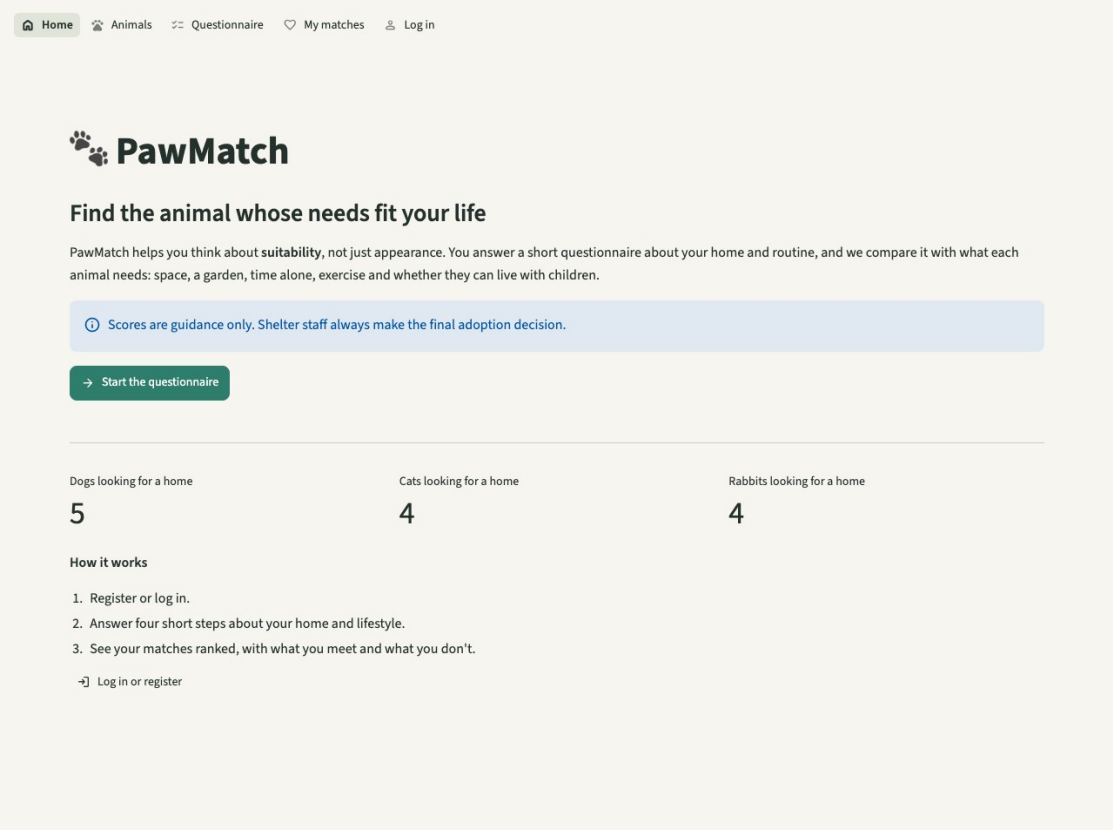
```
def require_login() -> User:
    user = current_user()
    if user is None:
        st.info("Please log in or register to use this page.")
        st.page_link(PAGES["account"], label="Log in or register", icon=":material/login:")
        st.stop()
    return user


def require_staff() -> User:
    """Page-level guard; the services also check the role, so this is defence in depth."""
    user = require_login()
    if not user.is_admin:
        st.error("This page is for shelter staff only.")
        st.stop()
    return user
```

Prototype evidence. Ev 3: the stage 3 test run (docs/evidence/stage3-pytest.txt):

```
tests/test_validation.py::test_image_type_and_size[d.jpg-2097153-False] PASSED [100%]

============================== 117 passed in 3.68s ==============================
```



Screen 01: Home. Purpose, suitability focus, guidance note and the questionnaire button (2.1, 2.2, 7.6).

Home | Animals | Questionnaire | My matches | Log in

Animals looking for a home

Species: All species

Size: All sizes

Search by name or breed

Showing 13 animal(s).



Bella

Cat · British shorthair cross · 7 yr

View profile



Milo

Cat · Domestic shorthair · 2 yr

View profile



Simba

Cat · Domestic longhair · 4 yr

View profile



Willow

Cat · Domestic shorthair · 12 yr

View profile



Archie

Dog · Cocker spaniel · 3 yr

View profile



Luna

Dog · Labrador cross · 4 yr

View profile



Max

Dog · German shepherd cross · 6 yr

View profile



Nala

Dog · Staffordshire bull terrier cross · 5 yr

View profile



Oscar

Dog · Jack Russell terrier cross · 8 yr

View profile



Clover

Rabbit · Rex rabbit · 1 yr

View profile



Daisy

Rabbit · Dutch rabbit · 4 yr

View profile



Poppy

Rabbit · Mini lop · 2 yr

View profile

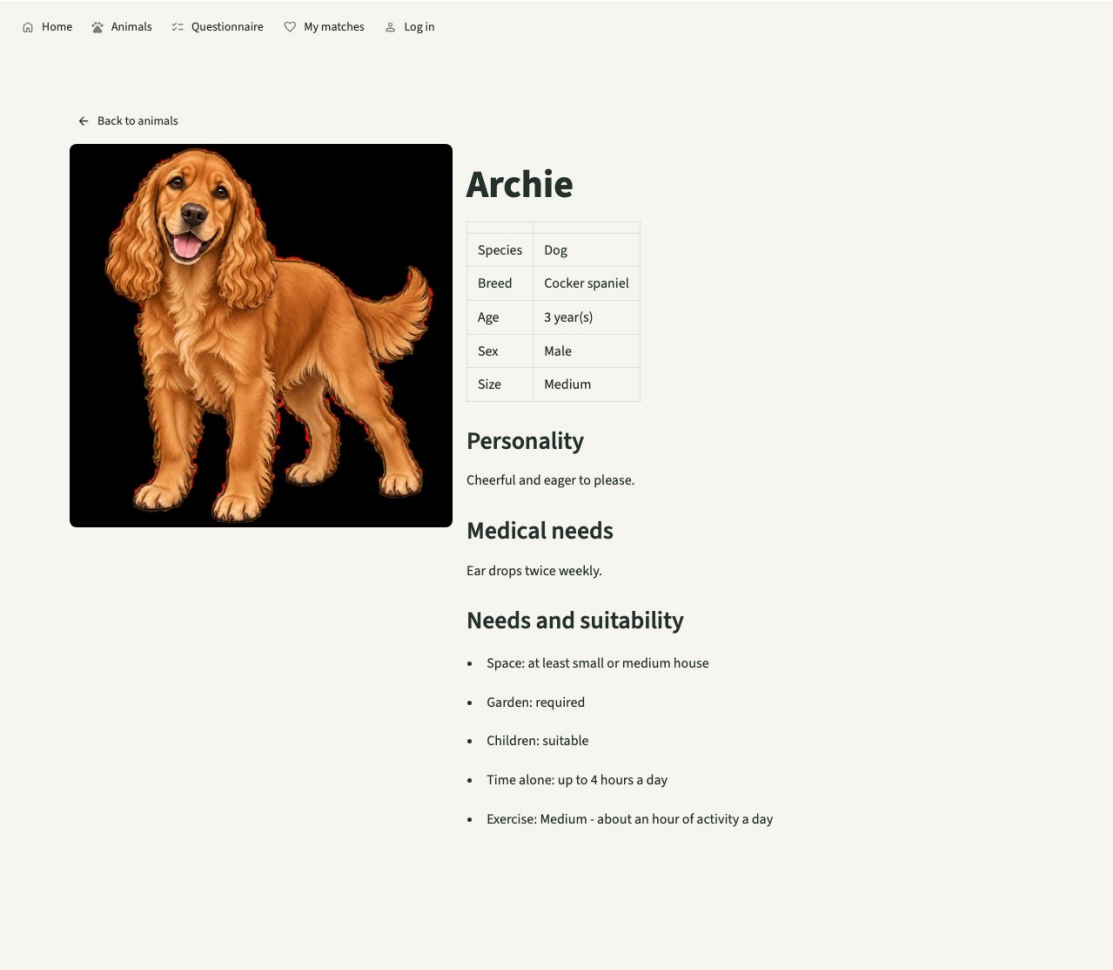


Teddy

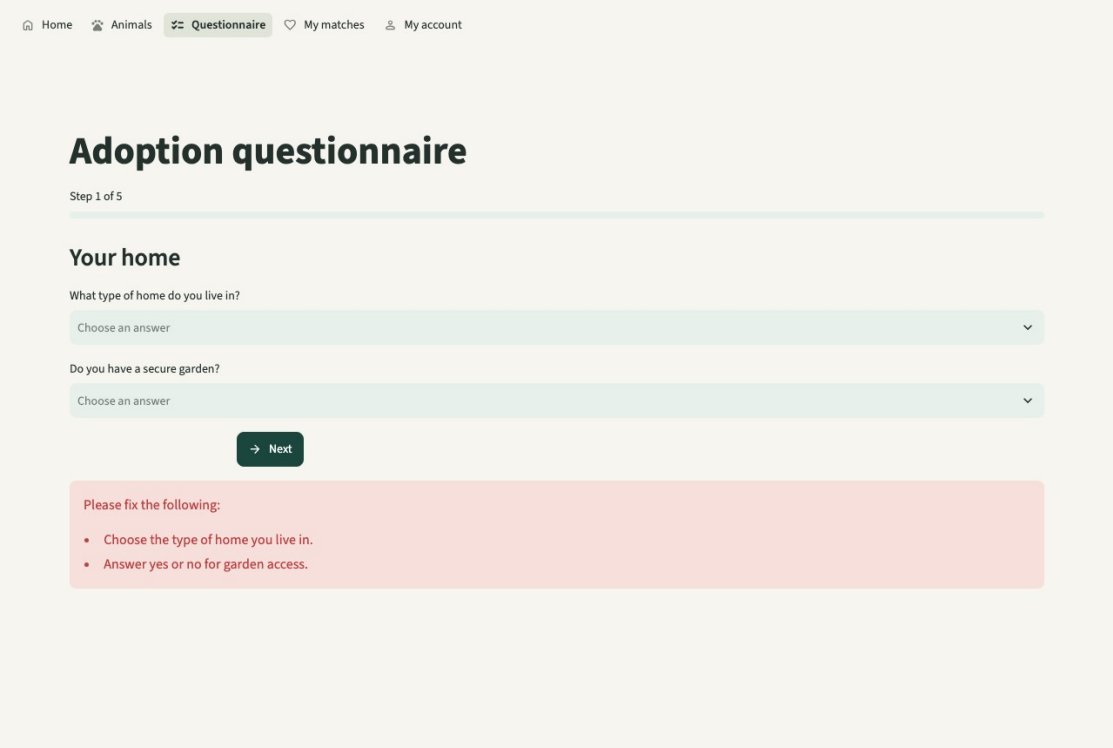
Rabbit · Lionhead · 3 yr

View profile

Screen 02: Animals. 13 cards with photo, name, species, breed and age; filters and search (3.1-3.3).



Screen 04: Profile read from the database, with the back link (4.1-4.4).



Screen 07: Next pressed with a blank answer: one box lists what is missing (6.4).

Home Animals Questionnaire My matches My account

My matches

These scores are guidance to help you think about suitability. Shelter staff always make the final adoption decision.

Show every animal

Showing 5 of 13 animals, best match first.



1. Archie — 100% Strong

Dog · Cocker spaniel · 3 yr · Medium

5 requirements met, 0 not met

✓ Time alone: you said 4 h a day; Archie can be left for up to 4 h.

✓ Children: there are children in your home and Archie is suitable for homes with children.

✓ Space: you have 'Small or medium house'; Archie needs at least 'Small or medium house'.

✓ Activity: you can offer level 2 of 3; Archie needs level 2 of 3.

✓ Garden: Archie needs a garden; you have one.

Matches your species or size preference, so it is shown higher.

View profile



2. Oscar — 100% Strong

Dog · Jack Russell terrier cross · 8 yr · Small

5 requirements met, 0 not met

Matches your species or size preference, so it is shown higher.

View profile



3. Luna — 92% Strong

Dog · Labrador cross · 4 yr · Large

4 requirements met, 1 not met

Matches your species or size preference, so it is shown higher.

View profile



4. Bella — 100% Strong

Cat · British shorthair cross · 7 yr · Medium

5 requirements met, 0 not met

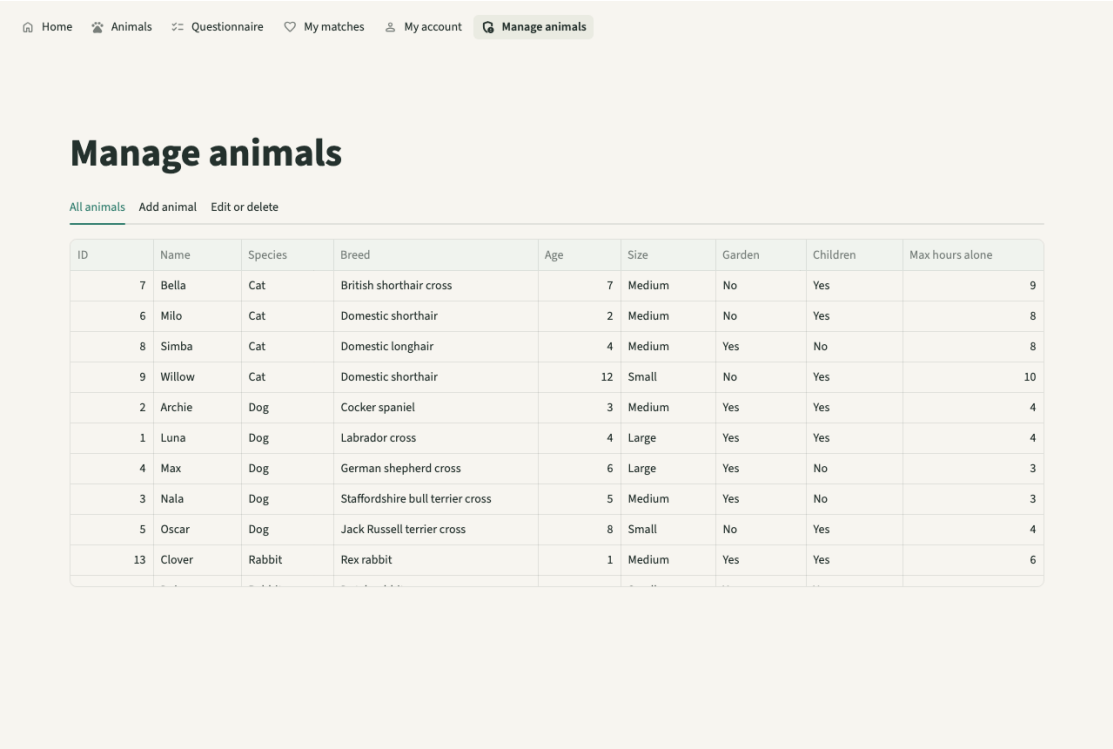
View profile



5. Clover — 100% Strong

Rabbit · Rex rabbit · 1 yr · Medium

Screen 10: My matches: score, category, met / not met and the guidance note (7.2-7.6).



Screen 13: Staff area for a staff account: list, add, edit or delete (8.1-8.4).

Testing

Test no.	Description	Input	Expected	Actual	Pass / Fail	Evidence (figure)
B1	A card for every animal	13 animals	13 cards	As expected	Pass	test_browse_shows_a_card_for_every_animal; Screen 02
B2	Filter rabbits	Species = Rabbit	4 rabbits	As expected	Pass	Screen 03
B6	Profile from database	Luna's id	Luna's details	As expected	Pass	test_profile_reads_from_database; Screen 04
D2	Blank answer blocks Next	step 1, one blank	Error, same step	First run: test crashed; later: 1 box not 2	Fail, then Pass	Ev 3a, 3b; Screen 07
D7	Questionnaire needs login	no session	'Please log in'	As expected	Pass	test_questionnaire_needs_login
E10	At least 4 results	submitted answers	5 cards, best first	As expected	Pass	test_results_show_at_least_four_ranked_matches; Screen 10
C8	Staff page not routable for adopters	adopter opens it by name	No such page	As expected	Pass	test_admin_page_is_not_routable_for_adopters

Failed test and remedial action

Test no.	What happened	Cause	Fix made	Re-test result
All 9 page tests	Every page test failed at once: 'AppTest script not found at .../tests/streamlit_app.py	AppTest.from_file resolves a relative path from the test file's folder, not the folder pytest	Gave the entry script as an absolute path built from the test file's location (ENTRY), so the	Pass: 9 of 9 (later 117 in all)

	'.	runs from.	tests run from any folder.	
D2	After errors were merged into one box, test D2 failed: 'assert 1 == 2' (it counted two error boxes).	The test checked the old design (one box per error), not the behaviour that matters.	Changed the test to expect one box that contains both messages; the behaviour is what users see.	Pass: 117 passed (Ev 3)

Ev 3a, the first failure (docs/evidence/stage3-fail1-apptest-path.txt):

```
E           FileNotFoundError: AppTest script not found at rebuild/tests/streamlit_app.py. Relative paths
are resolved against the file that calls AppTest.from_file().
E           FileNotFoundError: AppTest script not found at rebuild/tests/streamlit_app.py. Relative paths
are resolved against the file that calls AppTest.from_file().
E           FileNotFoundError: AppTest script not found at rebuild/tests/streamlit_app.py. Relative paths
are resolved against the file that calls AppTest.from_file().
```

Ev 3b, the second failure (docs/evidence/stage3-fail2-error-box.txt):

```
E        +  where 1 = len(ElementList(_list=[Error()]))
E        +    where ElementList(_list=[Error()]) = AppTest(\n    _script_path='rebuild/streamlit_app.py',
\n...SpecialBlock(\n            type='sidebar'\n        ),\n        2: SpecialBlock(\n
type='event'\n        )\n    }\n).error
1 failed in 0.67s
```

Review

Developer review: each page compared with its interface design and its criteria, in a browser at laptop width. Found: one red box per error made long forms noisy; after logging in, a staff user was sent to the staff page before the menu had registered it; the first test for 'adopters cannot open the staff page' caught any error, so it would have passed for the wrong reason.

Changes made after review

All errors for a form now show in one box (usability, S6), which caused the second failed test above. After login every user goes to Home and staff find 'Manage animals' in the menu. The staff-page test now expects the exact 'Could not find a navigation page' error, and a second test checks that the page's own guard stops an adopter who reaches it directly.

Stage 4: End-to-end run, screenshots and documentation

Aim

Runs the whole system as a user would, in a real browser driven by a script (docs/tools/capture_screens.py), at laptop (1280 px) and tablet (820 px) widths. It produces the 16 screenshots used as evidence, plus the README, a short write-up and a comparison page. Commit 213e208; checks criteria 1-9 together.

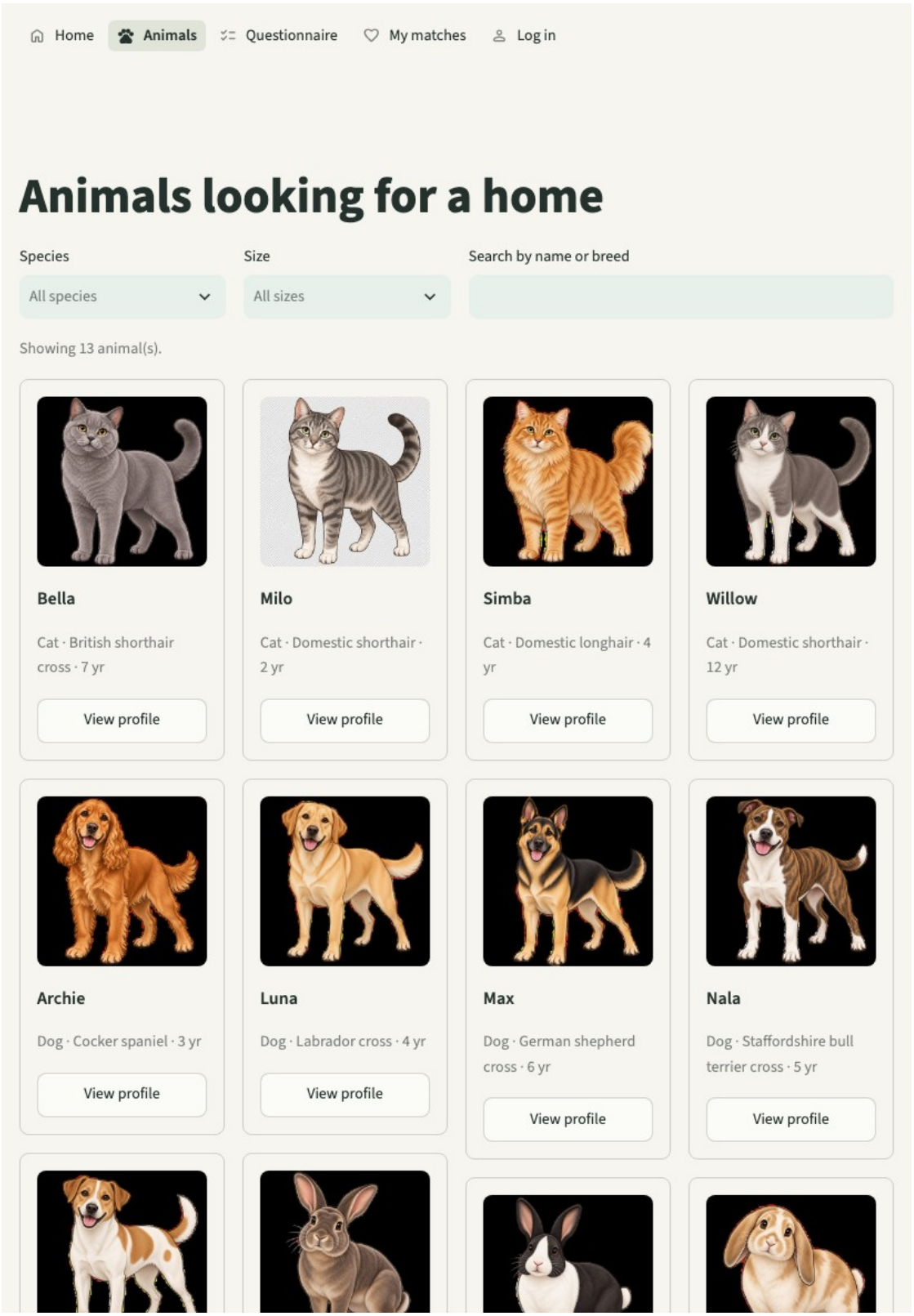
Code

Browser script: one screenshot step (docs/tools/capture_screens.py). The page height is measured first, so each screenshot shows the whole page.

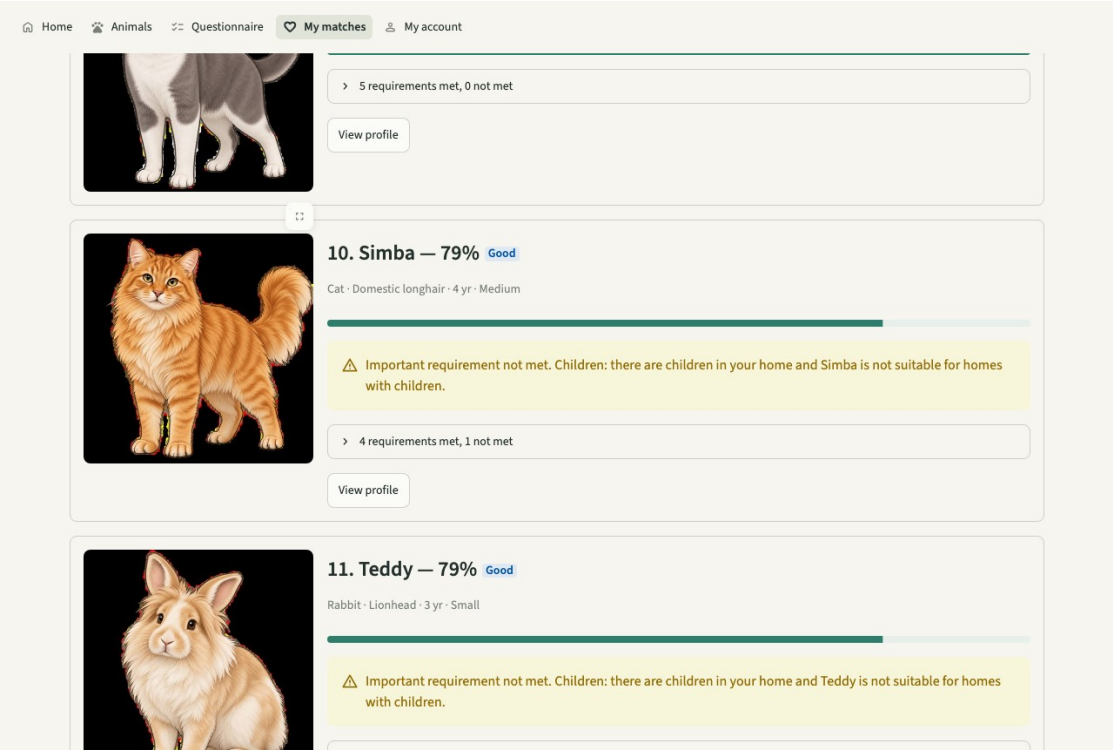
```
def shot(page: Page, out: Path, name: str, full: bool = True) -> None:
    """Streamlit scrolls an inner container, so grow the viewport to the content for a full shot."""
    settle(page)
    if full:
        height = page.evaluate("() => document.querySelector('[data-testid=stMain]').scrollHeight")
        page.set_viewport_size({"width": 1280, "height": min(max(int(height), 860), 3000)})
        page.wait_for_timeout(300)
    page.screenshot(path=str(out / f"{name}.png"))
    page.set_viewport_size({"width": 1280, "height": 860})
    print("saved", name, flush=True)


def choose(page: Page, label: str, option: str) -> None:
```

```
    page.get_by_label(label, exact=True).click()
    page.get_by_role("option", name=option, exact=True).click()
    settle(page)
```



Screen 16: The animal list at tablet width (820 px): cards rearrange, nothing is cut off (1.2).



Screen 11: A match with an important requirement not met: orange warning (7.5).

Testing

Test no.	Description	Input	Expected	Actual	Pass / Fail	Evidence (figure)
T2	Tablet width	820 px wide	No sideways scrolling	As expected	Pass	Screen 16
F4-F5	Register with errors, then log in	invalid then valid details	Errors, then logged in	As expected	Pass	Screens 05, 12
F6-F9	Questionnaire to results	full answers	Ranked results with warnings	As expected	Pass	Screens 06-11
F10-F12	Staff list, add with errors, edit	staff account	As planned	As expected	Pass	Screens 13-15
E2E-1	Fill the Age box	label 'Age'	Box filled	Script stopped: 2 elements matched	Fail, then Pass	capture_screens.py
E2E-2	Open My matches after login	go to its address	Results page	Logged out	Fail, then Pass	capture_screens.py

Failed test and remedial action

Test no.	What happened	Cause	Fix made	Re-test result
E2E-1	The browser script stopped: the label 'Age' matched two elements (the box and its 'Help for Age' button).	Finding by label matches any label that contains the word unless told to match exactly.	Used an exact label match (exact=True).	Pass: all 16 screenshots produced
E2E-2	After logging in, opening a page by its address showed the login page again.	Typing an address reloads the page, which starts a new Streamlit session, so the login is	The script now moves between pages with the menu, as a user would. The behaviour itself is a	Pass: all 16 screenshots produced

		lost.	real limitation, recorded in the evaluation (5.3, 5.4).	

Review

Developer review of the 16 screenshots against criteria 1-9 (no client was available to review them). Everything needed was visible, at both widths. Found: a full page reload logs the user out (E2E-2), and nothing on screen says so.

Changes made after review

Not changed in code: fixing it needs a signed browser cookie, which Streamlit does not provide; recorded as a limitation with a planned fix (5.5) instead of a rushed change. The README explains how to run and test the system so someone else can maintain it.

Stage 5: Design re-check and fixes

Aim

Re-reads every validation list and table design in the Design and compares each with the code, writing a test first for anything missing. Commit fdef5a0; components A, C and E.

Code

Name and breed rules (pawmatch/validation.py). One function serves people and animals; animals may also use digits. Every name must contain a letter.

```
def _check_name(value: Any, errors: Errors, field: str = "name", allow_digits: bool = False) -> str:
    """Person names: letters, spaces, hyphens, apostrophes. Animal names may also use digits
("R2")."""
    name = _text(value)
    allowed = " '-" + ("0123456789" if allow_digits else "")
    if not name:
        errors[field] = "Enter a name."
    elif not NAME_MIN <= len(name) <= NAME_MAX:
        errors[field] = f"Name must be {NAME_MIN} to {NAME_MAX} characters."
    elif not _has_letter(name) or any(not (ch.isalpha() or ch in allowed) for ch in name):
        errors[field] = "Name can only use letters, spaces, hyphens and apostrophes" + (
            " (and numbers)." if allow_digits else ".")
    return name

    breed = _text(raw.get("breed"))
    if not breed:
        errors["breed"] = "Enter a breed (use 'Mixed' if unknown)."
    elif not NAME_MIN <= len(breed) <= NAME_MAX:
        errors["breed"] = f"Breed must be {NAME_MIN} to {NAME_MAX} characters."
    elif not _has_letter(breed):
        errors["breed"] = "Breed must contain letters."
    cleaned["breed"] = breed
```

Upgrading an older database (pawmatch/db.py). CREATE TABLE IF NOT EXISTS leaves old tables alone, so the new column is added by hand when it is missing.

```
def init_schema(conn: sqlite3.Connection) -> None:
    conn.executescript(SCHEMA)
    # Databases made before stage 5 lack calculated_at (Design, Match Results table): add it in
place.
    columns = {row[1] for row in conn.execute("PRAGMA table_info(match_results)")}
    if "calculated_at" not in columns:
        conn.execute("ALTER TABLE match_results ADD COLUMN calculated_at TEXT NOT NULL DEFAULT ''")
```

Prototype evidence. Ev 5b: the stage 5 run after the fixes (docs/evidence/stage5-pytest.txt):

```
.................................................................... [ 53%]
..................................................................   [100%]
134 passed in 3.85s
```

Testing

Test no.	Description	Input	Expected	Actual	Pass / Fail	Evidence (figure)
A11	Name format	Mary-Jane, O'Neill / 12345, Sam3, Sam!	Letters only	First: 3 accepted	Fail, then Pass	Ev 5a, 5b
C11	Breed rules	Mixed / X, ###	X, ### rejected	First: accepted	Fail, then Pass	Ev 5a, 5b
C11b	Animal name format	R2 / 123, Luna!	123, Luna! rejected	First: accepted	Fail, then Pass	Ev 5a, 5b
E11	calculated_at stored	submission	Timestamp	First: no column	Fail, then Pass	Ev 5a, 5b
E11b	Old database upgraded	database without the column	Column added	First: missing	Fail, then Pass	Ev 5a, 5b
A7	Older duplicate-email test	'Sam 2', SAM@example.com	'already exists'	KeyError 'email'	Fail, then Pass	Ev 5c

Failed test and remedial action

Test no.	What happened	Cause	Fix made	Re-test result
A11, C11, E11 (9 cases)	9 new tests failed on their first run, as expected for tests written before the fix.	Design rules missing from the code (see stages 1 and 2).	Added the rules and the column.	Pass
A7	After the fix, an older test failed with KeyError 'email'.	Its second user was called 'Sam 2'. The new name rule rejected that before the duplicate-email check ran, so the test no longer tested what its name says.	Changed the test's name to 'Sam Two' so it again tests the duplicate email; the rule itself is right.	Pass: 134 passed (Ev 5b)

Ev 5a, the tests written first, before the fix (docs/evidence/stage5-fail-design-recheck.txt):

```
=========================== short test summary info ============================
FAILED tests/test_repository.py::test_saved_results_record_when_they_were_calculated
FAILED tests/test_repository.py::test_old_database_gains_calculated_at_column
FAILED tests/test_validation.py::test_registration_name_is_letters_spaces_and_hyphens[12345-False]
FAILED tests/test_validation.py::test_registration_name_is_letters_spaces_and_hyphens[Sam3-False]
FAILED tests/test_validation.py::test_registration_name_is_letters_spaces_and_hyphens[Sam!-False]
FAILED tests/test_validation.py::test_animal_name_format[123-False] - Asserti...
FAILED tests/test_validation.py::test_animal_name_format[Luna!-False] - Asser...
FAILED tests/test_validation.py::test_breed_length_and_not_only_symbols[X-False]
FAILED tests/test_validation.py::test_breed_length_and_not_only_symbols[###-False]
9 failed, 125 passed in 5.32s
```

Ev 5c, the older test that the new rule broke (docs/evidence/stage5-fail-regression-auth.txt):

```
=========================== short test summary info ============================
FAILED tests/test_auth.py::test_register_rejects_duplicate_email_case_insensitively
1 failed, 9 passed in 0.53s
```

Review

Developer review: listed every rule in the Design's validation sections and variables tables and ticked each against a test. After the fixes every listed rule has a test. Two Design points were kept as decisions rather than changes: login is by email (the Design mentions both), and passwords need at least 8 characters (the Design says both 'minimum of eight' and 'more than 8').

Changes made after review

None beyond the fixes: the full suite was run again (134 passed) and the screenshots still match, because the screens did not change.

4 Testing to inform evaluation

Model version addition

All tests below were run on the finished system (commit fdef5a0). Function tests (F) follow the main paths; robustness tests (R) try wrong, extreme or hostile input; T tests are measured. 'Actual' is what happened. Screens are in docs/img and the automated tests are in tests/.

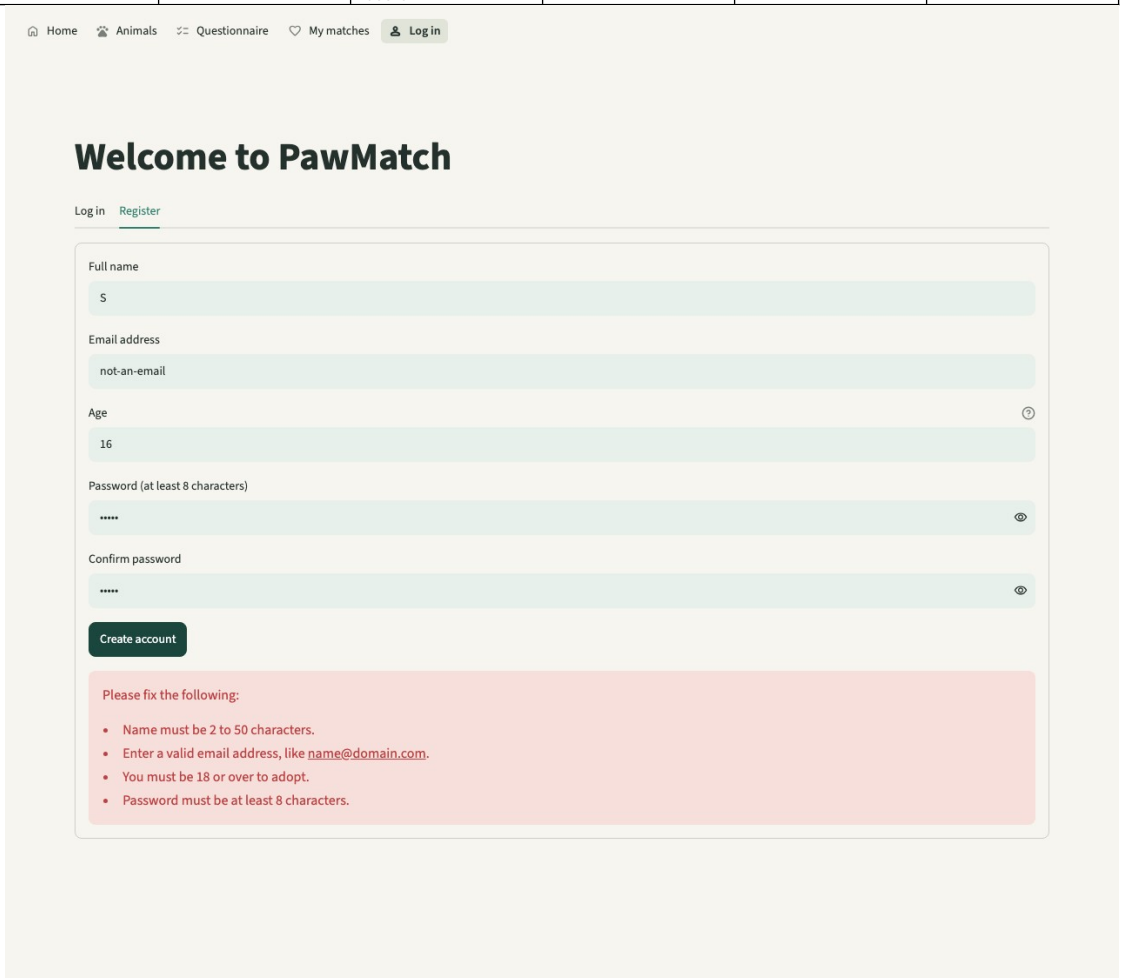
Measured results. Scoring and storing 500 animals: mean 0.01 s over 5 runs, slowest 0.011 s (target under 2 s). Listing 500 animals from the database: 0.003 s. Rendering the animal page with 500 animals: 0.4 s at worst over 3 headless runs, with 500 cards. Capacity: 500 animals and 100 users stored and read back. Contrast: text 12.4 : 1, button text 4.9 : 1.

```
............................................................          [100%]
134 passed in 3.85s
```

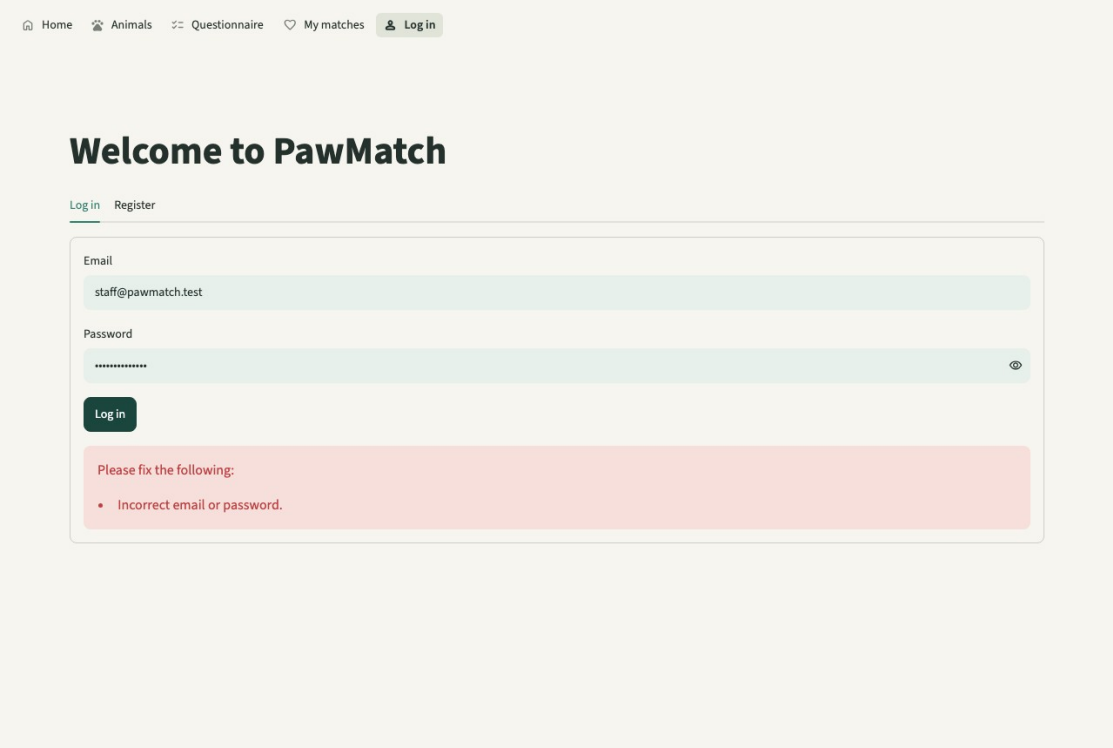
4.1 Function and robustness

Test no.	Success criterion	Input	Expected	Actual	Evidence (figure)
F1	2.1, 2.2	Open Home; click the button	Purpose shown; questionnaire opens	As expected	Screen 01
F2	3.1-3.3	Open Animals; filter Rabbit	13 cards; 4 rabbits	As expected	Screens 02, 03
F3	3.4, 4.1-4.4	View Luna's profile, then Back	All details; back to list	As expected	Screen 04
F4	5.1, 5.2	Register with errors, then correctly	All errors in one box; then account made	As expected	Screen 05
F5	5.3	Log in, then log out	Menu shows My account; then Log in	As expected	auth tests
F6	6.1-6.3	Answer steps 1-4	Progress bar, drop-downs, review	As expected	Screens 06, 08, 09
F7	7.1, 7.2, 9.3	Submit the answers	Every animal scored and stored	As expected	test_adopter_submission_stores_ranked_results
F8	7.3, 7.4, 7.6	Open My matches	5 shown, best first, met / not met, guidance note	As expected	Screen 10
F9	7.5	Answers that fail hours alone	Orange warning	As expected	Screen 11
F10	5.4, 8.1	Log in as staff	'Manage animals' in the menu	As expected	Screen 13
F11	8.2, 9.2	Add an animal	Stored and listed	As expected	test_admin_can_add_edit_and_delete
F12	8.3, 4.3, 9.4	Edit the breed	New breed on the profile	As expected	Screen 15
F13	8.4	Tick confirm, Delete	Animal and its results removed	As expected	test_delete_removes_animal_and_its_stored_results
R1	5.2	Age 17, 2.5, abc; email two@@x.com	Each rejected with its message	As expected	test_registration_age_boundaries
R2	5.5	Wrong password; unknown email	Same message; no session	As expected	Screen 12
R3	6.4	Next with a blank answer	Stays on step, error shown	As expected	Screen 07
R4	6.4	Submit 8 of 9 answers	Nothing stored	As expected	test_incomplete_submission_saves_nothing
R5	8.1, 8.5	Adopter asks for the staff page by name	No such page	As expected	test_admin_page_is_not_routable_for_adopters

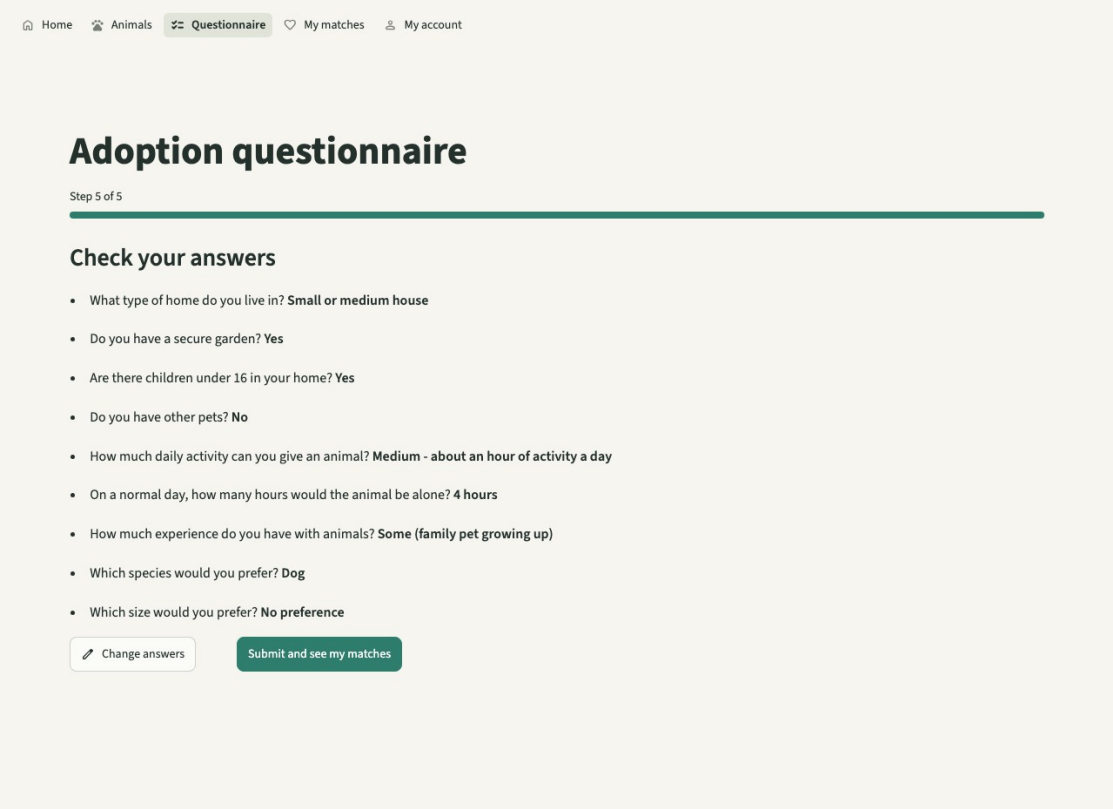
R6	8.2	Add with blanks and age 31	Every error listed; nothing stored	As expected	Screen 14
R7	8.3	Edit an animal deleted in another tab	'That animal no longer exists.'	As expected	test_edit_of_deleted_animal_reports_error
R8	8.5	Adopter reaches the staff page directly	'This page is for shelter staff only.'	As expected	test_admin_page_guard_stops_adopters_even_if_reached_directly
R9	9.1	Search ' OR 1=1 --	No animals, no error	As expected	test_search_text_is_treated_as_data_not_sql
R10	8.2	Upload a .gif; a 2 MB + 1 byte file	Rejected	As expected	test_image_type_and_size
R11	9.1	Insert species 'Hamster' directly into SQLite	Database refuses it	As expected	test_database_constraints_back_up_validation
R12	5.3	Reload the browser page while logged in	Still logged in	Logged out	Capture run (E2E-2)
T4	7.2	500 animals, 5 submissions	Under 2 s	Mean 0.01 s	Measured above
T5	9.1	500 animals, 100 users	All stored and listed	500 / 100	Measured above



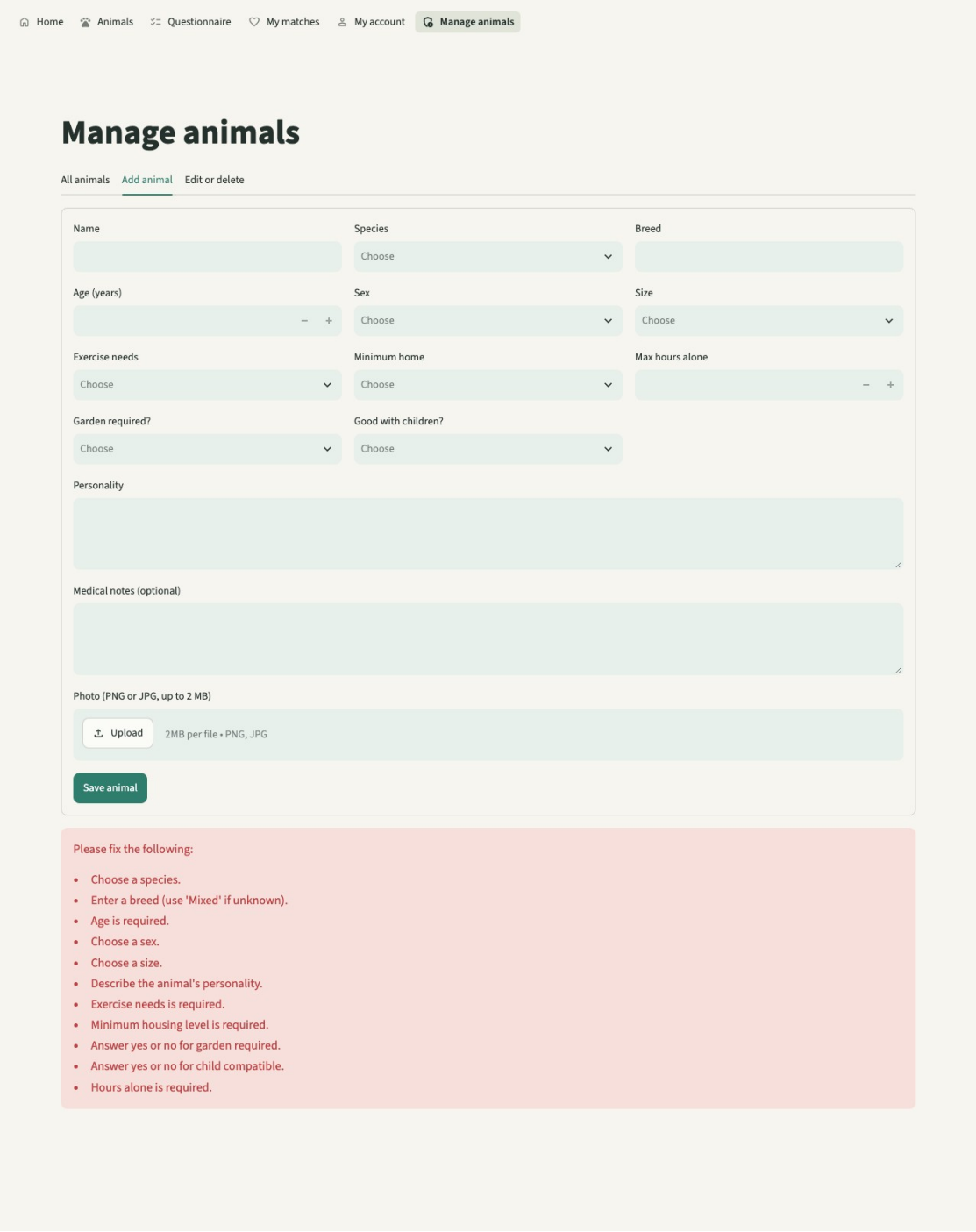
Screen 05 (F4, R1): registration with several mistakes. Every problem is listed in one box and the typed values stay.



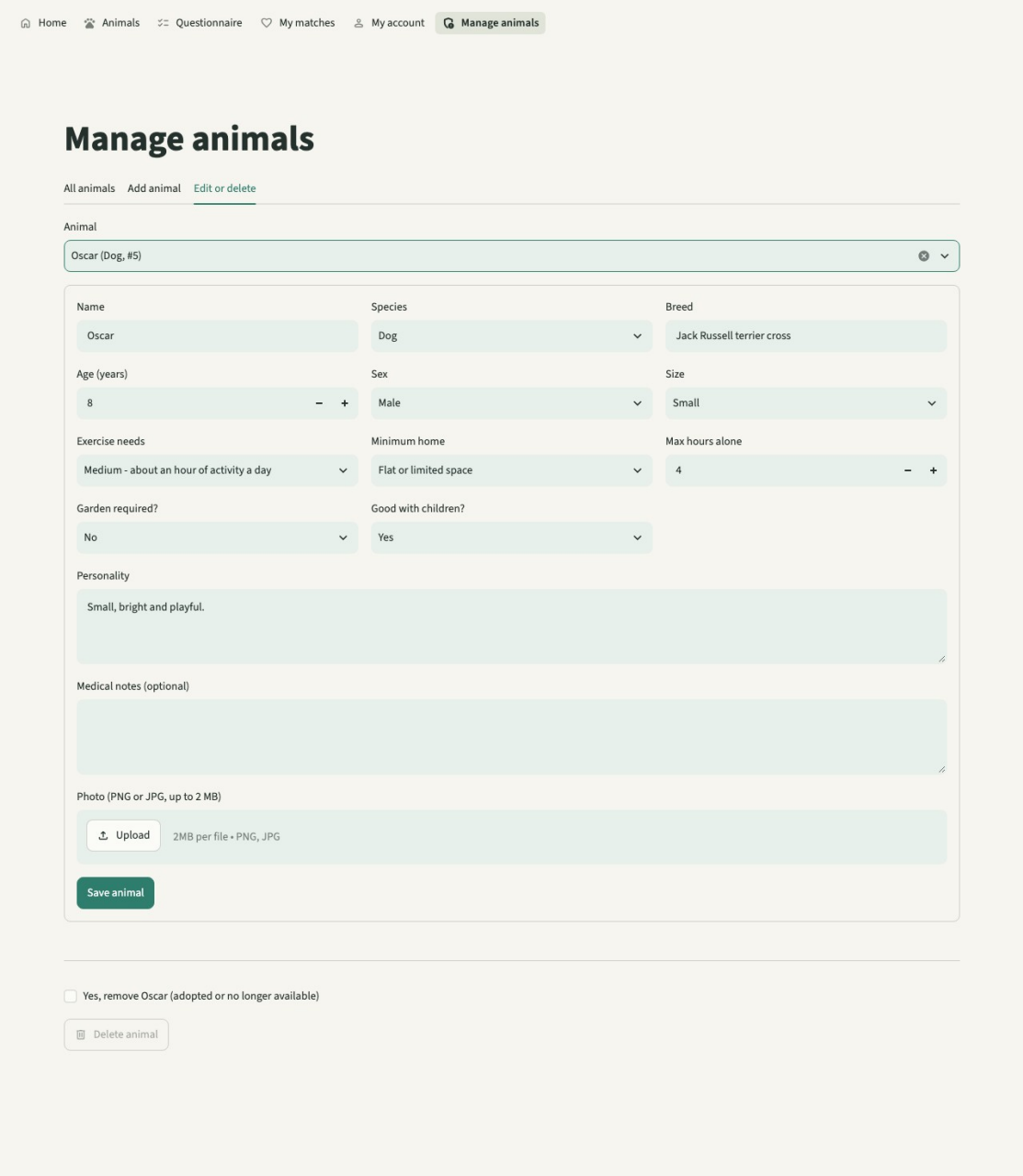
Screen 12 (R2): a wrong password gives the same message as an unknown email.



Screen 09 (F6): the review step lists every answer before submitting.



Screen 14 (R6): the staff form refuses an incomplete animal and lists each field to fix.



Screen 15 (F12, F13): editing an animal; Delete stays disabled until the confirmation is ticked.

4.2 Usability testing

Tester	Task given	What happened	Rating /10	Comment
Developer (not a stakeholder)	U1: register and log in	Done in 4 screens; errors clear (Screen 05)	Not collected	Needs a real adopter
Developer (not a stakeholder)	U2: complete the questionnaire	4 steps + review; blank answer blocked (Screens 06-09)	Not collected	Needs a real adopter
Developer (not a stakeholder)	U3: explain the top match	Met / not met list and warning are readable	Not collected	Needs a real adopter

		(Screens 10, 11)		
Developer (not a stakeholder)	U4: add, edit, delete an animal	All three done; delete needs the tick (Screens 13-15)	Not collected	Needs the shelter client
Developer (not a stakeholder)	U5: use it at tablet width	No sideways scrolling at 820 px (Screen 16)	Not collected	Real tablet not tried

What this shows and what it does not. The walkthrough was done by the developer in a headless browser using the planned usability tasks. It shows every task can be completed and the usability features are present. It is not usability testing with stakeholders: no adopter or member of shelter staff tried the system, so there are no ratings. This is why criterion 6.2 is only partly met; running U1-U5 with five adopters and the client is the first item in Future development.

5 Evaluation

Model version addition

Each requirement is cross-referenced to the tests in section 4 (F, R, T) and to the development tests. Of 38 requirements, 36 are met and 2 are partly met (6.2, 8.1); none are unmet. 'Met' is only used where a test or screenshot shows the measure from the Analysis was reached.

5.1 Success criteria

Criterion	Requirement	Evidence (test no.)	Met / Partly / Not met	Explanation
1 Navigation and clean interface	**4 requirements, below**		**Met**	
1.1	Pages must use a calm colour scheme, clear headings and identifiable buttons	T1; contrast 12.4 : 1 text, 4.9 : 1 buttons; Screens 01-15	Met	Contrast is well above 4.5 : 1. Headings and buttons are the theme's own; no page adds its own styling.
1.2	The interface must remain readable on laptop and tablet sized screens	T2; Screens 02 and 16	Met	Checked in a headless browser at both widths. Not checked on a real tablet or phone; see 5.3.
1.3	The system must include a navigation menu to access home page, animal preview page, profile page, and login page	T3; every page test opens pages through the one menu	Met	The menu is built once in streamlit_app.py, so no page can leave it out.
1.4	The menu and interface style must remain consistent across all the pages	T1; .streamlit/config.toml	Met	All styling comes from one 15-line theme file.
2 Home page	**2 requirements, below**		**Met**	
2.1	Should explain the purpose of the animal adoption matching system, especially including that the system focuses on suitability rather than only appearance	F1; test_home_page_explains_purpose; Screen 01	Met	The test reads the rendered page text.
2.2	Should have a clear button or navigation option that directs users to complete the adoption questionnaire	F1; Screen 01	Met	'Start the questionnaire' is the first button.
3 Animal preview page	**4 requirements, below**		**Met**	
3.1	The page should display	F2, T5;	Met	Every record in the table

	every available animal using compact animal cards	test_browse_shows_a_card_for_every_animal; 500 cards for 500 animals		appears. There is no 'adopted' status, so every stored animal counts as available.
3.2	Each card should show the species, name, breed, age and image	F2; Screen 02	Met	The card is one function, so every card is the same.
3.3	The system must support at least 3 animal species including dogs, cat and rabbits	F2; test_seed_has_three_species_and_is_idempotent; Screen 03	Met	Dogs, cats and rabbits; the database CHECK only allows these three.
3.4	Users must be able to open a full animal profile from the preview page	F3; test_profile_reads_from_database; Screen 04	Met	'View profile' on every card.
4 Animal profile page	**4 requirements, below**		**Met**	
4.1	Each profile must display the animals name, species, breed, age and sex	F3; Screen 04	Met	Shown in a table at the top of the profile.
4.2	Each profile also must include personality, medical needs, exercise needs and suitability requirements	F3; Screen 04	Met	Suitability is listed as plain sentences (garden, children, hours alone, home size).
4.3	Animal profile information must be retrieved from the database	F3, F12; round-trip tests	Met	The page reads the id from the address and queries the database each time.
4.4	Users must be able to return to the animal preview page or results page	F3; Screen 04	Met	The back link remembers whether the user came from the list or the results.
5 User and admin registration and login	**5 requirements, below**		**Met**	
5.1	Users must be able to create an account, using name, email and password and it can include other features such as age to ensure they are at a suitable age	F4; test_register_stores_hash_not_password	Met	Age is also collected so the 18+ rule can be checked.
5.2	The system must reject registration if required fields are empty and the email format is invalid, by validating inputs	F4, R1; test_registration_*; Screen 05	Met	Every rule in the Design's validation list has a test, including the name format added in stage 5.
5.3	Registered users must be able to login and logout using their correct account details	F5; test_login_success_and_email_is_normalised	Met	Limitation: reloading the browser page also logs the user out (5.3).
5.4	The admin login details should allow admins to access their own section with the CRUD interface	F10; test_admin_page_lists_animals_for_staff	Met	Staff use the same login form; the menu changes by role.
5.5	Incorrect Login details must display an error message and prevent access to the account	R2; test_wrong_password_and_unknown_email_give_the_same_message; Screen 12	Met	The same message means nobody can find out which emails have accounts.
6 Adopter questionnaire page	**4 requirements, below**		**Partly met**	
6.1	The questionnaire must collect information about the user's home, lifestyle, garden access, children, other pets, time available and previous animal experience	F6; test_steps_cover_every_question_once; Screens 06-09	Met	Other pets and experience are stored but not scored, because the Design gives them no weight.

6.2	The questionnaire must be split into clear sections or individual questions rather than appearing as one long form	F6; Screens 06-09; tester ratings not collected	Partly met	The sections are built and tested, but the planned tester rating was not carried out (5.3).
6.3	The questionnaire must use suitable input methods such as dropdowns, radio buttons or selection boxes but preferably dropdowns.	F6; Screens 06, 08	Met	Every answer is a drop-down.
6.4	The system must prevent the questionnaire from being submitted if important questions are unanswered	R3, R4; test_questionnaire_blocks_unanswered_step; test_incomplete_submission_saves_nothing; Screen 07	Met	Checked on each step and again for the whole set before saving.
7 Results and recommendations page	**6 requirements, below**		**Met**	
7.1	The system must compare adopter's questionnaire with each animal's requirements	F7; test_adopter_submission_stores_ranked_results	Met	Nothing is filtered out before scoring.
7.2	The system must calculate a numerical compatibility score for each animal	F7, T4; test_matching.py (29 cases); mean 0.01 s, slowest 0.011 s	Met	The formula is in the Design (worked example) and each factor has its own test.
7.3	The top 4 animals must be displayed from highest compatibility score to lowest at a minimum	F8; test_results_show_at_least_four_ranked_matches; Screen 10	Met	Five are shown by default and a toggle shows every animal. Animals with a welfare warning come after those without.
7.4	The results page must show which key requirements the user meets and does not meet for each animal	F8; Screens 10, 11	Met	In the 'requirements met / not met' list on each card.
7.5	The system must display a warning if the user does not meet important requirements for an animal	F9; test_important_unmet_requirements_raise_warnings; Screen 11	Met	Garden and activity lose points but do not warn.
7.6	The results page must explain that the compatibility scores are guidance, and the adoption staff have the final say	F8; Screen 10	Met	Shown above the results and on Home.
8 Admin management page	**5 requirements, below**		**Partly met**	
8.1	This page should be accessed separately with a separate admin login	F10, R5; test_admin_page_is_not_routable_for_adopters	Partly met	Only staff accounts reach it, but staff use the same login form as adopters, not a separate one.
8.2	Admin users must be able to add a new animal record with all the information required	F11, R6; test_admin_can_add_edit_and_delete; Screen 14	Met	Every field is validated and the database CHECKs repeat the rules.
8.3	Admin users must be able to edit existing animal records and save the updated information to the database	F12; test_update_changes_the_stored_record; Screen 15	Met	Editing an animal that was deleted meanwhile gives a clear error (R7).
8.4	Admin users must be able to delete animal records when animals adopted or no longer available	F13; test_delete_removes_animal_and_its_stored_results	Met	The record is deleted rather than marked 'adopted' (5.4).
8.5	Normal users must not	R5, R8;	Met	Checked three times:

	be able to access the admin management page	test_non_admins_are_refused_every_change; test_admin_page_guard_stops_adopters_even_if_reached_directly		menu, page guard, and inside each service.
9 Database	**4 requirements, below**		**Met**	
9.1	The database must store animal records, user account details, admin login details and questionnaire answers	T5; 500 animals and 100 users stored and read back	Met	Four tables with keys and CHECK constraints.
9.2	The database must allow animal records to be added, viewed, edited and deleted through the admin system	F11-F13	Met	Through the staff page and the services.
9.3	Questionnaire answers must be stored or processed so that they can be used by the compatibility scoring algorithm	F7; test_responses_and_results_are_stored_and_reloaded_in_rank_order	Met	Each submission is a new response row; results point to it and record when they were calculated.
9.4	Data must be retrieved correctly so animal profiles and recommendations display accurate information	F12; test_insert_then_get_round_trips_every_field	Met	Checked for every animal field.

5.2 Usability features

Usability feature	Evidence	Worked?	How to improve
Calm theme, one file	Screens 01-16; contrast 12.4 : 1	Yes	Ask testers about the colours (U1-U5)
Same menu on every page, by role	T3; R5; Screens 10, 13	Yes	Show who is logged in next to the menu
Questionnaire in steps with progress bar	F6; Screens 06-09	Yes, but not rated by users	Run U2 with five adopters
Drop-downs only	F6; R3	Yes: no invalid answer can be typed	Radio buttons for short lists may be quicker on tablets
All errors in one box	F4, R6; Screens 05, 14	Yes	Also mark the field itself red
Animal cards	F2; Screens 02, 16	Yes, also at tablet width	Add 'good with children' icons on the card
Score bar, category, met / not met	F8; Screen 10	Yes	Open the met / not met list for the top result by default
Warnings for important needs	F9; Screen 11	Yes	Link the warning to the profile section it refers to
Guidance note	Screens 01, 10	Yes	None needed
Delete confirmation	F13; Screen 15	Yes	Offer 'mark as adopted' instead of deleting (5.5)
Back links	F3; test B8	Yes	None needed

5.3 Partly met and unmet criteria

Criterion	Why it is not fully met	How further development would address it
6.2 Clear sections (rated 8/10 by testers)	The sections are built and tested, but the rating needs real users and none were available for this model version.	Run U2 with five adopters from the survey group; change wording where the score is below 8.
8.1 Separate admin login	Staff log in on the same form as adopters; the system then gives them the staff page. Access is safe (R5, R8) but the login itself is not separate.	Add a 'Staff login' page that only accepts admin accounts and remove staff access from the normal form.

Also noted although the criterion is met: 5.3 (log in and out) works, but a full page reload logs the user out (R12). 1.2 was checked at tablet width in a browser, not on a real tablet or phone.

5.4 Maintenance and limitations

Maintenance. The code is split so each change has one place to go: weights and cut-offs at the top of matching.py; every validation rule in validation.py; all SQL in db.py; colours in one 15-line theme file. Each function has a comment saying what it does and why. 134 automated tests run in about 5 seconds, so a maintainer can change a rule and know at once whether anything broke; this is how the stage 5 regression was caught. The README explains how to run, test and add a staff account, and the database upgrades itself when a column is added (stage 5).

Limitations.

- One shelter only, and three species, as the Analysis planned.
- A page reload logs the user out (Streamlit keeps the login in the browser session).
- No animal status: removing an adopted animal deletes the record and the results that mention it.
- Other pets and experience are asked and stored but not scored, because animals have no matching fields yet.
- No password reset and no limit on login attempts.
- Not tested with real users or on a real tablet or phone.
- Animal photos are stored as files on the same computer; a hosted version would need shared storage.

5.5 Future development

Improvement	How
Stay logged in after a page reload	Store a signed, expiring token in a browser cookie and check it in CurrentUser(); Streamlit has no built-in sign-in cookie, so this needs a small extra package or a reverse proxy.
Animal status (Available / Reserved / Adopted)	Add a status column; browse and matching only use Available animals; staff change status instead of deleting, so match history and records are kept.
Separate staff entrance	A 'Staff login' page that only accepts admin accounts, to meet requirement 8.1 to the letter.
Score other pets and experience	Add 'good with other animals' and 'suits first-time owners' to each animal, then two more weighted factors; the weights table and tests change, nothing else does.
Password reset and login limits	Email a one-time link (needs an email service); lock an account for a few minutes after 5 wrong passwords.
More than one shelter	A shelters table and a shelter_id on animals and staff, so each shelter manages only its own animals.
User testing	Run the planned usability tests U1-U5 with five adopters and the shelter client and record their ratings.

Appendix A AI use log

This whole model version was produced with AI, as listed below. A record of every change is the git history of the opus-rebuild branch.

Date	Tool	What I asked	What I used	Where in the project	Screenshot
23 Oct 2026	Claude Opus 5.5 (Anthropic)	Rebuild the app from Sienna's analysis and design	All code and tests	rebuild/	Git history
23 Oct 2026	Claude Opus 5.5	Fill every gap in the NEA document	All model additions	Sections 1-5	This document
23 Oct 2026	Claude Opus 5.5 (script)	Draw the diagrams	Diagrams D1-D8	Design	docs/tools/ make_diagrams.py
23 Oct 2026	Claude Opus 5.5 (script)	Take the screenshots	Screens 01-16	Development, Testing	docs/tools/ capture_screens.py
23 Oct 2026	Claude Haiku 5.5	Summarise the document structure	Planning only	None	None

Appendix B References

Sienna's Analysis references (above) are kept unchanged.

Streamlit (2026) Streamlit documentation: st.navigation, st.Page and AppTest. Available at: https://docs.streamlit.io/ (Accessed: 23 October 2026).

Python Software Foundation (2026) hashlib: scrypt. Available at: https://docs.python.org/3/library/hashlib.html (Accessed: 23 October 2026).

SQLite (2026) SQLite foreign key support. Available at: https://www.sqlite.org/foreignkeys.html (Accessed: 23 October 2026).

OWASP (2026) Password storage cheat sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 23 October 2026).

W3C (2018) Web Content Accessibility Guidelines 2.1: contrast (minimum). Available at: https://www.w3.org/TR/WCAG21/#contrast-minimum (Accessed: 23 October 2026).

pytest (2026) pytest documentation. Available at: https://docs.pytest.org/ (Accessed: 23 October 2026).

Microsoft (2026) Playwright for Python. Available at: https://playwright.dev/python/ (Accessed: 23 October 2026).